
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 13.09.2026, 15:32:24
Файлов исходного кода: 92
Суммарный объём кода: 1.38 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts
25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/ColoringViz.tsx
37. src/components/ComplexityClassesViz.tsx
38. src/components/ComponentsViz.tsx
39. src/components/DfsBridgesSimulator.tsx
40. src/components/DijkstraViz.tsx
41. src/components/DPVisualizer.tsx
42. src/components/EulerSimulator.tsx
43. src/components/ExpressQuiz.tsx
44. src/components/FloydViz.tsx
45. src/components/GraphTraversalViz.tsx
46. src/components/HeapViz.tsx
47. src/components/hints/demoKit.tsx
48. src/components/hints/demosExtra.tsx
49. src/components/hints/demosGraph.tsx
50. src/components/hints/demosStruct.tsx
51. src/components/hints/MiniDemo.tsx
52. src/components/hints/SimulatorModal.tsx
53. src/components/hints/TermPopover.tsx
54. src/components/JohnsonViz.tsx
55. src/components/KosarajuViz.tsx
56. src/components/KruskalSimulator.tsx
57. src/components/MnemonicCards.tsx

```
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
```

CI / автоматизация

```
91. .github/workflows/deploy-pages.yml
92. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/92: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml `

Полное содержимое после изменений

```yaml

```
 VITE_BASE: /algv0/
 run: npm run build

- name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
```

```
+
+   - name: Upload Pages artifact
+     uses: actions/upload-pages-artifact@v5
+     with:
+       path: ./dist
+
+ deploy:
+   name: Publish to GitHub Pages
+   needs: build
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+
+
+ ## `./.github/workflows/rebuild-pdf.yml`
+
+ ### Полное содержимое после изменений
+
+ ```yaml
+ name: Rebuild code PDF
+
+ # Пайплайн пересборки PDF при каждом коммите:
+ #
+ # исходный код → full_code_all_pages.txt (script)
+ #               |
+ #               └─→ page-0001.png ... page-NNNN.png (script)
+ #                   |
+ #                   └─→ full_code_all_page.pdf (script)
+ #
+ # Готовые TXT и PDF коммитятся обратно в ветку (их отдаёт скрипт)
+ # промежуточные PNG сохраняются как artifacts запуска.
```

- ```
node-version: "24"
cache: npm
```
- name: Install ImageMagick + DejaVu fonts  
run: |  
 sudo apt-get update -qq  
 sudo apt-get install -y --no-install-recommen  
 # На Ubuntu политика ImageMagick иногда запре  
 # разрешаем то, что нужно пайплайну (text -> p  
 for policy in /etc/ImageMagick-6/policy.xml /  
 if [ -f "\$policy" ]; then  
 sudo sed -i 's/rights="none" pattern="\{P  
 fi  
done  
convert -version
  - name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"

### Patch относительно `main`

```diff

diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

```
-----
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
+    <link rel="icon" type="image/svg+xml" href="%BASE_
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen
  ....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

```
````
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
```

```
new file mode 100644
```

```
index 00000000..3bbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
```

```
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```

```
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
```

```
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
```

```
+</svg>
```

```
````
```

```
## `src/App.tsx`
```



```

        if (e.key === "ArrowRight" && next) goTo(next.id)
        if (e.key === "ArrowLeft" && prev) goTo(prev.id);
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />

          <div className="flex flex-col lg:flex-row max-w-7xl" style={{margin: 0 auto}}>
            {/* Сайдбар только на десктопе – на мобильном скрывается */}
            <div className="hidden lg:block lg:w-80 shrink-0" style={{marginRight: 10px}}>
              <Sidebar selectedChapterId={activeChapterId}>
            </div>

            <main id="main-content" className="flex-1 min-w-0">
              {/* Шапка главы с быстрыми переходами */}
              <div className="flex items-center justify-between">
                <div className="min-w-0">
                  <div className="text-[10px] uppercase">
                    {activeChapter.category || "Универсальное"}
                  </div>
                  <h2 className="text-base sm:text-xl font-medium">
                    {activeChapter.title}

```

```
    <SimulatorModal vizId={modalVizId} onClose={() =>
      <footer className="p-8 text-center text-slate-600"
        © 2026 Universal Educational Guide. Интерактивные
      </footer>
    </div>
  );
}
```

Patch относительно `main`

```
```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
 const [activeChapterId, setActiveChapterId] = useState<string>();
 const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
```

```

return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Симулятор
 </button>
 </div>
 </div>
)

```

## Универсальное пособие • полный исходный код

```

 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
</div>
</div>
</header>
);
};
....
```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
```

```
        rel="noreferrer"
    ....

## `src/components/PythonCompiler.tsx`

### Полное содержимое после изменений

```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL);
 }
}
```

```

 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не вывела");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код – подробности в консоли");
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >

```

```

 onKeyDown={({event}) => {
 if ((event.ctrlKey || event.metaKey) &&
 event.preventDefault();
 void runCode();
 }
 })
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] res
 -none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

<div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={({event}) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
</div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

```

```

@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, PL
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/py
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/py
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {

```



```

+ setRuntimeMessage("Не удалось выполнить код — пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:p
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5
+ >
+ Как это работает <ExternalLink className="
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1.
+ <section className="rounded-2xl border borde

```

```

+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-l
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emera
+ </div>
+ <span className={`inline-flex items-centr
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800

```

Универсальное пособие • полный исходный код

```

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algov0/, локаль
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
\\`\\`\\`
```

### Patch относительно `main`

```
\\`\\`\\`diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
```

**\*\*строго\*\*** придерживаться готовых шаблонов разметки,

## ## Правила для малых страниц (Билетов / Глав)

1. **\*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь объект `{ "id": "...", "content": ... }` для вставки в ``src/data/content.ts``.
3. **\*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``)
  - Обязательная сетка аналогий (2 колонки: неформальный текст и формальный текст)
  - "Душные" детали (код, формулы, сложные доказательства)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы и не используй классы в файлах ``src/data/content.ts``. Не используй чистый markdown.

---

ФАЙЛ 4/92: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Универсальное пособие по алгоритмам – билеты
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

---

ФАЙЛ 5/92: metadata.json

```
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "svg-to-pdfkit": "^0.1.8",
 "tailwind-merge": "3.4.0"
 },
 "devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.24.0",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и структурам данных"
}
```

---

ФАЙЛ 7/92: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 125 | РАЗМЕР: 1.2 КБ

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с симуляцией алгоритмов и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
```

Аудит контента

```
```bash
node scripts/audit-coverage.mjs # какие темы без
node scripts/audit-coverage.mjs --md # markdown-табли
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`] (./TOPICS_AUDIT.md)

Как устроен интерфейс

- * **Темы 1–24** — канонический список билетов; одна стр.
(`src/data/content.ts` , таблица `CANON` задаёт точные
- * **Python-компилятор** — боковая панель (кнопка «Python
отладчик. Список «переменных темы» для каждой страниц
`src/data/vizSync.ts` ; значения из выполняемого Python
объекты демонстрации (матрицы, графы, деревья) напрям
эталонный код, инициализация подставляет заранее зада
- * **Содержание** — на десктопе боковая панель с поиском
на телефоне — липкая строка «Содержание · N из M» и в
- * **Переходы между темами** — кнопки «Назад / Вперёд» в
«Предыдущая / Следующая тема» под текстом; на клавиатур
- * **Всплывающие подсказки по терминам** — текст главы а
известные термины (BFS, куча, бор, суффиксная ссылка,
подчёркиваются, а по наведению/тапу показывается карт
мини-анимацией и кнопкой «Показать симулятор» (симуля
Словарь — `src/data/glossary.ts` , мини-анимации — `src
- * **Реестр интерактивов** — `src/components/vizRegistry`
и для панели под главой, и для модалки из подсказки.

Структура

...

```
src/
  App.tsx          — оболочка, привязка глав к визу
  components/      — интерактивные симуляторы (Дейк
  data/            — контент пособия (главы/билеты
```

Универсальное пособие • полный исходный код

Экспорт перехватывает основные структурные компоненты HTML:

1. Заголовки `

` и `` заменяются на `##` и `###`
2. Элементы `

.analogy-badge` получают выделение маркера
3. Теги `` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `- ` получают маркер `-`.
- 5. Результат извлекается в виде текста и чистится от лишнего.

ФАЙЛ 8/92: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.mjs`

> Критерии:

> * **Аналогия** — в HTML главы есть блок «Аналогия ...»

> * **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|-------|----|------------|
|-------|----|------------|

| | | |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

| | | |
|--------------------|-----------|-----------------------|
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8) |
|--------------------|-----------|-----------------------|

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

```

-----
3. Декартово дерево (Trear)	`trear`	Нет ни визуализации
8. Графы. Компоненты связности	`graph-components`	
24. Классы сложности, сведение задач	`complexity-classes`	

```

Сводная таблица по всем главам

| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|----|---|----------|---------|-----|----------|
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | | | | |
| 3 | 3. Декартово дерево (Trear) | 2 | – | – | – |
| 4 | 4. Splay-дерево | 2 | | – | – |
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | – |
| – | Планарность: K5, K3,3 и формула Эйлера | – | | | – |
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | | – |
| 8 | 8. Графы. Компоненты связности | 1 | – | – | – |
| 9 | 9. Графы. Топологическая сортировка | 1 | | | – |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | – |
| 12 | 12. Графы. Точки сочленения | 1 | | | – |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | – |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | – |
| 16 | 16. Графы. Кратчайший путь. Флойд | – | | | – |
| 17 | 17. Графы. Алгоритм Джонсона | – | | | – |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | | | – |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | | | – |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | | | – |
| 21 | 21. Префикс-функция (π), КМП | 4 | | | – |
| 22 | 22. Z-функция | 4 | | | – |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | | | – |
| 24 | 24. Классы сложности, сведение задач | 4 | – | – | – |
| – | Обзор: Кратчайшие пути и Графы | – | | | – |
| – | Алгоритмы на карте | – | – | – | – |
| – | Эйлер: Путь ≠ Цикл | – | | | – |
| – | Мосты в графах: код + визуализация | 1 | | | – |


```
    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

ФАЙЛ 10/92: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algov0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
```

```

/** Боковая (на мобильном – нижняя) панель компилятора
const [compilerOpen, setCompilerOpen] = useState(false);
/** Ширина панели компилятора (desktop), растягивается
const [compilerWidth, setCompilerWidth] = useState<number>(
  const cached = typeof window !== "undefined" ? Number(window.innerWidth) : 0;
  return Number.isFinite(cached) && cached >= 380 ? cached : 380;
));
useEffect(() => {
  window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту
const [compilerHeight, setCompilerHeight] = useState<number>(
  const cached = typeof window !== "undefined" ? Number(window.innerHeight) : 0;
  return Number.isFinite(cached) && cached >= 260 ? cached : 260;
));
useEffect(() => {
  if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем сам компилятор)
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>(
  const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null;
  if (cached !== null) return cached === "1";
  return typeof window !== "undefined" && window.innerWidth < 380;
));
const toggleSidebar = useCallback(() => {
  setSidebarCollapsed((c) => {
    window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
    return !c;
  });
}, []);

// Desktopный отступ контента = ширине панели компилятора
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.innerWidth < 380 ? compilerWidth : 0;
  };
  apply();
}, [compilerOpen]);

```

```

window.addEventListener("keydown", onKey);
return () => window.removeEventListener("keydown",
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter

return (
  <div className="min-h-screen bg-slate-950 text-slate-
    <Navbar
      compilerOpen={compilerOpen}
      onToggleCompiler={() => setCompilerOpen((o) =>
    />

    {/* При открытой панели контент сдвигается: снизу
    <div ref={contentRef} className={compilerOpen ? "
      <
        <MobileToc
          selectedChapterId={activeChapterId}
          setSelectedChapterId={goTo}
          currentTitle={activeChapter.title}
          index={index}
          total={chapters.length}
        />

        <div className="flex flex-col lg:flex-row m
          {/* Сайдбар только на десктопе – на мобил
          <div
            className={`hidden lg:block shrink-0 bo
            tion-all duration-200 ${sidebarCollapsed ?
          >
            {!sidebarCollapsed && <Sidebar selected
          </div>
          {/* Вкладка-переключатель содержания у ле
          <button
            type="button"
            onClick={toggleSidebar}
            className="hidden lg:flex items-center
            l-0 border-slate-700 bg-slate-900/90 px-1 p

```

```
        />
      </main>
    </div>
  </>

  <footer className="p-8 text-center text-slate-600">
    © 2026 Universal Educational Guide. Интерактивное
  </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
  compilerOpen && (
    <PythonCompiler
      chapterId={activeChapter.id}
      chapterTitle={activeChapter.title}
      width={compilerWidth}
      height={compilerHeight}
      onWidthChange={setCompilerWidth}
      onHeightChange={setCompilerHeight}
      onOpenGuide={() => {
        document.getElementById("chapter-viz")?.scrollIntoView()
      }}
      onClose={() => setCompilerOpen(false)}
    />
  )}
</div>
);
}
```

ФАЙЛ 12/92: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 KB

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";
```

```

        const parent = (node as Text).parentElement;
        if (!parent) return NodeFilter.FILTER_REJECT;
        if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
        if (!node.nodeValue || node.nodeValue.trim().length === 0)
            return NodeFilter.FILTER_ACCEPT;
    },
    });

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
}

for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE)
            continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createTextNode(
            text.slice(last, m.index) +
            document.createElement("span").setAttribute("class", id) +
            text.slice(m.index, m.index + m[0].length)));
        last = m.index + m[0].length;
    }
    frag.appendChild(document.createTextNode(
        text.slice(last, text.length)));
    document.body.appendChild(frag);
}

```

```
// страховка: контент перерисовали, а подсказок в нём
useEffect(() => {
  const el = ref.current;
  if (!el) return;
  if (!el.querySelector(".term-hint")) run();
});
}
```

ФАЙЛ 13/92: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
@import "tailwindcss";
```

```
@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
  .no-scrollbar::-webkit-scrollbar {
    display: none;
  }
  .no-scrollbar {
    scrollbar-width: none;
  }
}
```

```
@keyframes pulse-gold {
```

```
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
```

```
.chapter-body :where(.text-3xl) {
  font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
  font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
  font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
  from {
    width: 0%;
  }
  to {
    width: 100%;
  }
}
```



```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/92: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 817 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных",
    category: "Продвинутые структуры",
    content: `
```

```

<div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px; margin: 10px 0;">
  <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a2b3c; color: white; margin: 0;">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C2 - (1 << ky)][R2 - (1 << kx) + 1][ky]);
int minimum = min(ans1, ans2);</pre>
  </div>
</details>
</div>
</div>
</section>`,
  }, {
    id: "treap",
    title: "3. Декартово дерево (Treap)",
    type: "html",
    description: "Дерево поиска по ключу и Куча по приоритету",
    category: "Продвинутые структуры",
    content: `
<section id="treap" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">3</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Декартово дерево (Treap)</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">Описание</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Декартово дерево (Treap) — это бинарное дерево поиска, которое также является кучей. Оно сочетает в себе свойства бинарного дерева поиска и кучи. В Treap каждая узловая точка содержит пару значений: ключ (key) и приоритет (priority). Структура дерева определяется ключами, а свойство кучи — приоритетами. Приоритеты обычно выбираются случайным образом. Вставка, удаление и поиск в Treap выполняются за  $O(\log n)$  времени. Вставка новой точки выполняется спуском от корня: пока новая точка не станет дочерейми существующей, выполняются повороты. Повороты выполняются так, чтобы дерево оставалось сбалансированным. Весь процесс — в симуляторе под названием «соединить сам»</b> — кликаешь две точки,
      </div>
      <p class="text-slate-300 text-sm mb-4">Построение Treap можно реализовать с помощью массива, где индекс — это приоритет, а значение — это ключ. Тогда процесс вставки сводится к поиску свободного места в массиве. Сложность операции вставки —  $O(\log n)$ . Поиск по ключу также выполняется за  $O(\log n)$ . Удаление выполняется аналогично вставке. Treap используется в различных приложениях, где требуется эффективное хранение и поиск точек. Например, в алгоритме Крускала для нахождения минимального остовного дерева. В Treap каждая узловая точка содержит пару значений: ключ (key) и приоритет (priority). Структура дерева определяется ключами, а свойство кучи — приоритетами. Приоритеты обычно выбираются случайным образом. Вставка, удаление и поиск в Treap выполняются за  $O(\log n)$  времени. Вставка новой точки выполняется спуском от корня: пока новая точка не станет дочерейми существующей, выполняются повороты. Повороты выполняются так, чтобы дерево оставалось сбалансированным. Весь процесс — в симуляторе под названием «соединить сам»</b> — кликаешь две точки,
    </div>
  </div>
</section>

```

быстрее $n \cdot \log n$.

`<p class="text-slate-400 text-sm mb-1">`

`<p class="text-slate-400 text-sm">` • «Уб
аются merge за $O(h)$.

`</div>`

`<p class="text-slate-300 text-sm mt-4"><b c`
`"bg-slate-950 px-2 py-0.5 rounded text-xs f`
оение — `<code class="bg-slate-950 px-2 py-0`
`code>` (сортировка + n спусков). Худший случ
`text-xs font-mono text-rose-300 border bord`
`ed text-xs font-mono text-rose-300 border b`

`<p class="text-slate-500 text-xs mt-3 leadin`
и останется набор букв, как treap без y ост
но кричат по «!»: корень самый громкий, род
; равные x — идут влево по уставу.

`<details class="bg-slate-900/40 rounded-lg l`
`<summary class="p-4 font-bold text-slate`
`-b border-slate-700/50">`

Код: split (Скрыто)

`</summary>`

`<div class="p-5 text-sm text-slate-300`

`<pre class="bg-slate-950 p-4 rounde`
`pair<Node*, Node*> split(Node* t, int x) {`
`if (!t) return {nullptr, nullptr};`
`if (t->val <= x) {`
`auto [L, R] = split(t->right, x);`
`t->right = L;`
`return {t, R};`
`} else {`
`auto [L, R] = split(t->left, x);`
`t->left = R;`
`return {L, t};`
`}`
`</pre>`

`</div>`

```

{
  id: "splay-tree",
  title: "4. Splay-дерево",
  type: "html",
  description: "Самонастраивающееся дерево: поднимаем",
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ротов поднимает x в корень. Единственный ин
      </div>

      <div class="grid grid-cols-1 md:grid-cols-3">
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">мож
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-emerald-300 font-bold">
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">0(n
          <p class="text-slate-400 text-xs mt">
        </div>
      </div>
    </div>
  </div>
`

```

омент «отстёжки», а врезка «Механика одного

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Смотрите
<div class="overflow-x-auto -mx-2 px-2">
  <table class="w-full text-xs sm:text-sm">
    <thead>
      <tr class="text-left text-slate-300">
        <th class="py-2 pr-3 font-bold">Имя
        <th class="py-2 pr-3 font-bold">Возраст
        <th class="py-2 pr-3 font-bold">Пол
        <th class="py-2 font-bold">Детали
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-500">
        <td class="py-3 pr-3 font-bold">Иван
        <td class="py-3 pr-3">деда
        <td class="py-3 pr-3">сам х
        <td class="py-3">х — корень
      </tr>
      <tr class="border-b border-slate-500">
        <td class="py-3 pr-3 font-bold">Мария
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span class="text-slate-500">
        <td class="py-3">ветка укор
      </tr>
      <tr>
        <td class="py-3 pr-3 font-bold">Алексей
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span class="text-slate-500">
        <td class="py-3">р и г стал
      </tr>
    </tbody>
  </table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>
```

муна левой половины не будет правого сына — а под «горячие» ключи, дешёвые split/merge. во меняется даже при чтении (плохо для много

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
  -b border-slate-700/50">
```

Код: rotate + splay + find (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300"
  <pre class="bg-slate-950 p-4 rounded"
  </pre>
```

```
# узел: { key, left, right, parent }
```

```
def rotate_up(x):                                # x встаёт на место
    p = x.parent
    g = p.parent
    if p.left is x:                               # правый поворот
        p.left = x.right
        if x.right: x.right.parent = p
        x.right = p
    else:                                         # левый (зеркально)
        p.right = x.left
        if x.left: x.left.parent = p
        x.left = p
    p.parent = x
    x.parent = g
    if g:
        if g.left is p: g.left = x
        else: g.right = x
```

```
def splay(x):                                    # поднять x в корень
    while x.parent:
        p, g = x.parent, x.parent.parent
        if g is None:                            # ZIG
            rotate_up(x)
        elif (g.left is p) == (p.left is x):    # ZIG-ZIG
            rotate_up(p)
            rotate_up(x)
```

```

, y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 b
      rder border-emerald-500 shadow-md">
        ≈ Аналогия 1: Вырезание прямоу
      </div>
      <p class="text-slate-300 text-sm mb-2">
        з него маленький целевой прямоугольник в цел
        ый верхний угол отрезается дважды! Поэтому
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12"
  <div class="flex items-center mb-6 flex-wrap gap-3"
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4"
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ый граф можно раскрасить 4 цветами.</p>
      </div>
    <div class="grid grid-cols-1 gap-6">

```



```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    Аналогия 1: Единственный мост
</div>
<p class="text-slate-300 text-sm mb
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
</div>
</div>
</div>
</div>
</section>`,
},
{
  id: "graph-articulation",
  title: "12. Графы. Точки сочленения",
  type: "html",
  description: "Вершины, удаление которых убивает свя
  category: "Графы. Продвинутое",
  content: `
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-rose-300 italic m
        <p class="text-xl font-mono text-white"
      теЙ > 1).</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
          Аналогия 1: Центральный роутер
        </div>

```



```

    title: "19. Графы. Остовное дерево. Прима",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ✖ Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ли вирус). Мы стартуем из одного города, и
            Сеть растет только из одного связного куска
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    description: "",
  }

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
i], который одновременно является её суффикс
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb-4">
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
      </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg">
      <summary class="p-4 font-bold text-slate-300">
        -b border-slate-700/50">
          Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
          <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
  int j = pi[i-1];
  while (j > 0 && s[i] != s[j]) j = pi[j-1];
  if (s[i] == s[j]) j++;
  pi[i] = j;

```

```

<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
    -b border-slate-700/50">
      Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <pre class="bg-slate-950 p-4 rounded"
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
  if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
  while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
  if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
      </div>
    </details>
  </div>
</div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt"
  <div class="flex items-center mb-6 flex-wrap gap-3"
    <span class="bg-purple-600 text-white px-4 py-1"
    <h2 class="text-2xl sm:text-3xl font-bold text-white"
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-purple-400"
      <div class="bg-slate-900 p-4 rounded-lg mb-4"
        <p class="text-lg text-purple-300 italic"
        <p class="text-xl font-mono text-white"
      Сведение (Reduction) A->B: Если я умею реша

```

```

    type: "html";
    content: string;
}

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">Не бы,
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-blue-400 m

    <div class="bg-slate-900 p-4 rounded-lg mb-4
      <p class="text-lg text-blue-300 italic m
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounde
          <p class="text-sm text-blue-300
            <p><strong class="text-white">П
            <p><strong class="text-white">Ц
          </div>
        </div>
      </div>
    </div>
  </div>
  <div class="bg-slate-900 p-4 rounded-lg mb-4
    <p class="text-sm text-slate-300">
      <strong>Билет 13 (Различие пути и цикла
      <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
      <br>• <span class="text-yellow-400 font

```

```

        </summary>
        <div class="p-5 text-sm text-slate-300 space-y-4">
          <p class="text-blue-400">Теорема Эйлера
          <p>
            Связный граф содержит эйлеров цикл  $\Leftrightarrow$ 
            <strong>Эйлеров путь (не цикл)  $\Leftrightarrow$  ровно 2 вершины с нечётной степенью
          </p>
          <p class="text-rose-400">Почему Гамильтон?
          <p>
            Задача коммивояжёра (TSP) – это взломать комбинаторный экспонент.
            TSP – NP-трудная: полиномиальный алгоритм не найден.
            В общем, не парься. Просто запомни.
          </p>
        </div>
      </details>
    </div>
  </section>`,
  },
  {
    id: "bridges-code",
    title: "Мосты в графах: код + визуализация",
    type: "html",
    content: `<section id="bridges-code" class="mb-20 space-y-4">
    <div class="flex items-center mb-6">
      <span class="bg-emerald-600 text-white px-4 py-2 rounded">Код
      <h2 class="text-3xl font-bold text-white">Мосты
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
        <h3 class="text-xl font-bold text-emerald-400">Мосты в графах
      </div>

      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-lg text-emerald-300 italic">Мосты в графах – это графы, в которых
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">tin[v] – количество мостов, инцидентных вершине v
          <p><strong class="text-white">low[v] – количество мостов, лежащих на пути от v к корню
          <div class="p-3 bg-slate-950 rounded">

```

```

        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border-1 border-slate-700"
        <summary class="p-4 font-bold text-slate-400"
            order-slate-700/50">
            Полный код на C++ (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300 space-y-2">
            <p>Классическая реализация – <code class="text-slate-400">
                оходов.</p>

            <div class="bg-slate-950 p-4 rounded-lg"
                <pre class="text-xs font-mono text-slate-400">
#include <vector>;
#include <algorithm>;

using namespace std;

int n;                // число вершин
vector<vector<int>>> adj; // список смежностей
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем время

    for (int to : adj[v]) {
        if (to == p) continue;          // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                   // рекурсивный вызов
        }
    }
}

```



```
<div class="bg-slate-700/50 p-6 rounded-xl border-1
  <h3 class="text-xl font-bold text-blue-400 m-0">
```

```
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-blue-300 italic">Пример 1.
    <p class="text-3xl font-mono text-white">Граф K5
    <div class="text-sm text-slate-400 space-x-4">
      <p><span class="text-blue-400 font-mono">V=5,
      <p><span class="text-emerald-400 font-mono">E=10
      <p><span class="text-rose-400 font-mono">BAH=9
    </div>
  </div>
</div>
```

```
<p class="text-slate-300 text-sm">
  <strong>Следствие:</strong> Для планарного графа
ode>.
  Если ребер больше – граф гарантированно не планарен.
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-1">
    <div class="absolute -top-3 left-4 bg-slate-900
order-green-500 shadow-md">
      K5: Полный граф на 5 вершинах
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У K5: <code class="text-white">V=5, E=10 &gt; 9</code> – БАХ, непланарен!
      Все 5 вершин соединены со всеми. Напоминает K5.
      Теорема Куратовского: K5 – эталон непланарности.
      Если внутри графа спрятан K5 – забудь про планарность.
    </p>
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-1">
  <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
```

```
ают</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-fuchsia-500 shadow-md">
```

Аналогия: карта на стене

```
</div>
```

```
<p class="text-slate-300 text-sm mb"
  Красим карту регионов: соседние
  Регион = вершина, общая граница
  Для плоской карты (планарный граф)
  теорема о четырёх красках (Аппеля)
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
er border-amber-500 shadow-md">
```

Что нужно знать на экзамен

```
</div>
```

```
<ul class="text-slate-300 text-sm s"
  <li><strong class="text-white">
вободный у соседей. Дал  $\leq \Delta+1$  цветов ( $\Delta$  – м
  <li><strong class="text-white">
тит 2 цветов, хоть он и непланарен.</li>
  <li><strong class="text-white">
= n,  $\chi$ (цикла чётной длины) = 2,  $\chi$ (цикла неч
  <li><strong class="text-white">
й и не нечётный цикл,  $\chi(G) \leq \Delta$ .</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-400 text-xs mt-4">
```

Почему раскраска «сложная»: проверить р
-полная задача.

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком (удалят вершину) — районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
```

```
</summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400 font-bold">Особый

```

```
<p>
```

Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для неё у него больше 1 независимого ребенка

```

        Косарайю, свернув каждую SCC в одну супер-вершину
        (сделаем неориентированным), то любая <strong>связность
        о и есть критическая SCC! Если удалить эту вершину, то
        . Одно слабое звено изолирует целые касты SCC.
    </ul>
</div>
</div>
</section>`,
    },
];

```

ФАЙЛ 19/92: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРок: 86 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets1to5 } from "../tickets/ticket1_5";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as extraChapters } from "../content_extra";

/**
 * Канонический порядок тем (билеты 1–24) и «правильные» названия
 * одна страница может раскрывать одну тему или несколько тем,
 * поэтому названия выверяются по единому списку ниже.
 */
const CANON: Record<string, { num: number; title: string }> = {
  "segment-trees": { num: 1, title: "1. Дерево отрезков" },
  "sparse-table": { num: 2, title: "2. Двумерная разреженная таблица" },
  "treap": { num: 3, title: "3. Декартово дерево (Treap)" },
  "splay-tree": { num: 4, title: "4. Splay-дерево" },
  "prefix-sums-2d": { num: 5, title: "5. Двумерная матрица префиксов" },
  "graph-dfs-bfs": { num: 6, title: "6. Графы. Представление и обход" },
};

```

```
    "pathfinding-accel",
  });

// Дубликаты: берём последнюю (самую свежую) версию стр.
const dedup = new Map<string, Chapter>();
merged.forEach((c) => {
  if (c.id === "alg-map") return; // страница-заглушка
  if (stale.has(c.id)) return;
  dedup.set(c.id, c);
});

// Нормализуем названия по каноническому списку тем.
const withCanon = Array.from(dedup.values()).map((c) => {
  const canon = CANON[c.id];
  if (canon) return { ...c, title: canon.title, category: canon.category };
  return c;
});

// Сначала вводная страница, затем билеты 1–24 по порядку
const numbered = withCanon.filter((c) => CANON[c.id]);
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);
const extras = withCanon
  .filter((c) => !CANON[c.id])
  .sort((a, b) => a.title.localeCompare(b.title, "ru"));

export const chapters: Chapter[] = [...extras, ...numbered];
```

ФАЙЛ 20/92: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 184 | РАЗМЕР: 10 000 байт

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */
```

```

-----
const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
return `import sys as __sys, json as __json

__src = ${srcLiteral}
__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсинга
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("-inf")) else "..."
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in dict(v).items()}
    return __safe(v)

```

```

# Финальные локалы программы (после последней строки)
if event == "return" and frame.f_code.co_filename == __file__:
    loc, values = __capture(frame)
    __final.update(loc)
    __final_values.update(values)
return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,
    "finalLocals": __final,
    "finalValues": __final_values,
    "error": __err,
    "truncated": __was_truncated,
})

__OUT = __OUT.replace("\n", "\\n")
__OUT = __OUT.replace("\r", "\\r")
__OUT = __OUT.replace("\t", "\\t")

/**
 * Какие переменные изменились на этом шаге относительно предыдущего
 * они подсвечиваются (как amber-подсветка изменений в коде)
 * Новые переменные тоже считаются изменёнными.
 */
export function changedVars(
    prev: Record<string, string> | null | undefined,
    cur: Record<string, string>
) {
    const changed: string[] = []
    for (const key in cur) {
        if (key in prev) {
            if (prev[key] !== cur[key]) {
                changed.push(key)
            }
        } else {
            changed.push(key)
        }
    }
    return changed
}

```

* Любое слово из `labels`, встреченное в тексте главы,
* в подчёркнутый термин: при наведении (или тапе на мо
* с объяснением на пальцах и мини-визуализацией.
*/

```
export interface GlossaryEntry {  
  id: string;  
  /** Варианты написания в тексте (регистр не важен). */  
  labels: string[];  
  title: string;  
  /** Объяснение «на пальцах» – 1–2 фразы. */  
  short: string;  
  /** Строгая формулировка / что важно не забыть. */  
  formal?: string;  
  complexity?: string;  
  /** Какая мини-анимация показывается в карточке. */  
  demo?: DemoKind;  
  /** id полного симулятора (см. vizRegistry) – кнопка  
  simulator?: string;  
}
```

```
export const GLOSSARY: GlossaryEntry[] = [  
  {  
    id: "bfs",  
    labels: ["BFS", "обход в ширину", "поиск в ширину",  
    title: "BFS – обход в ширину",  
    short:  
      "Волна от источника: сначала все соседи, потом со  
    formal:  
      "Очередь (FIFO). Кладём старт, достаём вершину –  
        у рёбер.",  
    complexity: "O(V + E)",  
    demo: "bfs",  
    simulator: "queue-bfs",  
  },  
  {  
    id: "dfs",  
    labels: ["DFS", "обход в глубину", "поиск в глубину",  
    title: "DFS – обход в глубину",
```



```

    formal: "push / pop / top за  $O(1)$ . На стеке живёт р
    demo: "stack",
    simulator: "stack-dfs",
},
{
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот п
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "СНМ", "система непересекающихся мн
    title: "DSU — система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего
    formal:
        "find со сжатием путей + union по рангу/размеру.
    complexity: "почти  $O(1)$  — обратная функция Аккерман
    demo: "dsu",
    simulator: "mst-kruskal",
},
{
    id: "trie",
    labels: ["бор", "бора", "боре", "бору", "бором", "t
    title: "Бор (trie, префиксное дерево)",
    short:
        "Дерево, где путь от корня по буквам и есть слово
    formal: "Из корня по символам; в вершине хранится ф
    complexity: "поиск слова —  $O(|\text{слова}|)$ ",
    demo: "trie",
    simulator: "aho-corasick",
},
{
    id: "bfs-on-trie",

```

```

        "Порядок «сначала штаны, потом ботинки»: каждая с
          циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "пре
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от н
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]
    complexity: "препроцессинг O(n), запрос O(1)",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "sparse-table",
    labels: [
        "разреженная таблица", "разреженная таблица", "sp
        "разреженная", "двоичные подьёмы", "двоичный подь
    ],
    title: "Разреженная таблица (метод двоичных подьёмов)",
    short:

```

```

    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная
    formal: "V-1 ребро, никаких циклов. Краскал (жадно и
        но).",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизи
        "амортизированное", "амортизированного", "амортизи
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер
    formal: "Метод потенциалов: дорогая операция оплачи
        ассив.",
    demo: "amortized",
},
{
    id: "np",
    labels: [
        "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
        "класс P", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (су

```

```

        "π[i] — длина наибольшего собственного префикса s
        возвращая указатель по тексту.",
    complexity: "O(n + m)",
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "z-function",
    labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
    title: "Z-функция",
    short:
        "Для каждой позиции: сколько символов подряд, начи
        аждой букве.",
    formal:
        "z[i] = длина наибольшего общего префикса s и s[i]
    complexity: "O(n)",
    demo: "prefix-function",
    simulator: "string-z-func",
},
{
    id: "dp",
    labels: [
        "динамическое программирование", "динамики", "дин
        "мемоизация", "мемоизацию", "подзадач", "подзадач
    ],
    title: "Динамическое программирование",
    short:
        "Решаем задачу через ответы на её меньшие копии и
    formal:
        "Нужны: оптимальная подструктура + перекрывающиеся
        ция).",
    demo: "dp",
    simulator: "dynamic-programming",
},
{
    id: "shortest-path",
    labels: ["кратчайший путь", "кратчайшие пути", "кра
    title: "Кратчайший путь",

```

```

    "Пройти по каждому ребру ровно один раз, не отрыв
formal:
    "Связный граф: эйлеров цикл    все степени чётные;
simulator: "euler-path-vs-cycle",
},
{
  id: "planarity",
  labels: ["планарный", "планарность", "планарного",
  title: "Планарность",
  short:
    "Граф планарен, если его можно нарисовать на листе
    ,
  formal:
    " $V - E + F = 2$  для связного планарного графа. Кур
    ,
  simulator: "planarity-euler-formula",
},
{
  id: "splay",
  labels: [
    "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
  ],
  title: "Splay и повороты",
  short:
    "Каждый раз, когда к узлу обратились, его «вытягив
  formal:
    "Три случая: zig (родитель – корень), zig-zig (уз
    рацию.",
  demo: "amortized",
  simulator: "splay-tree",
},
{
  id: "adjacency",
  labels: ["матрица смежности", "список смежности", "
  title: "Способы хранить граф",
  short:
    "Матрица смежности – таблица «есть ли ребро», быс
    й, экономно для разреженных графов.",

```

```

        шить пересадку в k?",
    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Поряд
        ами.",
    complexity: "O(V³) времени, O(V²) памяти",
    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на кажд
    formal:
        "Множество посещённых + куча рёбер на границе. До
        усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prim",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближай
    formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
},
{
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruska
    title: "Алгоритм Краскала",
    short:

```

```

    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "с"],
    title: "Сжатие координат",
    short:
        "Координаты бывают гигантские и дробные, а нам ну
        рядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс
        ",
    complexity: "O(n log n)",
    demo: "coord-compress",
},
{
    id: "treap",
    labels: ["Treap", "декартово дерево", "декартова де"],
    title: "Treap = Tree + Heap",
    short:
        "Одно дерево живёт по двум правилам сразу: по ключу
        ча. Случайность и держит его сбалансированным.",
    formal:
        "Базис – split(t, x) (разрезать по ключу) и merge
        я через них. Ожидаемая высота O(log n).",
    complexity: "ожидаемо O(log n)",
    demo: "heap",
},
{
    id: "bst",
    labels: ["бинарное дерево поиска", "дерево поиска", "д"],
    title: "Бинарное дерево поиска",
    short:
        "Слева всё меньше корня, справа – всё больше. Пои
    formal:
        "Без балансировки дерево может вырождаться в списо
    complexity: "O(высоты): от O(log n) до O(n)",
    demo: "segment-tree",
},
{
    id: "inclusion-exclusion",

```

```

    id: "string",
    labels: ["строки", "строке", "строку", "строка", "с"],
    title: "Строка как массив символов",
    short:
        "Строковые алгоритмы держатся на трёх вопросах: где (Z-функция) и где встречаются слова словаря (бор),
    formal:
        "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция.
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворо"],
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт на место своего родителя, а родитель перемещается на место дившегося месту. Порядок ключей при этом не ломается.
    formal:
        "Правый поворот вокруг p:  $x = p.left; p.left = x$ .
        Левый поворот вокруг p:  $x = p.right; p.right = x$ .
        Zig-Zag.",
    complexity: "O(1)",
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig",
    labels: ["Zig", "Zag", "зиг"],
    title: "Zig — одиночный поворот",
    short:
        "Самый простой случай: родитель x уже сидит в корневом узле. Если операция не заканчивается, если глубина была нечётной.",
    formal: "Применяется ровно один раз за всю операцию",
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig-zig",

```



```
export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1">
  <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
    <p class="text-slate-300 text-sm mb-4">Пред
    оезда). Алгоритмы поиска кратчайшего пути п
    <div class="mt-8 mb-4">
      <div id="slot-mnemonic-cards"></div>
    </div>
  </div>
</div>
</section>`
  },
  {
    id: "dijkstra",
    title: "Билет 14. Дейкстра",
    type: "html",
    category: "Графы",
    content: `<section id="dijkstra-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1">
  <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
    <p class="text-slate-300 text-sm mb-4">Пред
    оезда). Алгоритмы поиска кратчайшего пути п
    <div class="mt-8 mb-4">
      <div id="slot-mnemonic-cards"></div>
    </div>
  </div>
</div>
</section>`
  }
]
```

```

</details>

    <div id="slot-dijkstra-viz" class="mt-8"></div>
  </div>
</section>`
},
{
  id: "bellman-ford",
  title: "Билет 15. Форд-Беллман",
  type: "html",
  category: "Графы",
  content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Форд-Беллман
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

    <div class="bg-slate-900 p-4 rounded-lg mb-6">
      <p class="text-lg text-rose-300 italic">Параноик считает, что все вокруг него заговорили против него.
      <p class="text-xl font-mono text-white">
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          ♂ Аналогия 1: Параноик
        </div>
        <p class="text-slate-300 text-sm mb-6">
          Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman">
        </div>

```

```

        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; background-color: #2d3748; color: white;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 2px solid #2ecc71; border-radius: 10px; padding: 5px;">
                Аналогия 1: Автомагистрали
            </div>
            <p class="text-slate-300 text-sm mb-4">
                Сколько времени долететь из Москвы в Париж, если мы садимся на самолет?
            </p>
            <div id="slot-chapter-image-floyd" style="height: 100px; background-color: #2d3748; border: 1px solid #000; border-radius: 10px;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 2px solid #e74c3c; border-radius: 10px; padding: 5px;">
                Время работы
            </div>
            <p class="text-slate-300 text-sm mb-4">
                Сколько времени работы три вложенных цикла. Сложность  $O(V^3)$ . Если  $V=1000$ , то  $10^9$  операций.
            </p>
        </div>

        <div id="slot-floyd-viz" class="mt-8"></div>
    </div>
</div>
</section>`
},
{
    id: "aho-corasick",
    title: "Билет 23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `<section id="aho-corasick" class="mb-12">
        <div class="flex items-center mb-6 flex-wrap gap-3">
            <span class="bg-indigo-600 text-white px-4 py-1 rounded">Ахо-Корасик
            <h2 class="text-2xl sm:text-3xl font-bold text-white">Ахо-Корасик
        </div>
    `
}
```

```

<summary class="p-4 font-bold text-slate-400 outline:
  pen:border-slate-700/50">
  Внутренняя структура автомата и код BFS (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
  <p>Каждая вершина имеет таблицу переходов <code>go</code> по
    ссылке <code>term_link</code> (ближайшая терминальная вершина)
  <div class="bg-black/50 p-4 rounded-lg font-mono">
struct Node {
    map<char, int> go;          // Предвычисленные переходы
    int link = 0;              // Суффиксная ссылка: куда перейти по
    int term_link = 0;         // Терминальная ссылка: мат. слово
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные ссылки
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {
            // Суффиксная ссылка ребёнка – это переход по
            nodes[child].link = nodes[nodes[v].link].go[ch];

            // Если суффиксная ссылка сама является словом
            // Иначе наследуем terminal_link
            if (nodes[nodes[child].link].is_terminal) {
                nodes[child].term_link = nodes[child].link;
            } else {

```

ФАЙЛ 23/92: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
  "euler-path-vs-cycle": [
    {
      question: "В связном графе ровно 2 вершины с нечётными степенями",
      options: [
        "Есть эйлеров цикл (вернусь домой!)",
        "Есть эйлеров путь (из одной нечётной в другую)",
        "Ни пути, ни цикла нет — это полный провал"
      ],
      correctIndex: 1,
      explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2 — эйлеров пути нет, заканчиваем в другой."
    },
    {
      question: "Почему гамильтонов цикл — это NP-полная задача?",
      options: [
        "Потому что Гамильтон — болезнь, и лечить её тяжело",
        "Потому что для эйлерова цикла есть простой критерий",
        "Потому что Эйлер был умнее Гамильтона"
      ],
      correctIndex: 1,
      explanation: "Для эйлерова цикла работает критерий нечётности степеней. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, можно ли решить за полиномиальное время."
    },
    {
      question: "Куда ведёт эйлеров путь, если нечётных вершин ровно 2?",
      options: [
        "В любую вершину",
        "В одну из нечётных вершин",
        "В любую вершину, кроме нечётных"
```

```

        correctIndex: 1,
        explanation: "DFS обходит каждую вершину и каждое
    }
],
"planarity-euler-formula": [
    {
        question: "Планарный граф имеет  $V=6$ ,  $E=12$ . Возможн
        options: [
            "Да, если это двудольный граф",
            "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
            "Да, если нет треугольных граней"
        ],
        correctIndex: 1,
        explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани пр
            Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с  $V=7$ 
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не
    },
    {
        question: "Почему  $K_5$  непланарен?",
        options: [
            "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин – несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У  $K_5$ :  $V=5$ ,  $E=10$ ,  $3V - 6 = 9$ .  $10 > 9$ 
    }
]
};

```

```

export function emitVizState(
  chapterId: string,
  snapshot: Omit<VizStateEventDetail, "chapterId">
) {
  if (typeof window === "undefined") return;
  const detail: VizStateEventDetail = { chapterId, ...snapshot };
  latestByChapter.set(chapterId, detail);
  window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>(
    "viz-state", { detail }
  ));
}

/** Вернуть демонстрацию в ручной режим при закрытии/перезагрузке */
export function clearVizState(chapterId: string) {
  if (typeof window === "undefined") return;
  latestByChapter.delete(chapterId);
  window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>(
    "clear-viz-state", { detail: { chapterId } }
  ));
}

/** Текущий снимок компилятора для прямой отрисовки масива */
export function useVizRuntime(): VizStateEventDetail | null {
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventDetail>(
    chapterId ? latestByChapter.get(chapterId) ?? null : null
  );

  useEffect(() => {
    setRuntime(chapterId ? latestByChapter.get(chapterId) : null);
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStateEventDetail>).detail;
      if (detail?.chapterId === chapterId) {
        setRuntime(detail.variables || detail.line !== null ? detail : null);
      }
    };
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, handler);
  }, [chapterId]);

  return runtime;
}

```

```
/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
  typeof value === "number" && Number.isFinite(value) ? value : null;
}

export const vizString = (value: DebugValue | undefined) => {
  typeof value === "string" ? value : null;
}

export const vizArray = (value: DebugValue | undefined) => {
  Array.isArray(value) ? value : null;
}

export const vizRecord = (value: DebugValue | undefined) => {
  value !== null && typeof value === "object" && !Array.isArray(value)
    ? (value as Record<string, DebugValue>)
    : null;
}

/* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
  chapterId: string;
  /** base-запись страницы – demoId пустой; подписанные демо
  demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта
export function emitVizDemo(chapterId: string, demoId: string) {
  if (typeof window === "undefined") return;
  latestDemoByChapter.set(chapterId, demoId);
  window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
    detail: { chapterId, demoId }
  }));
}

/**
* Подписка: панель компилятора следует за активной вкладкой
* Сразу отдаём последнее значение: компилятор часто отстает
* как пользователь переключился с 1D на 2D.
```



```

-----
/** Заголовок демонстрации на странице (если интерактив)
vizTitle?: string;
/** Чему соответствует один шаг визуализации (для тул)
stepNote?: string;
/** Переменные синхронизации. Пусто – на странице нет
variables: SyncVariable[];
/** Полный референсный Python-код, который кнопка-гла
code?: string;
}

```

```

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков: build → query → массовое
    stepNote: "Шаг = спуск по вершине v: целиком внутри
      lazy детям 2·v и 2·v+1.",
    code: `n = 6                                # длина массива A
A = [5, 8, 3, 12, 7, 2]
tree = [0] * (4 * n)
lazy = [0] * (4 * n)                            # lazy[v] – невыполненное «

```

```

def build(v, l, r):
    if l == r:
        tree[v] = A[l]
        return
    m = (l + r) // 2
    build(2 * v, l, m)
    build(2 * v + 1, m + 1, r)
    tree[v] = tree[2 * v] + tree[2 * v + 1]
build(1, 0, n - 1)

```

```

v, l, r = 1, 0, n - 1                # вершина и её отрезок
m = (l + r) // 2                      # граница детей 2v, 2v+1
ul, ur, add = 1, 3, 10                # массовое обновление: A[l.
    variables: [
        { name: "n", role: "длина исходного массива; рекур
        { name: "i", role: "индекс элемента массива (для
        { name: "v", role: "текущая вершина дерева – подсе

```

```

        { name: "case", role: "случай поворота на вкладке"
        { name: "i", role: "номер кадра: 0 – прицел на ре
    ],
},
"splay-rotations": {
    vizTitle: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    stepNote: "Шаг = один кадр поворота из трёх классич
    code: `x, p, g = "x", "p", "g" # узел, родитель,
print("zig:      ", x, "↑", p)
print("zig-zig: ", x, "↑", p, "↑", g)
print("zig-zag: ", x, "↗", g)`
    variables: [
        { name: "x", role: "узел, который поднимаем к корн
        { name: "p", role: "родитель узла x перед поворот
        { name: "g", role: "дедушка узла x – нужен в случ
    ],
},
"sparse-table": {
    vizTitle: "Разрезанная таблица (1D и 2D)",
    stepNote: "Шаг = одна ячейка  $st[i][j] = \min(st[i][j$ 
    code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
    st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j -
    variables: [
        { name: "n", role: "длина массива / сторона квадр
        { name: "k", role: "уровень таблицы: ячейка храни
        { name: "i", role: "начало блока в строке уровня
        { name: "j", role: "номер уровня при построении (
        { name: "r, c", role: "строка и столбец ячейки в
        { name: "kx, ky", role: "уровни по вертикали и го

```

```

-----
print("итог:", fib(n))`,
    variables: [
        { name: "n", role: "целевой номер числа Фибоначчи"},
        { name: "k", role: "аргумент текущего рекурсивного вызова"},
        { name: "memo", role: "кэш уже посчитанных значений"},
        { name: "r1, r2", role: "ответы двух рекурсивных вызовов"},
    ],
},
"heap-beam-search": {
    vizTitle: "Куча (priority queue)",
    stepNote: "Шаг = просеивание: вставленный элемент в кучу",
    code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
    variables: [
        { name: "n", role: "текущее число элементов в куче"},
        { name: "i", role: "индекс элемента, который сейчас просеиваем"},
        { name: "(i-1)//2", role: "индекс родителя вершины"},
        { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
    ],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень
print("pop →", top, "стек:", st)`,
    variables: [
        { name: "top", role: "вершина стека — то, откуда мы вышли"},
        { name: "v", role: "текущая вершина графа, которую мы посещаем"},
    ],
},

```

```

vizTitle: "Компоненты связности: BFS-разметка",
stepNote: "Шаг = вершина извлечена из очереди или с
code: `from collections import deque

adj = {0: [1, 2], 1: [0], 2: [0], 3: [4], 4: [3]}
comp = {}                                # comp[v] – номер компонен
count = 0
v, q = None, []                          # текущая вершина и очередь

for s in adj:                             # шаг: перезапуск обхода
    if s in comp:
        continue
    count += 1
    q = deque([s]); comp[s] = count - 1
    while q:                              # шаг: вершина из очереди
        v = q.popleft()
        for to in adj[v]:
            if to not in comp:
                comp[to] = comp[v]
                q.append(to)
print(comp, "компонент:", count)`
variables: [
    { name: "v", role: "текущая вершина обхода (метка
    { name: "comp", role: "словарь или список: номер
    { name: "q", role: "текущая очередь BFS", range:
    { name: "count", role: "сколько раз пришлось запу
],
},
"top-sort": {
    vizTitle: "Топологическая сортировка",
    stepNote: "Шаг = выход рекурсии из вершины v: она д
    code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:                    # шаг: ребро v → to
        if to not in used:

```

```

    },
    "graph-articulation": {
        vizTitle: "Точки сочленения",
        stepNote: "Шаг = обновление low[v] по ребру; вершина",
        code: `tin, low, timer = {}, {}, 0

v, to = 0, 1                                # текущее ребро DFS
tin[v] = tin.get(v, timer); low[v] = tin[v]

# шаг: возврат из to
low[v] = min(low[v], low.get(to, tin[v]))
critical = low.get(to, 0) >= tin[v]    # → v — точка соч.
print(f"v={v} to={to} critical={critical}")`,
        variables: [
            { name: "v", role: "текущая вершина DFS", range: 0..1000 },
            { name: "to", role: "сосед: либо ребёнок в DFS-дереве", range: 0..1000 },
            { name: "tin[v]", role: "время входа в вершину (поиск)", range: 0..1000 },
            { name: "low[v]", role: "минимальный tin, достигнутый из to", range: 0..1000 },
            { name: "i", role: "перебор стартовых вершин для DFS", range: 0..1000 }
        ],
    },
    "bridges-code": {
        vizTitle: "Мосты: DFS + tin/low",
        stepNote: "Шаг = строка псевдокода слева; ребро (v, to)",
        code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1, "B": 2, "G": 3}
timer = 3                                # как на демо

v, to = "B", "G"                        # шаг: возврат из G
is_bridge = low[to] > tin[v]             # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
        variables: [
            { name: "v", role: "текущая вершина DFS", range: 0..1000 },
            { name: "to", role: "сосед; ребро (v, to) проверяется", range: 0..1000 },
            { name: "tin[v]", role: "время входа в вершину — по DFS", range: 0..1000 },
            { name: "low[v]", role: "минимум tin, куда можно пройти из to", range: 0..1000 },
            { name: "timer", role: "глобальный счётчик времени", range: 0..1000 }
        ],
    },

```

```

while c in used: c += 1
color[v] = c
k = max(k, c + 1)`,
variables: [
    { name: "V", role: "число вершин графа", range: "1"},
    { name: "E", role: "число рёбер", range: "K5: 10"},
    { name: "F", role: "число граней в плоской укладке", range: "1"},
    { name: "v", role: "вершина, которую красят (метка)", range: "1"},
    { name: "color", role: "массив/словарь цветов вершин", range: "1"},
    { name: "k", role: "сколько цветов уже использовано", range: "1"},
],
},
dijkstra: {
    vizTitle: "Дейкстра",
    stepNote: "Шаг = старт, извлечение ближайшей вершины",
    code: `import heapq

g = {"A": [("B", 4), ("C", 2)], "B": [("G", 4), ("D", 5)],
      "E": [("F", 1)], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
    du, u = heapq.heappop(pq)
    if u in done:
        continue
    done.add(u)
    print(f"обрабатываем {u} (dist={du})")
    for v, w in g[u]:
        if v in done:
            continue
        print(f"смотрим ребро {u}→{v} (w={w})")
        if du + w < dist.get(v, float("inf")):
            dist[v] = du + w
            prev[v] = u
            heapq.heappush(pq, (dist[v], v))
print("итог:", dist)`
```

```

for i in range(n):
    for j in range(n):
        if i != j and i != k and j != k and d[i][k]:
            if d[i][k] + d[k][j] < d[i][j]:
                d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
variables: [
    { name: "n", role: "число вершин – размер квадрата"},
    { name: "k", role: "промежуточная вершина – ВНЕШНИЙ"},
    { name: "i", role: "начало пути – строка матрицы"},
    { name: "j", role: "конец пути – столбец матрицы"},
    { name: "dist[i][j]", role: "кратчайший путь i → j"},
],
},
"johnson-algo": {
    vizTitle: "Алгоритм Джонсона (перевзвешивание)",
    stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
    code: `# потенциалы из Беллмана-Форда от фиктивной вершины
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

u, v, w = "A", "B", -3      # ребро с отрицательным весом
w_new = w + h[u] - h[v]     # w' = w + h[u] - h[v] ≥ 0
print(f"w'({u}→{v}) = {w_new}")
# после Дейкстры: ответ = d[v] - h[start] + h[v]`,
variables: [
    { name: "h[v]", role: "потенциал вершины – кратчайшее расстояние от Беллмана-Форда"},
    { name: "u, v", role: "ребро (u, v) перевзвешивается"},
    { name: "d[v]", role: "расстояние Дейкстры в новой матрице"},
],
},
"mst-kruskal": {
    vizTitle: "Краскал и DSU",
    stepNote: "Шаг = следующее ребро из отсортированных",
    code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "A", "C"), (4, "D", "E"), (5, "D", "F"), (6, "E", "F")]
edges.sort()                # по весу

```

```

        { name: "start", role: "стартовая вершина, из неё
    ],
},
"mst-boruvka": {
    vizTitle: "Борувка",
    stepNote: "Шаг = каждая компонента одновременно выб
    code: `comp = {}`                                     # корень компо
for v in "ABC":
    comp[v] = v
cheapest = {}                                           # comp → самое дешёвое ребро
u, v, w = "A", "B", 1                                # шаг: ребро-кандидат
if comp[u] != comp[v]:
    cheapest[comp[u]] = (w, u, v)
print(cheapest)                                         # за фазу компонент станет
    variables: [
        { name: "comp", role: "компонента связности текущ
        { name: "cheapest[comp]", role: "самое дешёвое ре
        { name: "u, v, w", role: "ребро-кандидат: концы и
    ],
},
"string-kmp": {
    vizTitle: "Префикс-функция (КМП)",
    stepNote: "Шаг = обработка символа s[i]: откаты по
    code: `s = "aabaabaaba"
n = len(s)
i, j = 1, 0
pi = [None] * n                                         # пустые клетки: значения п
pi[0] = 0

for i in range(1, n):
    # шаг: считаем pi[i]
    j = pi[i - 1]                                       # длина совпадения
    while j > 0 and s[i] != s[j]:
        j = pi[j - 1]                                  # откат назад
    if s[i] == s[j]:
        j += 1
    pi[i] = j
print(pi)\`

```



```

-----
i, v = 0, 0                # позиция в тексте и вершина
trie = {"": {}}           # бор: вершина → переходы
state, c = "", "u"        # шаг: автомат идёт по символу

nxt = trie.get(state, {}).get(c, "") # go(state, c) с
print(f"δ({state!r}, {c!r}) → {nxt!r}")`,
    variables: [
        { name: "s", role: "текст для поиска (можно и text)" },
        { name: "i", role: "текущая позиция в тексте — локаль" },
        { name: "v", role: "номер текущей вершины автомата" },
        { name: "c / state", role: "символ и состояние — шаг" },
    ],
},
"complexity-classes": {
    vizTitle: "Классы сложности: матрёшка  $P \subseteq NP \subseteq PSPACE$ ",
    stepNote: "Шаг = подсветка класса/задачи по имени и сложности",
    code: `cls = "NP" # класс: "P", "NP", "PSPACE"`,
},
task = "3-SAT" # задача: "3-SAT", "Vertex Cover"

# сведение  $A \leq_p B$ : стрелка между задачами
reductions = [{"3-SAT", "Vertex Cover"}]
for a, b in reductions:
    print(f"{a}  $\leq_p$  {b}")`,
    variables: [
        { name: "cls", role: "какой класс подсветить: P / NP / PSPACE" },
        { name: "task", role: "какую задачу подсветить: 3-SAT / Vertex Cover" },
        { name: "reductions", role: "список пар задач — парадигма" },
    ],
},
intro: {
    // мнемодкарточки без шагов алгоритма
    variables: [],
},
"alg-map": {
    variables: [],
},
},
/* — Дополнительные демонстрации внутри страниц (вкл.

```

```
-----
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем",
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][
r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`,
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (
        { name: "S[i][j]", role: "сумма прямоугольника (0
    },
    { name: "r1, c1, r2, c2", role: "углы запроса; от
],
},
};
```

```
/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null,
    (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demo
    (chapterId && PAGE_SYNC[chapterId]) ||
    { variables: [] };
```

Универсальное пособие • полный исходный код

```
-----
* Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
* переменные демо (без всего кода алгоритма).
*/
export function buildInitTemplate(chapterId: string, chapterTitle: string, demoId: string) {
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
    : `# «${chapterTitle}»\n# демо на странице нет – св

  if (!sync.code) return head;
  const init = extractInit(sync.code);
  return `${head}\n${init}\n`;
}
```

=====

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

ФАЙЛ 26/92: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from '../types';
```

```
export const tickets1to5: Chapter[] = [
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description: "Дерево отрезков – структура данных для оптимизации",
    category: "Оптимизации и Продвинутое",
    content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
```

```
-----
    -b border-slate-700/50">
        Строгое доказательство / Код (Скр
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
function push(node) {
    if (lazy[node] !== 0) {
        lazy[2 * node] += lazy[node];
        tree[2 * node] += lazy[node];
        lazy[2 * node + 1] += lazy[node];
        tree[2 * node + 1] += lazy[node];
        lazy[node] = 0;
    }
}
</pre>
        </div>
    </details>
</div>
</div>
</section>
`
    },
    {
        id: "sparse-table",
        title: "2. Двумерная разреженная матрица (Sparse Table)",
        type: "html",
        description: "Разреженная таблица для идемпотентных",
        category: "Продвинутые структуры",
        content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-violet-500">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-slate-700">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
```



```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
    <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
          <p class="text-xl font-mono text-white">
            (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию.
          </p>
        </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вится корнем дерева (VIP-статус). Если ты ч
            ка почти не требуется времени!</p>
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
              Аналогия 2: Балансировка чере
            </div>
            <p class="text-slate-300 text-sm mb-4">
              и становиться кривым как бамбук. Но как тол
            </p>
          </div>
        </div>
      </div>
    <details class="bg-slate-900/40 rounded-lg">
      <summary class="p-4 font-bold text-slate-300 border border-slate-700/50">
        Код (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
  let p = x.parent;

```

```

        </div>
      </div>
    </div>
  </div>
</section>`
},
{
  id: "prefix-sums-2d",
  title: "5. Двумерная матрица префиксных сумм",
  type: "html",
  description: "Сжатие суммы подматрицы за  $O(1)$ ",
  category: "Алгоритмы матрицы",
  content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ,  $y1...y2$ ) =  $P[x2][y2]$  -  $P[x1-1][y2]$  -  $P[x2][y1-1]$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            ≈ Аналогия 1: Вырезание прямоугольника
          </div>
          <p class="text-slate-300 text-sm mb-4">
            з него маленький целевой прямоугольник в центре
            ый верхний угол отрезается дважды! Поэтому
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
(нельзя поехать и заработать). Он всегда в
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
                Ограничения
            </div>
            <p class="text-slate-300 text-sm mb-6">
дный и не умеет пересматривать уже "зафиксир
            </div>
        </div>
        <div id="slot-dijkstra-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "bellman-ford",
        title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="bellman-ford-content" class="mb-12 scroll-my-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">15
        <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Пути
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-rose-400/50">
            <h3 class="text-xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">Аналогия 1
                <p class="text-xl font-mono text-white">Графы. Пути
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">

```



```

        <p class="text-xl font-mono text-white">
    </div>

    <div class="bg-slate-800 p-6 rounded-lg border-2
        <div class="absolute -top-3 left-4 bg-slate-800
        border-emerald-500 shadow-md">
            Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Главный герой – <b>внешний цикл</b>
            шаг <code class="bg-slate-900 text-pink-400">
            шу себе проходить через вершину k?</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
            <li><b>Шаг k=0:</b> Ты знаешь только
            <li><b>Шаг k=1:</b> Разрешаем пересадки
            е> через путь <code class="text-indigo-300">
            <li><b>Шаг k=2:</b> Разрешаем пересадки
            нутри этих путей уже могут быть пересадки ч
            <li>...</li>
            <li><b>Шаг k=N:</b> Ты проверил все
        </li>
        </ul>
    </div>
</div>
</div>
</section>`
    },
    {
        id: "johnson-algo",
        title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

        category: "Графы. Остовные",
        content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия: Прокладка интернет
          </div>
          <p class="text-slate-300 text-sm mb-4">
            с самых дешевых дорог в стране". Он строит
            единую сеть.</p>
        </div>
      </div>
    </div>
  </div>
</section>`
      },
      {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-emerald-500">
  <h3 class="text-xl font-bold text-emerald-400">
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
ается с соседом.</p>
  </div>
  <div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
        Аналогия: Племена
      </div>
      <p class="text-slate-300 text-sm mb-4">
изкому племени для объединения. К вечеру пле
ства шлют гонцов к ближайшему чужому царству.
    </div>
  </div>
</div>
</div>
</section>`
  }
};
```

ФАЙЛ 28/92: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ: 1000

```
import { Chapter } from '../types';
```

```
export const tickets21to24: Chapter[] = [
```

```
  {
    id: "string-kmp",
    title: "21. Префикс-функция (π) – Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
```

Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить

```
</p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300
  <p>Тот же пример: <b>abcabcd</b></p>
  <ul class="list-disc pl-5 space-y-2
    <li><b>a</b>: Префиксов нет. <co
    <li><b>ab</b>: Из начала ("a")
  >
    <li><b>abc</b>: Опять мимо. <co
    <li><b>abca</b>: 0! Начало ("a"
</code></li>
    <li><b>abcab</b>: Начало ("ab")
  /li>
    <li><b>abcabc</b>: Начало ("abc
e></li>
    <li><b>abcabcd</b>: "d" всё исп
с' – не совпало. Откат по  $\pi$ :  $j = \pi[2] = 0$ .
 $\pi[6] = 0$ </code></li>
  </ul>
```

```
</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300
  <pre class="bg-slate-950 p-4 rounded
```

Аналогия 1: Линейка-шаблон

```
</div>
<p class="text-slate-300 text-sm mb-4">
  Тут мы берём всю строку и «прикидываю
  я начну читать строку отсюда, насколько длинн
  Это самый быстрый способ найти индекс
  >, считаешь Z-функцию, и там, где <code>Z[i]
</p>
</div>
```

<!-- Аналогия 2 -->

```
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 right-4 bottom-4
border-rose-500 shadow-md">
```

Аналогия 2: LLM Самокопираст

```
</div>
<p class="text-slate-300 text-sm mb-4">
  В LLM Z-функция может применяться к строке
  [i]</code> (где <i>i</i> указывает назад на индекс
  сама себя.
</p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg p-6">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300">
  <p>Тот же пример: <b>abcabcd</b></p>
  <ul class="list-disc pl-5 space-y-2">
    <li><code>Z[0]</code> – не считаем, так как Z[0] = 0</li>
    <li><code>Z[1]</code> (<b>bcabcd</b>): Z[1] = 0</li>
    <li><code>Z[2]</code> (<b>cabcd</b>): Z[2] = 0</li>
    <li><code>Z[3]</code> (<b>abcd</b>): Z[3] = 0</li>
```

```
<h3 class="text-xl font-bold text-blue-400 mb-4">

<div class="bg-slate-900 p-4 rounded-lg mb-6 text
  <p class="text-sm text-blue-300 italic mb-2">0с
  <p class="text-lg font-mono text-white">Бор (Тр
</div>
<p class="text-slate-300 text-sm">Позволяет найти
  го один проход.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      ет — мы падаем по <b>суффиксной ссылке</b> ("
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
      </div>
    </div>

<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 ou
    r-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-
    <pre class="bg-slate-950 p-4 rounded overfl
```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Судoku (P vs NP)
    </div>
    <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
  </div>
  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
      Аналогия 2: Машина-переводчик
    </div>
    <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче В, а сама В лежит в NP, то В – <b>NP-по
  </div>
</div>
</div>
</section>`
  }
};

```

ФАЙЛ 29/92: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```

        </div>
    </div>

    { /* DFS vs BFS */ }
    <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-indigo-500 shadow-md">
        <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS</h3>

        <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
            <!-- Аналогия DFS -->
            <div class="bg-slate-800 p-6 rounded-lg border border-indigo-500 shadow-md">
                <div class="absolute -top-3 left-4 right-4 bottom-4">
                    ♂ DFS (Поиск в глубину) = Жадный алгоритм
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    <b>Что делает:</b> Жадный алгоритм ищет первую доступную
                    у вершину (например, минимальную по номеру) и пробует другой путь.
                </p>
                <ul class="text-xs text-indigo-300 list-disc pl-2">
                    <li><b>Где используется:</b></li>
                    <li>Топологическая сортировка (линейное время)</li>
                    <li>Поиск мостов и точек сочленения</li>
                    <li>Сильно связанные компоненты Киркмана</li>
                    <li>Поиск циклов и просто проверка связности</li>
                </ul>
            </div>

            <!-- Аналогия BFS -->
            <div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
                <div class="absolute -top-3 left-4 right-4 bottom-4">
                    ♂ BFS (Поиск в ширину) = Волна
                </div>
                <p class="text-slate-300 text-sm mb-4">
                    <b>Что делает:</b> Концентрические волны от начальной точки.
                    Работает через Очередь (FIFO). Продвигается по всем соседям.
                </p>
            </div>
        </div>
    </div>

```



```

        </div>
    </div>
</section>`,
    },
    {
        id: "top-sort",
        title: "9. Графы. Топологическая сортировка",
        type: "html",
        description: "Разложение задач по порядку выполнения",
        category: "Графы",
        content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
перед.</p>
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Учеба в вузе
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
                </div>
            </div>
            <div class="bg-slate-900 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                    Ограничения: Циклы
                </div>
            </div>
        </div>
    </div>
</section>`
    }
]

```

```

        </div>
    </div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
            <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
        </div>

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">Макро-вершина — это вершина орграфа, состоящая из
            <div class="text-left font-mono text-sm">
                <p><strong class="text-white">Компоненты сильной связности (SCC) — это вершины ориентированного графа, в котором
            </div>
            </div>

            <p class="text-slate-300 text-sm mb-4">
                <strong>«Это просто цикл?» — Нет!</strong>
            </p>

            <ul class="list-disc list-inside text-sm text-slate-300">
                <li>Если у тебя есть цикл <code>A → B → C → A</code>, то это один цикл.
                <li>Если ты добавишь вершину <strong>Г</strong> к этому циклу, то получишь четыре вершины — одна большая SCC.</li>
                <li>Правило: <b>Если два цикла имеют хотя бы одну общую вершину, то они образуют один цикл.</b></li>
            </ul>
        </div>
    </div>
`
    }
}

```

```

</li>Разворачиваем этот список (
</li>Переворачиваем все ребра в
</li>Идем по <code>order</code>:
!</li>
</ol>
</div>
</details>
</div>
</div>
</section>`,
},
{
  id: "graph-bridges",
  title: "11. Графы. Мосты",
  type: "html",
  description: "Рёбра, удаление которых расхреначивает",
  category: "Графы. Продвинутые",
  content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">

```

Важно: Это работает только в тех случаях, когда мы переходим к точкам сочленения. То есть, **вершина** - это тупик со степенью 1).

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком. Если перекресток выйдет из строя, её перекроют (удалят вершину) – районы будут изолированы. **Хрупкость** системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
  <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной сети. Если один из них выйдет из строя, абелем (мостом). Сами свитчи – это точки сочленения.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    <div class="order-slate-700/50">
```

Душный мод: Код и корень DFS (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Общая информация
```

```

ong> и показывают физическую уязвимость свя
    <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
    <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
</ul>
</div>
</div>
</section>`,
},
{
  id: "graph-euler",
  title: "13. Графы. Эйлеров цикл",
  type: "html",
  description: "Пройти по всем ребрам ровно 1 раз.",
  category: "Графы",
  content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрыв
          </div>

```

```

    { id: 0, x: 250, y: 50, label: "0" },
    { id: 1, x: 100, y: 150, label: "1" },
    { id: 2, x: 250, y: 250, label: "2" },
    { id: 3, x: 400, y: 150, label: "3" },
    { id: 4, x: 550, y: 150, label: "4" },
    { id: 5, x: 700, y: 50, label: "5" },
    { id: 6, x: 700, y: 250, label: "6" },
  ];

  const edges: Edge[] = [
    { u: 0, v: 1 },
    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 },
    { u: 3, v: 4 }, // 3 and 4 are articulation points (3
    { u: 4, v: 5 },
    { u: 5, v: 6 },
    { u: 6, v: 4 },
  ];

  // Adjacency list
  const adj: number[][] = Array.from(
    { length: Object.keys(nodes).length },
    () => [],
  );
  edges.forEach((e) => {
    adj[e.u].push(e.v);
    adj[e.v].push(e.u);
  });

  interface StepInfo {
    desc: string;
    visited: Set<number>;
    ap: Set<number>;
    currentNode: number | null;
    tin: number[];
    low: number[];
  }

```

```

    for (const to of adj[v]) {
        if (to === p) continue;

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${v}],`
            );
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v}],`
            );

            if (currentLow[to] >= currentTin[v] && p !== ap.add(v)) {
                recordStep(
                    `Найдена точка сочленения: вершина ${v}`
                );
            }
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это`
    );
} else if (p === -1) {
    recordStep(

```



```

const reset = () => {
  setIsPlaying(false);
  setCurrentStepIndex(0);
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-3 g

```

```

const stroke = isCurrent
  ? "#60a5fa"
  : isAp
    ? "#f43f5e"
    : isVisited
      ? "#34d399"
      : "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-300"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.label}
    </text>

    {isVisited && (
      <>
        <text
          x={node.x}
          y={node.y - r - 15}

```

```
</div>
```

```
<div className="space-y-4 flex-1 overflow-y-auto">
  <div className="mb-4">
    <h5 className="text-xs text-slate-500 font-medium">
      Точка сочленения
    </h5>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
        Точка сочленения
      </div>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
          Посещена
        </div>
        <div className="flex items-center gap-2">
          <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
            Текущая
          </div>
          <div className="flex items-center gap-2">
            <div className="w-8 h-1 bg-rose-500 border-1px border-slate-500">
              Мост
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```
<div>
  <h5 className="text-xs text-slate-500 font-medium">
    ЛОГ ДЕЙСТВИЙ:
  </h5>
  <div className="space-y-2">
    {steps
      .slice(0, currentStepIndex + 1)
      .reverse()
      .map((step, idx) => (
        <div
          key={idx}
          className={`text-xs p-2 rounded $
            xt-slate-500`} `}
        >
          {step.desc}
        </div>
      ))
    }
  </div>
</div>
```

```

    hasNegativeCycle?: boolean;
    activeCodeLine: number;
}

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {
      return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
    }
  };
};

```

```

    let anyUpdate = false;
    for (const edge of edges) {
        const uDist = dist[edge.u];
        const vDist = dist[edge.v];

        let updated = false;
        if (uDist !== 999 && uDist + edge.w < vDist) {
            dist = { ...dist, [edge.v]: uDist + edge.w };
            prev = { ...prev, [edge.v]: edge.u };
            updated = true;
            anyUpdate = true;
        }

        simulationSteps.push({
            iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
            distances: { ...dist }, previous: { ...prev },
            log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
                ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + dist[edge.v] : ''}
            `,
            activeCodeLine: updated ? 6 : 5,
        });
    }

    if (!anyUpdate && iter < vCount - 1) {
        simulationSteps.push({
            iteration: iter, checkingEdge: null, distances: { ...dist },
            log: `Ранний выход после итерации ${iter}.`,
            activeCodeLine: 10,
        });
        break;
    }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount - 1) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
        }
    }
}

```

```

const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
  ? {
      ...baseStep,
      iteration: liveI ?? 0,
      checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
      distances: Object.fromEntries(nodes.map((node) => [node, liveDist[node]])),
      log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU ?? "-"}-${liveV ?? "-"}, w=${liveW ?? "-"}, dist=${liveDist[liveU ?? "-"] ?? "-"}
    }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-3">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white rounded-lg">
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-blue-300">Полный
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center gap-3">
      <div className="bg-slate-900 p-1 rounded-lg border border-slate-800">
        <button onClick={() => setHasCycle(false)} className="p-2 text-white">
          <button onClick={() => setHasCycle(true)} className="p-2 text-white">
        </div>
      <div gap-1 ${hasCycle ? 'bg-rose-600 text-white' : 'bg-slate-600 text-white'} px-3 py-2 rounded-lg">
        <button onClick={() => { setIsPlaying(false);

```

```

v;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

  let strokeColor = '#475569';
  let marker = 'url(#arrow-bf-normal)';
  if (isChecking) { strokeColor = '#f59e0b';
  else if (currentStep.hasNegativeCycle && isCyclePart) {
    strokeColor = '#f43f5e';
  }

  return (
    <g key={idx}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={isChecking ? 4 : 2} marker={marker} />
      <circle cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)} stroke={strokeColor} stroke-width={isChecking ? 4 : 2} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} />
      <text x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Neg' : 'Cycle'} font-size="11" font-weight="bold" />
    </g>
  );
}
}}}

{nodes.map((node) => {
  const dist = currentStep.distances[node.id];
  const isCheckingNode = currentStep.checkingNode === node.id;

  return (
    <g key={node.id} className="transition">
      <circle cx={node.x} cy={node.y} r={isCheckingNode ? 10 : 5} stroke={isCheckingNode ? '#7dd3fc' : '#64748b'} stroke-width={isCheckingNode ? 2 : 1} fill={isCheckingNode ? '#7dd3fc' : '#64748b'} />
      <text x={node.x} y={node.y + 5} font-size="10" font-weight="normal" />
      <text x={node.x} y={node.y - 28} font-size="10" font-weight="normal" />
      <text x={node.x} y={node.y + 35} font-size="10" font-weight="normal" />
      <text x={node.x} y={node.y + 45} font-size="10" font-weight="normal" />
    </g>
  );
})}

```

```

    <div className="grid grid-cols-4 gap-2 text-slate-600"
      {nodes.map(n => {
        const d = currentStep.distances[n.id];
        const isTgt = currentStep.checkingEdge === n.id;
        return (
          <div key={n.id} className={`p-2 rounded-lg border
            : 'border-slate-700/60 bg-slate-800/40'`}
            <div className="text-xs text-slate-500"
              <div className={`font-mono font-bold text-slate-400`}
                {d}
              </div>
            </div>
          </div>
        );
      })}
    </div>
  </div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 p-4"
  <div>
    <h4 className="text-lg font-bold text-slate-400">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4"
      {codeLines.map(item => {
        <div key={item.line} className={`py-1.5 px-2`}
          e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
          <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
          <span className="whitespace-pre flex-grow-1">{item.code}</span>
        </div>
      })}
    </div>
  </div>
</div>
</div>
</div>
);
}

```



```
const queue: number[] = [];
const visited = new Set<number>();

// Init
steps.push({
  activeLine: 1,
  queue: [],
  visited: [],
  currentNode: null,
  logs: "Инициализация: начинаем с корня (0).",
});

queue.push(0);
visited.add(0);
steps.push({
  activeLine: 2,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Добавляем стартовую вершину 0 в очередь и по",
});

while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}`,
  });

  const curr = queue.shift()!;
  steps.push({
    activeLine: 5,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Достаем из очереди (${curr}).`
```

```

        activeLine: 12,
        queue: [],
        visited: Array.from(visited),
        currentNode: null,
        logs: "Очередь пуста. Алгоритм завершен!"
    });

    return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
    const [stepIdx, setStepIdx] = useState(0);
    const [isPlaying, setIsPlaying] = useState(false);

    const step = STEPS[stepIdx];

    const handleNext = useCallback(() => {
        setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
    }, []);

    // @ts-expect-error зарезервировано под кнопку «назад»
    const handlePrev = useCallback(() => {
        setStepIdx(prev => Math.max(prev - 1, 0));
    }, []);

    useEffect(() => {
        if (!isPlaying) return;
        if (stepIdx >= STEPS.length - 1) {
            setIsPlaying(false);
            return;
        }
        const timer = setTimeout(handleNext, 1200);
        return () => clearTimeout(timer);
    }, [isPlaying, stepIdx, handleNext]);

    return (

```

```

50px]">
<svg className="w-full h-full min-h-[350px]"
  {/* Рёбра */}
  {EDGES.map((e, idx) => {
    const n1 = NODES.find(n => n.id === e.from)
    const n2 = NODES.find(n => n.id === e.to)
    const isVisited = step.visited.includes(n1)
    return (
      <line
        key={idx}
        x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
        className={`transition-all duration-500 ${
          isVisited ? 'stroke-slate-900' : 'stroke-slate-400'
        }`}
      />
    )
  })}
/>
)
})

```

```

{/* Вершины */}
{NODES.map(n => {
  const isCurrent = step.currentNode === n.id
  const isInQueue = step.queue.includes(n.id)
  const isVisited = step.visited.includes(n.id)

```

```

  let fill = '#1e293b'; // slate-800
  let stroke = '#475569'; // slate-600
  let scale = 1;
  void scale;

```

```

  if (isCurrent) {
    fill = '#3b82f6'; // blue-500
    stroke = '#60a5fa'; // blue-400
    scale = 1.3;
  } else if (isInQueue) {
    fill = '#059669'; // emerald-600
    stroke = '#34d399'; // emerald-400
    scale = 1.1;
  } else if (isVisited) {
    fill = '#0f172a'; // slate-900

```

```

</div>
<div className="p-4">
  {[
    { line: 1, code: 'queue = Queue()' },
    { line: 2, code: 'queue.push(start); vi' },
    { line: 3, code: '' },
    { line: 4, code: 'while not queue.isEmp' },
    { line: 5, code: '  node = queue.pop()' },
    { line: 6, code: '  for neighbor in gra' },
    { line: 7, code: '    if neighbor not in' },
    { line: 8, code: '      visited.add(ne' },
    { line: 9, code: '      queue.push(nei'
  ]}.map((cl) => {
    const isActive = step.activeLine === cl
    return (
      <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
      <span className="opacity-40 w-6 inl
      <span className="whitespace-pre">{c
    </div>
    );
  })}
</div>
</div>

<div className="bg-slate-800/80 border border
  <div className="text-sm font-bold text-eme
    <span>Состояние Очереди (Queue):</span>
    <span className="text-xs text-slate-500
  </div>
  <div className="flex items-center gap-2 mi
    {step.queue.length === 0 ? (
      <span className="text-slate-500 text-x
    ) : (
      step.queue.map((qId, i) => (
        <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
        {qId}

```

```
      captionColor: 'text-rose-400',
    },
    floyd: {
      src: floydImg,
      alt: 'Все ко Всем Флойд',
      caption: '  Все связаны со всеми!',
      borderColor: 'border-emerald-500/30',
      captionColor: 'text-emerald-400',
    },
  };

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-emerald-500`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
    </div>
  );
}
```

ФАЙЛ 34/92: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
  prev?: Chapter;
  next?: Chapter;
```

```

    );
  }

  return (
    <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
      {prev ? (
        <button
          onClick={() => onGo(prev.id)}
          className="group text-left p-4 rounded-xl bg-white border border-gray-200 transition-all"
        >
          <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
            <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
          </div>
          <div className="text-sm font-bold text-slate-700">
            {prev.title}
          </div>
        </button>
      ) : (
        <div className="hidden sm:block" />
      )}

      {next && (
        <button
          onClick={() => onGo(next.id)}
          className="group text-right p-4 rounded-xl bg-white border border-gray-200 transition-all sm:col-start-2"
        >
          <div className="flex items-center justify-end gap-1.5 text-indigo-400 mb-1">
            Следующая тема <ChevronRight className="w-3.5 h-3.5" />
          </div>
          <div className="text-sm font-bold text-slate-700">
            {next.title}
          </div>
        </button>
      )}
    </nav>
  );
};

```

```

const cancelHide = useCallback(() => {
  if (hideTimer.current) {
    window.clearTimeout(hideTimer.current);
    hideTimer.current = null;
  }
}, []);

const scheduleHide = useCallback(() => {
  cancelHide();
  hideTimer.current = window.setTimeout(() => setAnchor(
), [cancelHide]);

useEffect(() => {
  setAnchor(null);
  setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect()
  },
  [cancelHide]
);

```

```

>
  {saving === "work" ? (
    <Loader2 className="w-3.5 h-3.5 animate-spin" />
  ) : saving === "done" ? (
    <Check className="w-3.5 h-3.5 text-emerald-500" />
  ) : (
    <Download className="w-3.5 h-3.5" />
  )}
  {saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
  onClick={() => window.print()}
  title="Распечатать или сохранить в PDF среде"
  className="flex items-center gap-1.5 text-x-small
  300 hover:text-white hover:border-indigo-500" />
>
  <Printer className="w-3.5 h-3.5" /> Печать
</button>
<span className="text-[11px] text-slate-500">
</div>

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

<p className="mt-6 flex items-start gap-2 text-slate-500
  ed-lg px-3 py-2">
  <MousePointerClick className="w-4 h-4 shrink-0" />
  <span>
    Подчёркнутые термины – интерактивные: наведи
    объяснение на пальцах, мини-анимацию и кноп
  </span>

```



```
    </article>

    <TermPopover
      anchor={anchor}
      onClose={() => setAnchor(null)}
      onOpenSimulator={(id) => {
        setAnchor(null);
        onOpenSimulator(id);
      }}
      onCardEnter={cancelHide}
      onCardLeave={scheduleHide}
    />
  </>
);
};
```

ФАЙЛ 36/92: src/components/ColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useMemo, useState } from "react";
import { Palette, RotateCcw, Wand2 } from "lucide-react";
import { useVizRuntime, vizNumber, vizRecord, vizArray,
```

```
/**
```

```
 * Тема 7 «Графы. Планарные. Покраска».
```

```
 * Жадная раскраска планарного графа: кликни цвет, потом
```

```
 * Компилятор читает v (текущая вершина), color[] (массив
```

```
 */
```

```
interface V {
  id: number;
  label: string;
  x: number;
  y: number;
}
```

```

const color: number[] = NODES.map(() => -1);
for (let v = 0; v < NODES.length; v++) {
  const used = new Set(ADJ[v].map((u) => color[u]).filter(u => color[u] !== -1));
  let c = 0;
  while (used.has(c)) c++;
  color[v] = c;
}
return color;
}

```

```

export function ColoringViz() {
  const [colors, setColors] = useState<number[]>(NODES.map(() => -1));
  const [brush, setBrush] = useState(0);

```

// — Синхронизация с Python-компилятором.

```

const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveVLabel = vizString(vars?.v);
const liveVNum = vizNumber(vars?.v);
const liveColorRec = vizRecord(vars?.color) ?? vizRecord({});
const liveColorArr = vizArray(vars?.color) ?? vizArray([]);
const liveK = vizNumber(vars?.k);

```

```

const liveColors: number[] | null = (() => {
  if (liveColorArr) {
    const arr = liveColorArr.map((c) => vizNumber(c));
    if (arr.every((c) => c === null || c >= -1)) {
      const padded = [...arr.map((c) => c ?? -1)];
      while (padded.length < NODES.length) padded.push(-1);
      return padded.slice(0, NODES.length);
    }
    return null;
  }
  if (liveColorRec) {
    return NODES.map((n) => {
      const byLabel = liveColorRec[n.label];
      const byId = liveColorRec[String(n.id)];
      const raw = byLabel !== undefined ? byLabel : byId;

```

```

    const next = [...colors];
    next[id] = next[id] === brush ? -1 : brush;
    setColors(next);
  });

  return (
    <div className="bg-slate-800/90 p-5 rounded-2xl border">
      <div className="flex flex-wrap items-center justify-center">
        <div>
          <h3 className="text-lg font-bold text-white">
            <p className="text-xs text-slate-400">
              Смежные вершины не должны совпадать по цветам
            </p>
          </div>
          <div className="flex items-center gap-2">
            <button
              onClick={() => setColors(greedy())}
              className="flex items-center gap-1 bg-indigo-500 text-white"
            >
              <Wand2 className="w-3.5 h-3.5" /> Жадная раскраска
            </button>
            <button
              onClick={() => setColors(NODES.map((u, v) => -1))}
              className="flex items-center gap-1 bg-slate-500 text-white"
            >
              <RotateCcw className="w-3.5 h-3.5" /> Сброс
            </button>
          </div>
        </div>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr] gap-5">
        <div className="bg-slate-950 rounded-xl border border-slate-800 p-5">
          <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,#475569_1px)]" />
          <svg className="w-full h-[340px] relative z-10" />
          {EDGES.map(([u, v], i) => {
            <line key={i} x1={NODES[u].x} y1={NODES[u].y} x2={NODES[v].x} y2={NODES[v].y}
              stroke="#475569" strokeWidth={2.2} />
          })}
        </div>
      </div>
    </div>
  );
}

```

```

    <div className="space-y-3">
      <div className="bg-slate-950 rounded-xl border-2 border-slate-800">
        <div className="text-[10px] uppercase tracking-wider">
          <Palette className="w-3 h-3" /> Кисть
        </div>
        <div className="grid grid-cols-4 gap-2">
          {COLORS.map(({c, i}) => (
            <button
              key={c.name}
              onClick={() => setBrush(i)}
              title={c.name}
              className={`h-8 rounded-lg border-2 text-white
                opacity-70 hover:opacity-100`}
              style={{ backgroundColor: c.hex }}
            />
          ))}
        </div>
      </div>
      <div className="bg-slate-950 rounded-xl border-2 border-slate-800">
        <div className="text-[10px] uppercase tracking-wider">
          <div className={`text-2xl font-extrabold font-mono`}
            {compilerLinked && liveK !== null ? liveK}
          </div>
          <div className="text-[10px] text-slate-500">
            </div>
        </div>
        <div className="bg-slate-900/60 rounded-xl border-2 border-slate-800">
          <b className="text-slate-200">Жадный алгоритм Дейкстры</b>
          

На этом графе жадная раскраска даёт 4 цвета  
6 цветов (в планарном всегда есть вершина с  
степенью не более 3).


        </div>
      </div>
    </div>
  );
}

```

```
-----
  { id: "chess", name: "обобщённые шахматы", cls: "EXPTIME" },
];
```

```
const REDUCTIONS: [string, string, string][] = [
  ["sat", "vc", "3-SAT ≤p VC"],
  ["sat", "ham", "3-SAT ≤p Гамильтон"],
  ["vc", "knap", "VC ≤p Рюкзак"],
];
```

```
const CLASS_META: Record<ClassId, { title: string; color: string; note: string }> = {
  P: { title: "P", color: "#34d399", note: "решаются за полиномиальное время" },
  NP: { title: "NP", color: "#fbbf24", note: "решение проверяется за полиномиальное время" },
  PSPACE: { title: "PSPACE", color: "#818cf8", note: "решаются за полиномиальную память" },
  EXPTIME: { title: "EXPTIME", color: "#f472b6", note: "решаются за экспоненциальное время" },
};
```

```
const CLASS_INFO: Record<ClassId, string> = {
  P: "P — класс задач, у которых ЕСТЬ быстрый (полиномиальный) алгоритм",
  NP: "NP — задачи, у которых быстрой проверки ПРОЧЕСТЬ решение достаточно. Р ≤ NP: нашёл быстро → и проверяй быстро.",
  PSPACE: "PSPACE — хватает полиномиальной ПАМЯТИ (времена не считаются).",
  EXPTIME: "EXPTIME — есть алгоритм за 2^(poly n). Обобщение PSPACE.",
};
```

```
const RECT: Record<ClassId, { x: number; y: number; w: number; h: number }> = {
  P: { x: 40, y: 90, w: 220, h: 210 },
  NP: { x: 30, y: 40, w: 400, h: 260 },
  PSPACE: { x: 15, y: 20, w: 440, h: 320 },
  EXPTIME: { x: 5, y: 5, w: 470, h: 360 },
};
```

```
export function ComplexityClassesViz() {
  const [selected, setSelected] = useState<ClassId | null>(null);
  const [selectedProblem, setSelectedProblem] = useState<string | null>(null);
```

```
// — Синхронизация с Python-компилятором: cls — имя класса
```

```
    return "Кликни на класс или задачу. «A ≤p B» = «A не
```

```
  }}());

  const reset = () => {
    setSelected(null);
    setSelectedProblem(null);
  };

```

```
  return (
    <div className="bg-slate-800/90 p-5 rounded-2xl border" >
      <div className="flex flex-wrap items-center justify-center" >
        <div>
          <h3 className="text-lg font-bold text-white">
            <p className="text-xs text-slate-400">Стрелки
          </p>
        </div>
        <button
          onClick={reset}
          className="flex items-center gap-1 bg-slate-700" >
          <RotateCcw className="w-3.5 h-3.5" /> Сброс
        </button>
      </div>

```

```
    <div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr]" >
      <div className="bg-slate-950 rounded-xl border" >
        <svg className="w-full" viewBox="0 0 480 380">
          {[["EXPTIME", "PSPACE", "NP", "P"] as Class[]}
          const r = RECT[cls];
          const dim = activeCls !== null && activeCls === cls ? dim : null;
          return (
            <g key={cls} onClick={() => setSelected(cls)} >
              <rect
                x={r.x} y={r.y} width={r.w} height={r.h}
                fill={CLASS_META[cls].color + (dim ? " stroke:#fff" : "")}
                stroke={CLASS_META[cls].color}
                strokeWidth={activeCls === cls ? 3.5 : 1}
                opacity={dim ? 0.45 : 1}
              />
            </g>
          );
        </svg>
      </div>
    </div>

```

```

        <rect
            x={p.x - 46} y={p.y - 13} width={92}
            fill={hit ? "#c2410c" : p.npCompleted}
            stroke={hit ? "#fb923c" : CLASS_META[cls].stroke}
            strokeWidth={hit ? 2.5 : 1.2}
        />
        {p.name.split("\n").map((line, li, arr) =>
            <text key={li} x={p.x} y={p.y + 4 + li * 1.2}
                fontWeight="600">
                    {line}
                </text>
            )}}
        </g>
    );
    }
}
</svg>
</div>

<div className="space-y-3">
    <div className="grid grid-cols-2 gap-2">
        {(Object.keys(CLASS_META) as ClassId[]).map(cls =>
            <button
                key={cls}
                onClick={() => setSelected(selected === cls ? null : cls)}
                className={`rounded-lg border px-3 py-2
                    ${activeCls === cls ? "text-white" : "text-black"}
                `}
                style={{
                    borderColor: CLASS_META[cls].color,
                    backgroundColor: activeCls === cls ? CLASS_META[cls].color : null
                }}
            >
                {CLASS_META[cls].title}
                <div className="text-[9px] font-normal" style={
                    {cls === "P" ? "быстро решаем" : cls === "M" ? "математика" : "физика"}
                }>
                </div>
            </button>
        )}
    </div>

```

```

    label: string;
    x: number;
    y: number;
}

// Демо-граф: три «острова» по 3–4 вершины.
const NODES: V[] = [
  { id: 0, label: "A", x: 60, y: 60 },
  { id: 1, label: "B", x: 170, y: 30 },
  { id: 2, label: "C", x: 150, y: 130 },
  { id: 3, label: "D", x: 40, y: 160 },
  { id: 4, label: "E", x: 300, y: 60 },
  { id: 5, label: "F", x: 410, y: 90 },
  { id: 6, label: "G", x: 330, y: 160 },
  { id: 7, label: "H", x: 90, y: 260 },
  { id: 8, label: "I", x: 200, y: 240 },
  { id: 9, label: "J", x: 330, y: 270 },
];

const EDGES: [number, number][] = [
  [0, 1], [1, 2], [2, 3], [3, 0], [0, 2], // остров 1: A B C
  [4, 5], [5, 6], [6, 4], // остров 2: E F G (цикл)
  [7, 8], [8, 9], // остров 3: H I J
];

const ADJ: number[][] = (() => {
  const adj: number[][] = NODES.map(() => []);
  EDGES.forEach(([u, v]) => {
    adj[u].push(v);
    adj[v].push(u);
  });
  return adj;
})();

const COMP_COLORS = ["#6366f1", "#f59e0b", "#10b981", "#ff7f50"];

interface StepFrame {
  comp: (number | null)[];

```



```

        }
    }
    }
    push({ queue: [], current: null, scanning: null, log:
  }
  push({ queue: [], current: null, scanning: null, log:
    });
  return frames;
}

```

```
const FRAMES = buildFrames();
```

```

export function ComponentsViz() {
  const [step, setStep] = useState(0);
  const [playing, setPlaying] = useState(false);

  useEffect(() => {
    if (!playing) return;
    if (step >= FRAMES.length - 1) {
      setPlaying(false);
      return;
    }
    const t = window.setTimeout(() => setStep((s) => Ma
    return () => window.clearTimeout(t);
  }, [playing, step]);

```

```

// — Синхронизация с Python-компилятором: v — текуща
// или список меток, q — очередь, count — число компо
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveVLabel = vizString(vars?.v);
const liveVNum = vizNumber(vars?.v);
const liveCompRec = vizRecord(vars?.comp);
const liveCompArr = vizArray(vars?.comp);
const liveQ = vizArray(vars?.q);
const liveCount = vizNumber(vars?.count);

```

```
const liveCompByLabel = useMemo(() => {
```

```

const shownCurrent = compilerLinked ? liveCurrent : f
const shownScanning = compilerLinked ? null : frame.s
const shownQueue: number[] = liveQueueLabels
  ? liveQueueLabels
    .map((lab) => NODES.findIndex((n) => n.label === lab))
    .filter((i) => i >= 0)
  : frame.queue;

return (
  <div className="bg-slate-800/90 p-5 rounded-2xl border" >
    <div className="flex flex-wrap items-center justify-center" >
      <div>
        <h3 className="text-lg font-bold text-white">
          <p className="text-xs text-slate-400">
            Каждой вершине присваивается номер компонент
          </p>
        </div>
        <div className="flex items-center gap-2">
          <button
            onClick={() => { setPlaying(false); setStep(0); }}
            className="flex items-center gap-1 bg-slate-700 hover:bg-slate-600" >
            <RotateCcw className="w-3.5 h-3.5" /> Сброс
          </button>
          <button
            onClick={() => setStep((s) => Math.max(0, s + 1))}
            disabled={step === 0}
            className="bg-slate-700 hover:bg-slate-600" >
            <RotateCw className="w-3.5 h-3.5" /> Шаг
          </button>
          <button
            onClick={() => setPlaying((p) => !p)}
            className={`flex items-center gap-1 px-3 py-2 ${playing ? 'bg-emerald-600 hover:bg-emerald-500' : 'bg-slate-700 hover:bg-slate-600'}`} >
            {playing ? <Pause className="w-3.5 h-3.5" /> : <Play className="w-3.5 h-3.5" />
          </button>
        </div>
      </div>
    </div>
  </div>
)

```

```

        <g key={n.id}>
          {(isCurrent || isScanned) && (
            <circle cx={n.x} cy={n.y} r={18} fill="white" stroke="black" stroke-width="2" />
            <circle cx={n.x} cy={n.y} r={13} fill="white" stroke="black" stroke-width="2" />
            <text x={n.x} y={n.y + 4} textAnchor="center" font-size="10px" font-weight="bold">{n.label}</text>
            <text x={n.x} y={n.y - 18} textAnchor="center" font-size="10px" font-weight="normal">{c === null ? "-" : `c${c}`}</text>
          )}
        </g>
      );
    }
  }
</svg>
<div className="absolute top-2 left-2 bg-slate-950 rounded-xl border-2 border-slate-800 z-20">
  Компонент: <span className="font-bold text-white">{n.label}</span>
</div>
{compilerLinked && (
  <div className="absolute top-2 right-2 bg-emerald-300 rounded-xl border-2 border-emerald-400 z-20">
    синхронизация с Python
  </div>
)}
</div>

<div className="space-y-3">
  <div className="bg-slate-950 rounded-xl border-2 border-slate-800 p-4">
    <div className="text-[10px] uppercase tracking-wider">
      <div className="flex flex-wrap gap-1 min-h-[100px]">
        {shownQueue.length === 0 && <span className="text-slate-500">Нет данных</span>}
        {shownQueue.map((q, i) => (
          <span key={` ${q} - ${i} `} className="px-2 py-1 text-white font-mono">
            {NODES[q].label}
          </span>
        ))}
      </div>
    </div>
  </div>

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    ", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        ", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        ", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
  { line: "            // ЭТО МОСТ!", highlight: "normal" },
  { line: "            ", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
}

```

```

    },
    {
        currentNode: 6, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
        activeEdge: [1,6], backEdge: null,
        log: "Из В идём в G. tin[G]=3, low[G]=3.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
        activeEdge: null, backEdge: null,
        log: "У G нет соседей (кроме В). Возврат. low[G]=3 : ",
        activeCodeLine: [13,14,15]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
        activeEdge: [1,2], backEdge: null,
        log: "Из В идём в С. tin[C]=4, low[C]=4.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа А (уже посещён)! Обратное ребро",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
        activeEdge: [2,3], backEdge: null,
        log: "Идём C→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],

```



```

<button onClick={() => { setDfsAuto(false); s
  disabled={dfsIdx === DFS_STEPS.length-1}
  className="p-1.5 hover:bg-slate-700 rounded
  <SkipForward className="h-3.5 w-3.5" />
</button>
<button onClick={() => { setDfsAuto(false); s
  className="p-1.5 hover:bg-slate-700 rounded
  <RefreshCw className="h-3.5 w-3.5" />
</button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
  {/* Graph visualization */}
  <div className="md:col-span-3 bg-slate-950 roun
    <div className="absolute inset-0 bg-[radial-g
      >
    <svg className="w-full h-[280px] z-10 relative
      {GRAPH_EDGES.map(e => {
        const u = GRAPH_NODES.find(n => n.id ===
        const v = GRAPH_NODES.find(n => n.id ===
        const bridge = step.bridges.includes(e.key
        const activeTree = step.activeEdge && (
          (step.activeEdge[0]===e.u && step.active
          (step.activeEdge[0]===e.v && step.active
        ));
        const activeBack = step.backEdge && (
          (step.backEdge[0]===e.u && step.backEdge
          (step.backEdge[0]===e.v && step.backEdge
        ));
        let sc = "#475569", sw = 2, dash = "none"
        if (bridge) { sc = "#eab308"; sw = 4; }
        else if (activeTree) { sc = "#10b981"; sw
        else if (activeBack) { sc = "#f43f5e"; sw
        else if (step.visitedIndices.includes(e.u
        return <line key={e.key} x1={u.x} y1={u.y
          stroke={sc} strokeWidth={sw} strokeDash
          className="transition-all duration-500"

```

```

    { /* Code panel + stack */ }
    <div className="md:col-span-2 space-y-3">
      { /* Code panel */ }
      <div className="bg-slate-950 rounded-lg border" style={{ border: '1px solid #333' }}>
        <div className="bg-slate-900 px-3 py-1.5 text-slate-400">
          <div className="p-2 overflow-x-auto">
            <pre className="text-[10px] font-mono leading-relaxed">
              {CODE_LINES.map(({cl, i}) => {
                const hl = step.activeCodeLine.includes(cl)
                return <div key={i}
                  className={`px-1 transition-all duration-200
                    ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}
                  `}>
                <span className="text-slate-600 mr-1">{i}</span>
                {cl.line || ' '}</div>
              })}
            </pre>
          </div>
        </div>
      </div>

    { /* Stack */ }
    <div className="bg-slate-900/50 rounded-xl border" style={{ border: '1px solid #333' }}>
      <div className="text-[10px] font-bold text-slate-400">
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.stack.length}</span>
      </div>
      <div className="flex flex-col-reverse gap-1">
        {step.stack.length === 0
          ? <div className="text-[10px] text-slate-400">Пустой стек</div>
          : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0.5 transition-all duration-200
                ${i === step.stack.length - 1 ? "bg-emerald-600/20 text-emerald-400" : "bg-slate-900 text-slate-400"}
              `}>Вершина {GRAPH_NODES.find(n=>n.id === id).name}</div>
            })}
      </div>
    </div>
  </div>
)

```



```

    visited: { [key: string]: boolean };
    previous: { [key: string]: string | null };
    log: string;
    activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
    { u: 'B', v: 'G', w: 4 },
    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];

  const adjList: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (adjList[n.id] = []));
  edges.forEach(e => {
    adjList[e.u].push({ v: e.v, w: e.w });
  });

  const codeLines = [

```

```

    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: `  Выбираем вершину ${u} с мин. непосещённой вершиной`,
    activeCodeLine: 5,
  });

  // Граф неориентированный: при релаксации смотрим соседей
  const neighbors = edges.filter(e => e.u === u || e.v === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: `  Смотрим соседа ${v}. Текущее расстояние: ${dist[u] + edge.w}.`,
      activeCodeLine: 8,
    });

    if (dist[u] + edge.w < dist[v]) {
      dist[v] = dist[u] + edge.w;
      prev[v] = u;
      simulationSteps.push({
        currentNode: u, checkingEdge: { u, v },
        distances: { ...dist }, visited: { ...visited },
        log: `  Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]}.`,
        activeCodeLine: 9,
      });
    }
  }
}

```

```

    checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null
    distances: Object.fromEntries(nodes.map((node) => [node, 0]))
    previous: Object.fromEntries(nodes.map((node) => [node, null]))
    visited: Object.fromEntries(nodes.map((node) => [node, false]))
    log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    {/* Шапка симулятора */}
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-indigo-600 text-white" >
          <span className="text-2xl" > </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <p className="text-sm text-indigo-300" >Полный код
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center gap-2" >
      <button
        onClick={() => { setIsPlaying(false); setCurrentStep(0); }}
        className="flex items-center gap-1 bg-slate-800 text-white p-2
          ion-colors"
      >
        <span>Сброс</span>
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1 px-4 py-2
          ${isPlaying ? 'bg-amber-600 hover:bg-amber-500' : 'bg-slate-800 hover:bg-slate-700'}
          text-white`
        >
        {isPlaying ? 'Пауза' : '▶ Пуск'}
      </button>
    </div>
  </div>
);

```

```

        {currentStep.checkingEdge.u === edge.u &&
        currentStep.currentNode === edge.v}

const isOptimal = currentStep.previous[edge.u] === edge.v

let marker = 'url(#arrow-dh-normal)';
if (isChecking) marker = 'url(#arrow-dh-a';
else if (isOptimal) marker = 'url(#arrow-dh-n');

return (
  <g key={idx}>
    <line
      x1={uNode.x} y1={uNode.y}
      x2={vNode.x} y2={vNode.y}
      stroke={isChecking ? '#f59e0b' : isOptimal ? '#1e293b'}
      strokeWidth={isChecking ? 5 : isOptimal ? 1}
      markerEnd={marker}
      className="transition-all duration-300"
      strokeDasharray={isChecking ? '4,4' : ''}
    />
    <circle
      cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
      r="12" fill="#1e293b"
      stroke={isChecking ? '#f59e0b' : '#1e293b'}
    />
    <text
      x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)}
      fill={isChecking ? '#f59e0b' : '#f8d7da'}
      fontSize="11" fontWeight="bold" textAnchor="center"
    >
      {edge.w}
    </text>
  </g>
);
}})
}}

```

```

{/* Вершины */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node

```

```

        const isCurrentNode = currentStep.currentNode === u
        return (
          <div key={u} className={`p-3 rounded-lg ${
            isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
          }`} >
            <div className="flex items-center gap-2">
              <span className={`px-2 py-0.5 rounded ${
                isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-300'
              }`} >
                {u}
              </span>
              <span className="text-slate-400">→</span>
            </div>
            <div className="flex flex-wrap gap-2">
              {adjList[u].map((edge, idx) => {
                const isCheckingThis = currentStep.currentNode === edge.v
                return (
                  <span key={idx} className={`px-2 py-0.5 rounded ${
                    isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-300'
                  }`} >
                    {idx}
                  </span>
                  <span>{edge.v}</span>
                  <span className={isCheckingThis ? 'text-amber-900/60' : 'text-slate-300'}>
                    {edge.w}
                  </span>
                )
              })}
            </div>
          </div>
        )
      })
    </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10" >

```

```

        </div>
      </div>

      {/* Код алгоритма */}
      <div className="w-full lg:w-1/2 bg-slate-900/60" >
        <div>
          <h4 className="text-lg font-bold text-slate-100">Алгоритм
          <div className="bg-slate-950/80 rounded-lg p-4" >
            {codeLines.map((item) => {
              const isActive = currentStep.activeCodeLine === item.line
              return (
                <div key={item.line} className={`py-1 px-2 ${
                  isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-100'
                }`} >
                  <span className="text-slate-600 select-none">{item.line}</span>
                  <span className="whitespace-pre flex-1">{item.code}</span>
                </div>
              )
            })}
          </div>
        </div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 41/92: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '../vizRuntime';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```

        desc: `Вызов fibМемо(${n}). Проверяем наличие в
        codeLine: 2,
        memoState: { ...memoMap }
    });

    if (n in memoMap) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'memo_hit',
            desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
            codeLine: 2,
            memoState: { ...memoMap }
        });
        return memoMap[n];
    }

    if (n <= 2) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'base_case',
            desc: `Базовый случай: n <= 2 (n=${n}). Возвр
            codeLine: 3,
            memoState: { ...memoMap }
        });
        return 1;
    }

    generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
        codeLine: 4,
        memoState: { ...memoMap }
    });

```

```

    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
  ? Object.fromEntries(Object.entries(liveMemo).flatMap(
    ([key, value]) => {
      const number = vizNumber(value);
      return number === null ? [] : [[Number(key), number]];
    }
  ))
  : baseStep?.memoState || {};
const currentStep: DPStep | null = baseStep && compilerLinked
  ? {
    ...baseStep,
    n: liveK ?? baseStep.n,
    memoState,
    desc: `Компилятор: k=${liveK ?? "-"}; таблица memo: ${memoState}`
  }
  : baseStep;

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800" >
    { /* Header & Controls */ }
    <div className="flex flex-col lg:flex-row lg:items-center" >
      <div>
        <div className="flex items-center gap-3 mb-2" >
          <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-white" >
            Компилятор
          </h3>
          <p className="text-sm text-slate-400" >
            k: {liveK ?? "-"}; таблица memo: {memoState}
          </p>
        </div>
      </div>
    </div>
  </div>
);

```



```

        className={`flex items-center gap-1.5 px-4 py-2`}
        isPlaying
        ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
        : 'bg-emerald-600 text-white hover:bg-emerald-700'
      `}
    >
      {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
      {isPlaying ? 'Пауза' : 'Пуск'}
    </button>
    <button
      onClick={() => { setIsPlaying(false); setStepIndex(stepIndex + 1); }}
      disabled={currentStepIndex === steps.length - 1}
      className="p-2 text-slate-400 hover:text-slate-300"
      title="Шаг вперед"
    >
      <ChevronRight className="w-5 h-5" />
    </button>
  </div>
</div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-lg">
  <h4 className="text-xs font-bold uppercase tracking-wide">Dynamic Programming
  <div className="flex flex-wrap items-center gap-2">
    {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
      const isCached = num in memoState || num <= 2;
      const val = num <= 2 ? 1 : memoState[num];
      const isCurrent = currentStep?.n === num;

      return (
        <div
          key={num}
          className={`flex flex-col items-center justify-center gap-2 p-2 rounded-sm`}
          isCurrent
          ? 'bg-emerald-950 border-emerald-500'
          : isCached
          ? 'bg-slate-900 border-emerald-500/50'
          : 'bg-slate-950 border-slate-700'
        >
          {num}
          {isCurrent ? ' (current)' : isCached ? ' (cached)' : ''}
        </div>
      );
    })}
  </div>
</div>

```

```

    </button>
    <button
      onClick={() => setActiveTab('code')}
      className={`flex items-center gap-2 px-6 py-3
        activeTab === 'code'
          ? 'border-emerald-500 text-emerald-400 bg-slate-900'
          : 'border-transparent text-slate-400 hover:text-slate-300'
      }`>
      >
      <Code className="w-4 h-4" />
      Интерактивный код
    </button>
  </div>

```

```

{ /* Tab Content */ }
<div className="min-h-[320px] flex flex-col justify-between"
  {activeTab === 'table' && {
    <div className="bg-slate-950 p-6 rounded-2xl border border-slate-800"
      <h5 className="font-bold text-white text-sm"
        <span className="text-emerald-400"> {steps.length} шагов
      </span>
      </h5>
      <p className="text-sm text-slate-400">
        В классической рекурсии для N={targetN} необходимо вычислить
        вычисленное значение в таблице, мы моментами не можем
      </p>
      <div className="p-4 bg-slate-900 rounded-xl border border-slate-800"
        <span>Всего шагов симуляции: {steps.length}</span>
        <span className="text-emerald-400 font-bold"> {steps[targetN]}
      </span>
      </div>
    </div>
  )}

```

```

{activeTab === 'code' && {
  <div className="bg-slate-950 rounded-2xl border border-slate-800"
    <div className="bg-slate-900 px-6 py-3 border border-slate-800"
      <span className="text-xs font-bold font-mono"> {code}</span>
      <span className="text-xs text-emerald-400"> {steps[targetN]}
    </span>
  </div>
}

```

```

        ></div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 42/92: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useMemo, useState } from "react";
import { Undo } from "lucide-react";
import { useVizRuntime, vizArray, vizString } from "../viz";

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  // — Синхронизация с Python-компилятором: path — пос.
  // вершин (["A", "B", "C"...]), degrees/deg — словарь с
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const livePathLabels = vizArray(vars?.path)?.map((x) =>

```

```

    });

    const clickC1 = (vId: number) => {
      if (c1Done && c1Path.length > 0) return;
      if (c1Path.length === 0) {
        setC1Path([vId]);
        setC1Msg("Старт! Кликай соседние вершины, чтобы пр
        return;
      }
      const last = c1Path[c1Path.length - 1];
      const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
      if (!edge) {
        setC1Msg("Нет ребра между этими вершинами! Выбери
        return;
      }
      const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
      if (c1VisitedEdges.includes(key)) {
        setC1Msg("Это ребро уже пройдено! Эйлер не прощае
        setC1Done(true); return;
      }
      const nextEdges = [...c1VisitedEdges, key];
      const nextPath = [...c1Path, vId];
      setC1VisitedEdges(nextEdges);
      setC1Path(nextPath);

      if (nextEdges.length === C1_EDGES.length) {
        const startDegree = C1_EDGES.filter(e => e.u === c1Path[0]).length;
        const endDegree = C1_EDGES.filter(e => e.v === c1Path[c1Path.length - 1]).length;
        const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
        if (isCycle && nextPath[0] === vId) {
          setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф
        } else {
          setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
        }
        setC1Done(true);
      } else {
        setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
      }
    }
  }

```

```

        </g>;
      }}}}
    </svg>
    <div className="absolute top-2 left-2 bg-slate-100" >
      Ребёнок: {shownVisited.length}/{C1_EDGES.length}
      {compilerLinked && <span className="ml-2 text-slate-500">
    </div>
    <div className="absolute bottom-2 left-2 bg-slate-100" >
      Степени: {degrees.map((d) => `${d.label}:${d.degree}`)}
    </div>
  </div>

  <div className={`p-3 rounded-xl border text-sm ${
    c1Done && c1VisitedEdges.length === C1_EDGES.length
      ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
      : c1Done
      ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
      : "bg-slate-900/50 border-slate-700 text-slate-50"
  }`}>
    {c1Msg}
  </div>
</div>
);
}

```

ФАЙЛ 43/92: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
  id: number;
  question: string;
  options: string[];
  correctIndex: number;
  type: 'multiple-choice' | 'true-false' | 'text';
}

```

```

    'Он использует меньше памяти благодаря коммунистиче
    'Он умеет работать с отрицательными весами без ци
    'Он позволяет выполнять шаги слияния колхозов (ко
    'Он был разработан в Токио и поэтому самый быстрый
  ],
  correctAnswer: 2,
  explanation: 'Абсолютно верно! В Боровке каждая ком
    делить вычисления на GPU или многопроцессорных кл
},
{
  id: 4,
  question: 'В чем главная суть Динамического програм
  options: [
    'Писать код очень быстро, динамично нажимая на кл
    'Запоминать (кешировать) результаты уже решенных
    'Использовать динамическую типизацию как в Python
    'Постоянно менять требования к проекту в процессе
  ],
  correctAnswer: 1,
  explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
  id: 5,
  question: 'Какая структура данных идеально помогает
  options: [
    'СНМ (Система непересекающихся множеств / Disjoin
    'Стек (LIFO).',
    'Красно-черное дерево.',
    'Хэш-таблица со списками коллизий.'
  ],
  correctAnswer: 0,
  explanation: 'Блестяще! СНМ (DSU) позволяет за прак
    '
  }
];

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<num

```

```

    <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-600 shadow-xl shadow-indigo-500/30">
      <Award className="w-10 h-10 text-white" />
    </div>
    <h3 className="text-3xl font-bold text-white mb-6">Ты настоящий сеньор-раскрасчик?</h3>
    <p className="text-slate-400 text-sm mb-6">Ты набрал {score} баллов из {quizQuestions.length} возможных. Это означает:

    <div className="bg-slate-950 border border-slate-800 p-6">
      <p className="text-slate-400 text-sm uppercase font-medium">Оценки</p>
      <p className="text-5xl font-black text-indigo-400">{score}</p>
      <p className="text-slate-300 text-sm">
        {score === quizQuestions.length
          ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
          : score >= 3
            ? ' Отличный результат! Главные концепции ты освоил!'
            : ' Стоит повторить главы пособия и еще раз пройти тест.'
        }
      </p>
    </div>

    <button
      onClick={handleRestart}
      className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
    >
      <RotateCcw className="w-5 h-5" />
      <span>Пройти тест заново</span>
    </button>
  </div>
);
}

return (
  <div className="bg-slate-900/90 border border-slate-800 p-6">
    <div className="flex items-center justify-between">
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-4 py-2 rounded-lg">Экспресс-тест</span>
      </div>
    </div>
  </div>
);

```

```

        ) : isAnswered && isSelected && !isCo
        <XCircle className="w-6 h-6 text-ro
    ) : (
        <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
        {String.fromCharCode(65 + idx)}
        </div>
    )}
    </div>
    <div className="flex-1 text-sm md:text-l
        {option}
    </div>
</div>
    );
    }}}
</div>
</div>

{isAnswered && (
    <div className="bg-slate-950 border border-slat
        <div className="flex items-center gap-2 mb-2
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
        <p className="text-slate-300 text-sm leading-
            {currentQ.explanation}
        </p>
    </div>
)}

<div className="flex justify-end pt-4 border-t bo
    <button
        onClick={handleNext}
        disabled={!isAnswered}
        className={
            "px-8 py-3.5 font-bold rounded-xl transition
            (!isAnswered
                ? "bg-slate-800 text-slate-500 border bor

```



```

    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

```

```

const initialMatrix = [
  [0, 4, 999, 2, 999, 999, 999],
  [999, 0, 3, 999, 999, 3, 999],
  [999, 999, 0, 999, 2, 999, 999],
  [999, 999, 999, 0, 4, 2, 999],
  [999, 999, 999, 999, 0, 999, 1],
  [999, 999, 999, 999, 999, 0, 5],
  [999, 1, 999, 999, 999, 999, 0]
];

```

```

const adjList: { [key: number]: { v: number; w: number }[] } = {
  0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
  1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
  2: [{ v: 4, w: 2 }],
  3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
  4: [{ v: 6, w: 1 }],
  5: [{ v: 6, w: 5 }],
  6: [{ v: 1, w: 1 }],
};

```

```

const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V)' },
  { line: 2, text: '  dist = copy(matrix)' },
  { line: 3, text: '  for k from 0 to V - 1: // Пос' },
  { line: 4, text: '    for i from 0 to V - 1:' },
  { line: 5, text: '      for j from 0 to V - 1' },
  { line: 6, text: '        if dist[i][k] + d' },
  { line: 7, text: '          dist[i][j] = ' },
  { line: 8, text: '  return dist' },
];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];

```

```

        activeCodeLine: 8,
    });

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const rawMatrix = vizArray(vars?.d) ?? vizArray(vars?.d);
const liveMatrix = rawMatrix
    ? baseStep.matrix.map((row, i) => {
        const values = vizArray(rawMatrix[i]);
        return row.map((_ , j) => vizNumber(values?.[j]))
    })
    : undefined;

const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;
const currentStep: Step = compilerLinked
    ? {
        ...baseStep,
        k: liveK ?? -1,
        i: liveI ?? -1,
        j: liveJ ?? -1,
    }
    : baseStep;

```

```

<svg className="w-full h-auto max-h-[500px]" >
  <defs>
    <marker id="arrow-fw-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
    <marker id="arrow-fw-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
    <marker id="arrow-fw-via" markerWidth="6"
    th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
  </defs>
  {Object.keys(adjList).flatMap((uStr) => {
    const u = parseInt(uStr);
    return adjList[u].map((edge) => {
      const uNode = nodesSvg.find((n) => n.id
      const vNode = nodesSvg.find((n) => n.id
      const isTarget = currentStep.i === u &&
      const isVia = (currentStep.i === u && c
      let strokeColor = '#475569'; let marker
      if (isTarget) { strokeColor = '#10b981'
      else if (isVia) { strokeColor = '#818cf8
      return (
        <g key={` ${u} - ${edge.v} `}>
          <line x1={uNode.x} y1={uNode.y} x2=
          markerEnd={marker} strokeDasharray={isTarg
          <circle cx={({uNode.x + vNode.x) / 2
            isVia ? '#818cf8' : '#64748b'} strokeWidth
          <text x={({uNode.x + vNode.x) / 2} y=
            '8fafc'} fontSize="10" fontWeight="bold" tex
          </g>
        );
      });
    });
  })}
  {nodesSvg.map((node) => {
    const isK = currentStep.k === node.id;
    const isI = currentStep.i === node.id;
    const isJ = currentStep.j === node.id;
    let fill = '#1e293b'; let stroke = '#64748b';
    if (isK) { fill = '#4f46e5'; stroke = '#a

```

```

rentStep.j === edge.v);
    return (
      <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
      {edge.v} ({edge.w})
    </span>
  )
  )}}
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  /* Матрица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-950">
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border">
        <thead><tr className="bg-rose-950/40 text-white">
          <th className="p-2 border-r border-rose-950">
            {nodeNames.map((n, idx) => <th key={idx}
-400 font-bold' : ''}`}>{n.split(' ')[0]} {
          </tr></thead>
          <tbody className="text-xs font-mono">
            {currentStep.matrix.map((row, i) => (
              <tr key={i} className="border-b border-rose-950">
                <td className={`p-2 bg-rose-950/40 text-white
tStep.k === i ? 'text-amber-400' : ''}`}>
                  {nodeNames[i].split(' ')[0]} {i}
                </td>
                {row.map((val, j) => {
                  const isUpd = currentStep.i === i
                  const isVia = currentStep.k !== -

```

ФАЙЛ 45/92: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};
```

```

    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начинаем из ${start}, ray.from(vis)[]` });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Делаем шаг из ${v}, q=${q}, visitedNodes=${visitedNodes}[]` });
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, desc: `Визит в ${u}, backtrack=${backtrack}[], visitedNodes=${visitedNodes}[]` });
            }
        }
    }

    steps.push({ type: 'backtrack', node: '', desc: `Обратный шаг от ${start}, ray.from(vis)[]` });

    return steps;
}

```

```

export function GraphTraversalViz() {
    const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);

    const steps = useMemo(() => mode === 'dfs' ? generateStepsDFS(start, adj, vis) : generateStepsBFS(start, adj, vis), [mode, start, adj, vis]);
    useVizStepSync(stepIdx, setStepIdx, steps.length - 1);
}

```



```

        {/* Structure View (Stack / Queue) */}
        <div className="bg-slate-900 border border-emerald-500 p-2">
          <div className={`absolute -top-3 left-3 right-3 bottom-3
            ${mode === 'dfs' ? 'bg-slate-900 border border-emerald-500'}
            ${mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
          `}>
            <div className="flex flex-col gap-2">
              {step.queueOrStack && step.queueOrStack.map((item, idx) => {
                <div key={idx} className={`w-full p-2 text-emerald-200
                  ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                    ? `border-bottom-10px-rgba(245,158,11,0.2)` : ''}
                `}>
                  Узел: {item}
                  {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                    ? <span className="ml-2">Вызывает: {step.queueOrStack[idx + 1]}</span>
                    : ''}
                </div>
              })}
              {(!step.queueOrStack || step.queueOrStack.length === 0)
                ? <div className="text-slate-500">Пусто</div>
                : ''
              }
            </div>
          </div>
        </div>

        <div className="flex flex-col sm:flex-row gap-2">
          {/* Controls */}
          <div className="flex bg-slate-900 border border-emerald-500 p-2 text-slate-400 hover:text-white hover:background-color-slate-800">
            <button onClick={() => {setAutoPlay(!autoPlay)}}>
              {autoPlay ? 'Пауза' : 'Воспроизведение'}
            </button>
            <button onClick={() => {setStep(step + 1)}}>
              Следующий
            </button>
            <button onClick={() => {setStep(step - 1)}}>
              Предыдущий
            </button>
            <button onClick={() => {setStep(0)}}>
              Сброс
            </button>
            <button onClick={() => {setStep(step + 1)}}>
              Завершить
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>

```

import { useVizRuntime, vizArray, vizNumber } from '../'

```
interface Patient {  
  id: string;  
  name: string;  
  emoji: string;  
  symptom: string;  
  priority: number; // 1 to 99 (Severity)  
  animState: 'in' | 'idle' | 'winner' | 'out';  
}
```

// Max-heap helpers for Patients

```
function siftUp(arr: Patient[]): Patient[] {  
  const a = arr.map(x => ({ ...x }));  
  let idx = a.length - 1;  
  while (idx > 0) {  
    const parent = Math.floor((idx - 1) / 2);  
    if (a[parent].priority < a[idx].priority) {  
      [a[parent], a[idx]] = [a[idx], a[parent]];  
      idx = parent;  
    } else break;  
  }  
  return a;  
}
```

```
function siftDown(arr: Patient[]): Patient[] {  
  const a = arr.map(x => ({ ...x }));  
  const n = a.length;  
  let idx = 0;  
  while (true) {  
    let largest = idx;  
    const l = 2 * idx + 1;  
    const r = 2 * idx + 2;  
    if (l < n && a[l].priority > a[largest].priority) largest = l;  
    if (r < n && a[r].priority > a[largest].priority) largest = r;  
    if (largest !== idx) {  
      [a[largest], a[idx]] = [a[idx], a[largest]];  
      idx = largest;  
    }  
  }  
  return a;  
}
```

```

    });
    start.forEach((p, i) => {
      arr = heapInsert(arr, { id: `init${i}`, ...p, animS
    });
    return arr;
  })();

```

```
let uid = 300;
```

```

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>();
  const runtime = useVizRuntime();
  const liveHeap = vizArray(runtime?.variables?.h) ?? v
  const liveI = vizNumber(runtime?.variables?.i);
  const visualHeap: Patient[] = liveHeap
    ? liveHeap.map((value, index) => {
      const tuple = vizArray(value);
      const priority = vizNumber(tuple?.[0]) ?? vizNum
      return {
        id: `python-${index}`,
        name: tuple ? String(tuple[1] ?? value) : Str
        emoji: " ",
        symptom: `индекс ${index}`,
        priority,
        animState: "idle",
      };
    })
    : heap;
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(
  const [log, setLog] = useState<string>('');
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);
  const [healingPatient, setHealing] = useState<Patient

```

```
const addLog = (msg: string) => setLog(prev => [msg,
```

```
// --- INSERT: Новый пациент поступает в приемный пок
```

```

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
  };

const handleCustomInsert = (e: React.FormEvent) => {
  e.preventDefault();
  if (!customName.trim()) return;
  insertPatient(customName, customPriority);
  setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
  if (busy || heap.length === 0) {
    setShake(true);
    addLog('⚠ Нет пациентов для экстренной госпитализации');
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);
  const topPatient = heap[0];
  setHealing(topPatient);

  addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

  // Step 1: Root lights up as winner
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, winner: true } : x));

  setTimeout(() => {
    // Step 2: Root exits the tree, tree reorganizes
    setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, winner: false } : x));

    setTimeout(() => {
      // Run binary heap extraction
      setHeap(prev => {
        if (prev.length === 0) return [];
        const last = prev.pop();
        const root = prev[0];
        const newRoot = last;
        const newHeap = [...prev];
        newHeap[0] = newRoot;
        // Bubble down
        let i = 0;
        while (true) {
          const left = 2 * i + 1;
          const right = 2 * i + 2;
          let smallest = i;
          if (left < newHeap.length & newHeap[left].priority < newHeap[smallest].priority) {
            smallest = left;
          }
          if (right < newHeap.length & newHeap[right].priority < newHeap[smallest].priority) {
            smallest = right;
          }
          if (smallest === i) break;
          [newHeap[i], newHeap[smallest]] = [newHeap[smallest], newHeap[i]];
          i = smallest;
        }
        return newHeap;
      });
    }, 100);
  }, 100);
}, [heap, busy]);

```

```

        x2={`${lp.x}%`} y2={`${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  if (r < visualHeap.length) {
    const rp = nodePos(r);
    res.push(
      <line key={`er${idx}`}
        x1={`${p.x}%`} y1={`${p.y + 4}%`}
        x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  return res;
});

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-emerald-950/40 border border-e
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-emerald-400 te
          ty Queue)</h3>
        <p className="text-xs text-slate-300 mt-1 lea
          В реанимации не важно, кто пришел раньше, а
          остояние</b> (максимальный приоритет).
          Куча (Heap) автоматически перестраивает пац
          оказывался на самом верху дерева (в корне).
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex flex-col lg:flex-row items-s

```



```

        </div>
    );
    }}}
</div>

<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 round
    <div className="absolute top-4 left-4 text-[1
        Операционная койка
    </div>

<div className="flex-1 flex flex-col justify-
    {healingPatient ? (
        <div className="flex flex-col items-cente
            <div className="relative">
                <span className="text-6xl block anim-l
                <span className="absolute -top-4 -right
                <span className="absolute -bottom-2 -
            </div>
        <div>
            <h4 className="text-white font-black
                <span>Пациент: {healingPatient.name
            </h4>
            <p className="text-emerald-400 text-x
                Состояние: Стабилизация ({healingPa
            </p>
            <div className="flex justify-center g
                <span className="w-2 h-2 rounded-fu
                <Heart className="w-4 h-4 text-rose
            </div>
        </div>
    </div>
) : (
    <div className="text-center text-slate-60
        <span className="text-5xl mb-2 block">

```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — мож
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
  const [step, setStep] = useState(0);
```

```
  const paused = useContext(DemoPauseContext);
```

```
  const period = Math.round(ms * SPEED);
```

```
  useEffect(() => {
```

```
    if (steps <= 1 || paused) return;
```

```
    const t = setInterval(() => setStep((s) => (s + 1)),
```

```
    return () => clearInterval(t);
```

```
  }, [steps, period, paused]);
```

```
  return step;
```

```
}
```

```
export const C = {
```

```
  idle: "#334155",
```

```
  idleStroke: "#475569",
```

```
  text: "#e2e8f0",
```

```
  dim: "#94a3b8",
```

```
  active: "#6366f1",
```

```
  done: "#10b981",
```

```
  warn: "#f59e0b",
```

```

    </g>
  );

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);

```

ФАЙЛ 48/92: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 4], j: [9, 9], k: [3, 3] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;

```



```

const pos: Record<string, [number, number]> = {
  A: [26, 40], B: [62, 92], C: [26, 92],
  D: [120, 40], E: [120, 92],
  F: [190, 34], G: [232, 78], H: [178, 96],
};
const colors = [C.active, C.warn, C.done];
const step = useLoop(comps.length + 1, 900);
const compOf = (v: string) => comps.findIndex((e) => e === v);

return (
  <Svg caption={step === 0 ? "Граф распался на острова" : ""}
    {comps.flatMap((edges, ci) =>
      edges.map(([u, v], i) => (
        <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
      ))
    )}
    {Object.entries(pos).map(([k, [x, y]]) => {
      const ci = compOf(k);
      return <Node key={k} x={x} y={y} r={12} label={ci}
    })}
  </Svg>
);
};

```

```

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 40], B: [150, 40], C: [250, 40],
  const edges: [string, string, number][] = [
    ["A", "B", 1],
    ["B", "C", 1],
    ["A", "C", 1],
    ["A", "D", 1],
    ["B", "D", 1],
    ["C", "D", 1],
    ["D", "E", 1],
    ["E", "F", 1],
    ["F", "G", 1],
    ["G", "H", 1],
    ["H", "I", 1],
    ["I", "J", 1],
    ["J", "K", 1],
    ["K", "L", 1],
    ["L", "M", 1],
    ["M", "N", 1],
    ["N", "O", 1],
    ["O", "P", 1],
    ["P", "Q", 1],
    ["Q", "R", 1],
    ["R", "S", 1],
    ["S", "T", 1],
    ["T", "U", 1],
    ["U", "V", 1],
    ["V", "W", 1],
    ["W", "X", 1],
    ["X", "Y", 1],
    ["Y", "Z", 1],
    ["Z", "A", 1],
    ["A", "B", 1],
    ["B", "C", 1],
    ["C", "D", 1],
    ["D", "E", 1],
    ["E", "F", 1],
    ["F", "G", 1],
    ["G", "H", 1],
    ["H", "I", 1],
    ["I", "J", 1],
    ["J", "K", 1],
    ["K", "L", 1],
    ["L", "M", 1],
    ["M", "N", 1],
    ["N", "O", 1],
    ["O", "P", 1],
    ["P", "Q", 1],
    ["Q", "R", 1],
    ["R", "S", 1],
    ["S", "T", 1],
    ["T", "U", 1],
    ["U", "V", 1],
    ["V", "W", 1],
    ["W", "X", 1],
    ["X", "Y", 1],
    ["Y", "Z", 1],
    ["Z", "A", 1],
  ];
  return (
    <Svg
      caption={
        step === 0
          ? "Вершины – объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра – связи между ними"
            : "Вес ребра – цена перехода"
      }
    >

```

```

<Svg
  caption={
    step === 0
      ? "Исходные координаты – огромные и с дырами"
      : step === 1
        ? "Сортируем уникальные значения"
        : "Каждое число заменяем его номером → плотн
  }
>
  {raw.map((v, i) => {
    <g key={i}>
      <rect x={startX + i * (boxW + 4)} y={20} width
      <text x={startX + i * (boxW + 4) + boxW / 2}
        {v}
      </text>
    </g>
  })}

  {step >= 1 &&
    sorted.map((v, i) => {
      <g key={v}>
        <rect x={30 + i * 52} y={62} width={46} hei
        <text x={53 + i * 52} y={77} textAnchor="mi
          {v}
        </text>
      </g>
    })}

  {step >= 2 &&
    raw.map((v, i) => {
      <g key={`c${i}`}>
        <rect x={startX + i * (boxW + 4)} y={98} wi
        <text x={startX + i * (boxW + 4) + boxW / 2
          {sorted.indexOf(v)}
        </text>
      </g>
    })}
</Svg>

```

```

    }}}
  </Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
  <SplayFrames
    captions={["р – корень, х под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p":
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p":
    ]}
  />
};

/** Zig-Zig – х и р с одной стороны, крутим сначала верш
export const ZigZigDemo: React.FC = () => {
  <SplayFrames
    captions={["Линия g → p → x («бамбук»)", "Поворот В
    frames={[
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
      { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
      { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
    ]}
  />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
  <SplayFrames
    captions={["Зигзаг: g влево, х вправо", "Поворот НИ
    frames={[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
    ]}
  />

```

```

const path = ["A", "B", "D", "E", "F", "C"];
const step = useLoop(path.length + 1, 750);
const visited = path.slice(0, step);
const head = visited[visited.length - 1];
return (
  <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
    {GRAPH_EDGES.map(([u, v], i) => {
      const iu = visited.indexOf(u);
      const iv = visited.indexOf(v);
      const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
      return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
    })}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={k === head ? "red" : "gray"} />
    ))}
  </Svg>
);
};

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"],
    ["носки", "боты"],
    ["труссы", "носки"],
    ["штаны", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, [0, 0]]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
      {edges.map(([u, v], i) => (
        <Edge key={i} a={pos[u]} b={pos[v]} color={placed.indexOf(u) < placed.indexOf(v) ? "red" : "gray"} />
      ))}
      {nodes.map((n) => (
        const idx = placed.indexOf(n.id);
        return (
          <g key={n.id}>
            <rect x={pos[n.id][0] - 10} y={pos[n.id][1] - 10} x2={pos[n.id][0] + 10} y2={pos[n.id][1] + 10} fill={idx <= placed.length - 1 ? "red" : "gray"} />
            <text x={pos[n.id][0]} y={pos[n.id][1]} text={n.id} />
          </g>
        );
      ))}
    </Svg>
  );
};

```

```

    });
    const edges: [string, string][] = [
      ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
    ];
    const cut = step === 1;
    return (
      <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
        {edges.map(([u, v], i) => {
          const isBridge = u === "C" && v === "D";
          if (isBridge && cut) return null;
          return <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />
        })}
        {Object.entries(pos).map(([k, [x, y]]) => (
          <Node key={k} x={x} y={y} label={k} fill={cut ? "white" : "black"} />
        ))}
      </Svg>
    );
  });
};

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
      {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />))}
      {Object.entries(pos).map(([k, [x, y]]) => (
        cut && k === "C" ? null : (
          <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "white" : "black"} />
        )
      ))}
    </Svg>
  );
};

```

```

const inScc = (u: string, v: string) => step === 1 &&
return (
  <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
    {edges.map(([u, v], i) => {
      <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
    }}}
    {Object.entries(pos).map([k, [x, y]] => {
      <Node key={k} x={x} y={y} label={k} fill={step
    }}}
  </Svg>
);
};

```

ФАЙЛ 50/92: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
  const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
  const step = useLoop(states.length, 1000);
  const heap = states[step];
  const pos: [number, number][] = [[130, 24], [80, 68],
  const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
  return (
    <Svg caption="Новый элемент всплывает наверх, пока
      <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
      <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
      {heap.map((v, i) => {
        <Node key={i} x={pos[i][0]} y={pos[i][1]} label
      }}}
    </Svg>
  );
};

```

```

        >
      </g>
    )})
    <line x1={24} y1={71} x2={10} y2={71} stroke={C.d
  </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = 0"
      {p.map((par, i) =>
        par !== i ? (
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}
            fill="none" stroke={rootColor[root(i)]} stroke-width={2}
          ) : null
        )}
      {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
    </Svg>
  );
};

```

```

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {

```

```

    const p = TRIE_NODES[n.parent as number];
    return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
  ]]}
  {links.slice(0, step).map(([from, to], i) => {
    const a = TRIE_NODES[from];
    const b = TRIE_NODES[to];
    return (
      <path key={i} d={`M ${a.x} ${a.y} Q ${a.x + 1} ${a.y}, ${b.x} ${b.y}
        fill="none" stroke={C.warn} strokeWidth={1}
      >
    );
  })}
  {TRIE_NODES.map((n) => (
    <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
  ))}
</Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
            fill={n.lvl === step ? C.active : C.idle} stroke={C.warn}
          >
          <text x={n.x} y={n.y} textAnchor="middle" dom
        </g>
      ))}
    >
  );
};

```



```

const len = [1, 2, 4][step];
const count = 8 - len + 1;
return (
  <Svg caption={`Уровень j=${step}: готовые ответы дл
    {Array.from({ length: 8 }).map((_, i) => {
      <g key={i}>
        <rect x={12 + i * 30} y={18} width={26} height={18} />
        <text x={25 + i * 30} y={29} textAnchor="middle" />
      </g>
    })}
    {Array.from({ length: count }).map((_, i) => {
      <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18} width={26} height={18}
        fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
    })}
  </Svg>
);
};

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => {
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}
          fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
      })}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
      <text x={130} y={20} textAnchor="middle" fontSize={14} />
    </Svg>
  );
};

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (

```

```

    });
};

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $`}
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : fill}
                {filled && {
                  <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
                    fontSize={10} fontWeight="700" fill="black"/>
                }}
            </g>
          );
        })
      )
    );
  <text x={130} y={122} textAnchor="middle" fontSize={14}>
    каждая клетка считается один раз и переиспользуется
  </text>
</Svg>
);
};

```

ФАЙЛ 51/92: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
```

```
scc: SccDemo,  
"prefix-function": PrefixFunctionDemo,  
dp: DpDemo,  
floyd: FloydDemo,  
components: ComponentsDemo,  
graph: GraphBasicsDemo,  
"coord-compress": CoordCompressDemo,  
zig: ZigDemo,  
"zig-zig": ZigZigDemo,  
"zig-zag": ZigZagDemo,  
};  
  
export const MiniDemo: React.FC<{ kind: DemoKind }> = (  
  const [paused, setPaused] = useState(false);  
  const Cmp = REGISTRY[kind];  
  if (!Cmp) return null;  
  return (  
    <DemoPauseContext.Provider value={paused}>  
      <div  
        className="bg-slate-950/70 rounded-lg border bo  
        onMouseEnter={() => setPaused(true)}  
        onMouseLeave={() => setPaused(false)}  
      >  
        <Cmp />  
      </div>  
    </DemoPauseContext.Provider>  
  );  
);  
};
```

ФАЙЛ 52/92: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useEffect } from "react";  
import { createPortal } from "react-dom";  
import { X } from "lucide-react";
```

```
        className="p-2 rounded-lg bg-slate-800 hover:
        aria-label="Заккрыть симулятор"
      >
        <X className="w-5 h-5" />
      </button>
    </div>
    <div className="overflow-y-auto overscroll-cont
      <Component />
    </div>
  </div>
</div>,
document.body
);
};
```

ФАЙЛ 53/92: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
```

```

        window.removeEventListener("scroll", onClose, true);
    };
    }, [anchor, onClose]);

```

```

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

```

```

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo-500" >
      <div className="flex items-start gap-2 px-3 pt-3" >
        <h4 className="text-sm font-bold text-indigo-400" >
          {entry.term}
        </h4>
        <button
          onClick={onClose}
          className="text-slate-500 hover:text-white"
          aria-label="Заккрыть подсказку"
        >
          <X className="w-4 h-4" />
        </button>
      </div>
      <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
        <p className="text-[13px] leading-relaxed text-slate-300">
          {entry.description}
        </p>
      </div>
    </div>
  </div>

```

```

-----
import { RotateCcw, SkipForward, Undo } from 'lucide-react'
import { useVizStepSync, vizString } from '../data/vizSteps'

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 4, labelDx: 0, labelDy: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 5, labelDx: 0, labelDy: 0 },
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
  ];

  const isNodeVisible = (id: string) => {
    if (id === 'S' && (step === 0 || step === 7)) return true;
    return false;
  };

  const getPotential = (id: string) => {
    if (step >= 2 && step < 7) {
      if (id === 'S') return 0;
      if (id === 'A') return 0;
      if (id === 'B') return -2;
      if (id === 'C') return -1;
    }
    return 0;
  };
}

```

```

    if (e.id === 'BC' && step === 5) return "url(#a
    if (e.id === 'BC' && step > 5) return "url(#arr
    if (e.id === 'CA' && step === 6) return "url(#a
    if (e.id === 'CA' && step > 6) return "url(#arr

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
    if (e.u === 'S') return e.weight;

    if (step < 4) return e.weight;

    if (e.id === 'AB') {
        if (step === 4) return `${e.weight} + (${0}
        return e.newWeight;
    }
    if (e.id === 'BC') {
        if (step < 5) return e.weight;
        if (step === 5) return `${e.weight} + (${-2}
        return e.newWeight;
    }
    if (e.id === 'CA') {
        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${-1}
        return e.newWeight;
    }
    return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

```

```

        );
        case 4: return (
            <div>
                <b>Шаг 3.1: Пересчет ребра A → B.</b>
                Старый вес: <span className="text-gray-400">1</span>
                Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-emerald-300">0</span>
            </div>
        );
        case 5: return (
            <div>
                <b>Шаг 3.2: Пересчет ребра B → C.</b>
                Старый вес: 1. <br/>
                Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-emerald-300">0</span>
            </div>
        );
        case 6: return (
            <div>
                <b>Шаг 3.3: Пересчет ребра C → A.</b>
                Старый вес: 2. <br/>
                Вычисляем:  $2 + (-1) - 0 =$  <span className="text-emerald-300">1</span>
            </div>
        );
        case 7: return (
            <div className="text-emerald-300">
                <b>Готово!</b> Фиктивную вершину S
                Все новые веса ребер в графе стали 0
                При этом старые кратчайшие пути остались
                теми же!
            </div>
        );
        default: return "";
    }
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">
        <h4 className="text-sm md:text-base font-bold">
            <span className="w-2.5 h-2.5 bg-amber-400">

```



```

        const midX = (u.x + v.x) / 2
        const midY = (u.y + v.y) / 2

        const edgeColorClass = getEdgeColorClass(u, v)
        const textHigh = isEdgeTextHigh(u, v)
        const textEmer = isEdgeTextEmer(u, v)

        return (
          <g key={i}>
            <line
              x1={u.x} y1={u.y}
              className={`stroke-${edgeColorClass}`}
              markerEnd={getMarkerEnd(u, v)}
            />

            <text
              x={midX} y={midY}
              textAnchor="middle"
              dominantBaseline="bottom"
              className={`text-${edgeColorClass}`}
              style={{
                textHeight: textHigh,
                textEmer: textEmer,
                isS ? 'font-size: 1.2em' : ''
              }}
              >{getEdgeContent(u, v)}
            </text>
          </g>
        )
      })
    })
  }
}

```

```

  { /* Nodes */ }
  { nodes.map(n => {
    if (!isNodeVisible(n.id)) return null
    const pot = getPotential(n)

    return (
      <g key={n.id} className={`node-${pot}>
        <circle

```

```

        className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
        <RotateCcw strokeWidth={3}
      </button>
      <button onClick={() => setStep(1)
        className="p-3 bg-slate-800
-slate-400 transition-colors" title="Шаг на
        <Undo strokeWidth={3} size=
      </button>
      <button onClick={() => setStep(
        className={`flex-1 font-bol
          ${step === MAX_STEP
            ? 'bg-emerald-600/50'
            : 'bg-indigo-600 ho
          {step === MAX_STEP ? "Готов
          {step !== MAX_STEP && <Skip
      </button>
    </div>
  </div>

  {/* Progress Dots */}
  <div className="flex justify-center
    {Array.from({length: MAX_STEP +
      <div key={i} className={`w-1
125' : i < step ? 'bg-indigo-900' : 'bg-sla
    }}}
  </div>
</div>
</div>
</div>
</div>
);
}

```

```

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map(({step, index}) => ({ step, index }))
    .filter(({ step }) => (v === null || step.currentNode !== null));
  return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  // Generate step-by-step Kosaraju steps
  const generatedSteps: AlgoStep[] = [];
  const visited = new Set<number>();
  const order: number[] = [];
  const sccMap: Record<number, number> = {};

  const recordStep = {
    phase: AlgoStep["phase"],
    desc: string,
    currentNode: number | null,
    transposed: boolean,
  };

```

```

    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs1(to);
        }
    }

    order.push(u);
    recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Запи-
        у,
        false,
        -1,
    );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        dfs1(i);
    }
}

const topoOrder = [...order].reverse();
recordStep(
    "transpose",
    `Первый проход завершен! Стек выхода в топологиче-
    ебра.`,
    null,
    true,
    -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

```

```

        "dfs2",
        `DFS 2: вершина ${u} из стека уже покрашена в
        u,
        true,
        sccId,
    );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент,
    null,
    true,
    sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    phase: "init",
    desc: "Загрузка...",
    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,
};

useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
        timer = setInterval(() => {
            setCurrentStepIdx((prev) => prev + 1);
        }, 1000);
    return () => {
        if (timer) clearInterval(timer);
    };
}, [isPlaying, currentStepIdx]);

```

```

        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"
            {/* Markers for directed arrows */}
            <defs>
                <marker
                    id="arrow"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
                </marker>
                <marker
                    id="arrow-active"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"

```

```

        if {
            step.currentNode === edge.u ||
            step.currentNode === edge.v
        } {
            stroke = "#10b981";
            markerId = "arrow-active";
        }
    }

    // Adjust slightly for bidirectional link
    const isBidi =
        (edge.u === 3 && edge.v === 4) ||
        (edge.u === 4 && edge.v === 3);
    const dy = isBidi ? (edge.u === 3 ? -10 :

    return (
        <line
            key={`edge-${idx}`}
            x1={x1}
            y1={y1 + dy}
            x2={x2}
            y2={y2 + dy}
            stroke={stroke}
            strokeWidth="2.5"
            markerEnd={`url(#${markerId})`}
            className="transition-all duration-300"
        />
    );
}}}

{/* Nodes */}
{initialNodes.map((node) => {
    const isCurrent = step.currentNode === node.id;
    const isVisited =
        step.visited.has(node.id) ||
        (step.phase === "dfs1" && step.currentNode === node.id);
    const sccId = step.sccMap[node.id];

```

```

        fontWeight="bold"
      >
        SCC #{sccId}
      </text>
    })
  </g>
);
}}
</svg>

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
  1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
)}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-4
  Порядок обхода и стек
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-xs text-white leading-re
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-a
    {/* Top-Sort output / Order stack represent
    <div>
      <span className="text-[10px] text-slate-5
        СТЕК ВЫХОДА (DFS 1 ORDER):
      </span>
      <div className="flex flex-wrap gap-1 bg-s

```



```

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        Шаг {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1.5">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-200 text-white px-2 py-1 rounded">
          Назад
        </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-200 text-white px-2 py-1 rounded">
          Вперед
        </button>
      </div>
    </div>
  </div>
</div>
</div>
);
};

```

ФАЙЛ 56/92: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';
import { useVizRuntime, vizNumber, vizString } from 'viz-runtime';

```

```
-----
{ id: '0-1', u: 0, v: 1, weight: 2, status: 'pending'
{ id: '3-4', u: 3, v: 4, weight: 3, status: 'pending'
{ id: '1-2', u: 1, v: 2, weight: 4, status: 'pending'
{ id: '0-3', u: 0, v: 3, weight: 5, status: 'pending'
{ id: '1-4', u: 1, v: 4, weight: 6, status: 'pending'
{ id: '4-5', u: 4, v: 5, weight: 7, status: 'pending'
{ id: '3-6', u: 3, v: 6, weight: 8, status: 'pending'
{ id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'
{ id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'
{ id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'
{ id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'
{ id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'
];
```

```
export const KruskalSimulator: React.FC<{ defaultAlgo?:
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdge
  const [stepIndex, setStepIndex] = useState<number>(0)
  const [heapQueue, setHeapQueue] = useState<string[]>(
  const [activeCheatTab, setActiveCheatTab] = useState<
```

```
// — Синхронизация с Python-компилятором: i — индекс
// списке, u/v — метки вершин, w — вес. Подсвечиваем
```

```
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveW = vizNumber(vars?.w);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveEdges: Edge[] = (() => {
  if (liveI === null && liveW === null && liveU === null)
  const target =
    (liveI !== null && initialEdges[liveI]) ||
    (liveU && liveV
      ? initialEdges.find(
        (e) =>
          (initialVertices[e.u].label === liveU &&
```

```

    },
    // Step 3: Inspect D-E (3)
    () => {
        setEdges(prev => prev.map(e => e.id === '3-4' ? {
            setLogs(prev => [{
                title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
                description: 'Оно соединяет две другие бесцветные вершины D и E',
                type: 'info'
            }, ...prev]));
    },
    // Step 4: Include D-E
    () => {
        setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {
            setEdges(prev => prev.map(e => e.id === '3-4' ? {
            setLogs(prev => [{
                title: 'Успех: Ребро D-E в остова',
                description: 'Вершины D и E окрашены в синий цвет',
                type: 'success'
            }, ...prev]));
    },
    // Step 5: Inspect B-C (4)
    () => {
        setEdges(prev => prev.map(e => e.id === '1-2' ? {
            setLogs(prev => [{
                title: 'Шаг 3: Берем ребро B-C (вес 4)',
                description: 'Оно соединяет красную вершину B и бесцветную вершину C',
                type: 'info'
            }, ...prev]));
    },
    // Step 6: Include B-C
    () => {
        setVertices(prev => prev.map(v => v.id === 2 ? {
            setEdges(prev => prev.map(e => e.id === '1-2' ? {
            setLogs(prev => [{
                title: 'Успех: Ребро B-C в остова',
                description: 'Вершина C перекрашена в красный цвет',
                type: 'success'
            }, ...prev]));
    },

```



```
// Step 19: Inspect E-H (11) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
  setLogs(prev => [{
    title: 'Шаг 10: Берем ребро E-H (вес 11)',
    description: 'Оба конца E и H уже фиолетовые. С
    type: 'warning'
  }, ...prev]));
},
// Step 20: Reject E-H
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Выбрасываем ребро E-H.',
    type: 'error'
  }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
  setEdges(prev => prev.map(e => e.id === '7-8' ? {
  setLogs(prev => [{
    title: 'Шаг 11: Берем ребро H-I (вес 12)',
    description: 'Соединяет фиолетовую H и последнюю
    type: 'info'
  }, ...prev]));
},
// Step 22: Include H-I
() => {
  setVertices(prev => prev.map(v => v.id === 8 ? {
  setEdges(prev => prev.map(e => e.id === '7-8' ? {
  setLogs(prev => [{
    title: ' Успех: MST построено Краскалом!',
    description: 'Все 9 вершин окрашены в единый фи
      + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
    type: 'success'
  }, ...prev]));
},
},
```

```

() => {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
    description: 'Плесень расширяется в сторону верш
    type: 'info'
  }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
  setVertices(prev => prev.map(v => v.id === 0 ? {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    eap' } : e));
  setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
  setLogs(prev => [{
    title: 'Успех: Плесень поглотила вершину A',
    description: 'Новое ребро A-B(2) добавлено в куч
    type: 'success'
  }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
  setEdges(prev => prev.map(e => e.id === '0-1' ? {
  setLogs(prev => [{
    title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
    description: 'Плесень стремительно бежит к верши
    type: 'info'
  }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
  setVertices(prev => prev.map(v => v.id === 1 ? {
  setEdges(prev => prev.map(e => e.id === '0-1' ? {
    eap' } : e));
  setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
  setLogs(prev => [{
    title: 'Успех: Вершина B захвачена плесенью',
    description: 'В кучу добавлено B-C(4). Ребро B-

```

```

        title: 'Отказ: Игнорируем внутреннее ребро',
        description: 'Ребро В-Е отброшено. Плесень не р
        type: 'error'
    }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
        description: 'Плесень из центра Токио (Е) тянет
        type: 'info'
    }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
        eap' } : e));
    setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
    setLogs(prev => [{
        title: 'Успех: Вершина F захвачена',
        description: 'Ребро F-I(13) добавлено в кучу. Р
        type: 'success'
    }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
        description: 'Плесень расширяется вниз к G.',
        type: 'info'
    }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {

```

```

() => {
  setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
      eap' } : e));
  setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)']);
  setLogs(prev => [{
    title: 'Успех: Вершина H захвачена',
    description: 'В кучу добавлено H-I(12).',
    type: 'success'
  }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
      title: 'Шаг 11: Куча выдает E-H (вес 11)',
      description: 'Обе вершины E и H уже в плесени.',
      type: 'warning'
    }, ...prev]));
},
// Step 21: Reject E-H
() => {
  setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setHeapQueue(['H-I (12)', 'F-I (13)']));
  setLogs(prev => [{
    title: 'Отказ: Игнорируем E-H',
    description: 'Выбрасываем из кучи.',
    type: 'error'
  }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
  setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
      title: 'Шаг 12: Куча выдает H-I (вес 12)',
      description: 'Плесень тянется к последней чистой вершине.',
      type: 'info'
    }, ...prev]));
},

```



```

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
  setActiveAlgo(algo);
  setVertices(initialVertices);
  setEdges(initialEdges);
  setStepIndex(0);
  setHeapQueue([]);
  if (algo === 'kruskal') {
    setLogs([
      {
        title: 'Введение в раскраску (Краскал)',
        description: 'Представь, что все 9 вершин гра
        type: 'info',
      },
    ]);
  } else if (algo === 'prim') {
    setLogs([
      {
        title: 'Введение в Плесень (Прима)',
        description: 'Логика «Плесени»: выбираем стар
        едь (Heap) и захватываем соседей!',
        type: 'info',
      },
    ]);
  } else {
    setLogs([
      {
        title: 'Введение в Коллективизацию (Борувка)',
        description: 'Логика «Борувки»: Каждая из 9 д
        чтобы объединиться в колхоз.',
        type: 'info',
      },
    ]);
  }
};

```

```

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex

```

```

    <p className="text-slate-400 text-sm">
      Наблюдай за работой Краскала (Раскраска),
    </p>
  </div>

```

```

  { /* Algorithm Tabs */ }
  <div className="flex flex-wrap items-center gap-2">
    <div className="flex items-center bg-slate-500 p-2">
      <button
        onClick={() => handleReset('kruskal')}
        className={
          "flex items-center gap-1.5 px-3 py-2"
        +
          (activeAlgo === 'kruskal'
            ? 'bg-indigo-600 text-white shadow-md'
            : 'text-slate-400 hover:text-slate-300')
        >
        <GitGraph className="w-3.5 h-3.5" />
        <span>Краскал (Билет 18)</span>
      </button>
      <button
        onClick={() => handleReset('prim')}
        className={
          "flex items-center gap-1.5 px-3 py-2"
        +
          (activeAlgo === 'prim'
            ? 'bg-emerald-600 text-white shadow-md'
            : 'text-slate-400 hover:text-slate-300')
        >
        <Layers className="w-3.5 h-3.5" />
        <span>Прима (Билет 19)</span>
      </button>
      <button
        onClick={() => handleReset('boruvka')}
        className={
          "flex items-center gap-1.5 px-3 py-2"

```

```

    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-3"
    {/* Graph Canvas */}
    <div className="lg:col-span-2 bg-slate-950 rounded-2xl p-4 active min-h-[480px] overflow-hidden">

      {/* Legend */}
      <div className="absolute top-4 left-4 bg-slate-900 p-4 -center gap-3 text-xs">
        <div className="flex items-center gap-1.5">
          <div className="w-3 h-3 rounded-full bg-white">
        </div>
        {activeAlgo === 'kruskal' ? (
          <>
            <div className="flex items-center gap-1.5">
              <div className="w-3 h-3 rounded-full bg-white">
            </div>
            <div className="flex items-center gap-1.5">
              <div className="w-3 h-3 rounded-full bg-white">
            </div>
            <div className="flex items-center gap-1.5">
              <div className="w-3 h-3 rounded-full bg-white">
            </div>
          </>
        ) : activeAlgo === 'prim' ? (
          <>
            <div className="flex items-center gap-1.5">
              <div className="w-3 h-1 bg-sky-600">
            </div>
            <div className="flex items-center gap-1.5">
              <div className="w-3 h-3 rounded-full bg-white">
            </div>
          </>
        ) : (
          <>
            <div className="flex items-center gap-1.5">

```

```

        <line
          x1={u.x + "%"}
          y1={u.y + "%"}
          x2={v.x + "%"}
          y2={v.y + "%"}
          stroke={strokeColor}
          strokeWidth={isInspecting ? 4.5 :
            strokeDasharray={isRejected ? '4,4' : ''}
          className={"transition-all duration-300"}
        />
        <text
          x={({u.x + v.x} / 2 + "%")
          y={({u.y + v.y} / 2 + "%")
          fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
          fontSize="4.5"
          fontFamily="monospace"
          fontWeight="bold"
          textAnchor="middle"
          dominantBaseline="middle"
          className="select-none filter drop-shadow"
          dy="-1.5"
        >
          {edge.weight}
        </text>
      </g>
    );
  }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: vertex.x + "%",
        className={"absolute -translate-x-1/2 -translate-y-1/2
font-bold text-base border-2 transition-all"}

```

```

        {heapQueue.length === 0 ? (
          <span className="text-xs text-slate-300">
            0 shadow-md animate-pulse" : "bg-slate-800 text-white">
              {item}
            </span>
          ) : (
            heapQueue.map(({item, idx}) => (
              <span key={idx} className={"px-2 py-1 rounded-sm"}>
                {item}
              </span>
            ))
          )
        </div>
      </div>
    )}

    <div className="flex-1 p-4 overflow-y-auto">
      {logs.map((log, idx) => (
        <div
          key={idx}
          className={
            "p-4 rounded-xl border transition-all" +
            (log.type === 'success'
              ? 'bg-emerald-950/40 border-emerald-500'
              : log.type === 'warning'
              ? 'bg-amber-950/40 border-amber-500'
              : log.type === 'error'
              ? 'bg-rose-950/40 border-rose-500'
              : 'bg-slate-900/60 border-slate-700')
          >
          <div className="flex items-center gap-2">
            {log.type === 'success' && <CheckCircle className="text-emerald-500"/>
            {log.type === 'warning' && <HelpCircle className="text-amber-500"/>
            {log.type === 'error' && <XCircle className="text-rose-500"/>
            {log.type === 'info' && <Play className="text-slate-500"/>
          </div>
          <h5 className="font-bold text-sm text-white">{log.message}</h5>
          <p className="text-xs text-slate-300">{log.timestamp}</p>
        </div>
      )}
    </div>
  </div>

```

```

{activeCheatTab === 'kruskal' && {
  <div className="grid grid-cols-1 lg:grid-cols-2" style={{gridTemplateColumns: '1fr 1fr'}}>
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <p className="text-2xl font-black text-white">Крускал</p>
        <p className="text-slate-400 text-xs mt-2">Алгоритм нахождения минимального остовного дерева</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Award className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <p className="text-2xl font-black text-white">Авард</p>
        <p className="text-slate-400 text-xs mt-2">Алгоритм нахождения минимального остовного дерева</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Code className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <p className="text-2xl font-black text-white">Код</p>
        <p className="text-slate-400 text-xs mt-2">Алгоритм нахождения минимального остовного дерева</p>
      </div>
    </div>
    <div className="lg:col-span-2">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Code className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

```

ложении:</p>

<p className="text-xs text-slate-300 le
ие чистые вершины через самое дешевое досту

</div>

</div>

<div className="lg:col-span-2">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Code className="w-3.5 h-3.5 text-eme

</p>

<pre className="text-xs text-emerald-40

80 flex-1">

```
{`import heapq
```

```
def prim(graph, start):
```

```
    visited = [False] * len(graph)
```

```
    heap = [(0, start, -1)] # (weight, node, parent)
```

```
    mst = []
```

```
    while heap:
```

```
        w, u, prev = heapq.heappop(heap)
```

```
        if visited[u]: continue
```

```
        visited[u] = True
```

```
        if prev != -1: mst.append((prev, u, w))
```

```
        for next_w, v in graph[u]:
```

```
            if not visited[v]:
```

```
                heapq.heappush(heap, (next_w, v, u))
```

```
    return mst`}
```

</pre>

</div>

</div>

</div>

}}

```
{activeCheatTab === 'boruvka' && {
```

```
    <div className="grid grid-cols-1 lg:grid-cols
```

```
        <div className="lg:col-span-1 space-y-4">
```



```
        if cheapest[ru] == -1 or cheapest[ru][2]
        if cheapest[rv] == -1 or cheapest[rv][2]
# Объединяем все колхозы по выбранным мостам
for bridge in cheapest:
    if bridge != -1 and union(bridge[0], bridge
        mst.append(bridge)`}
    </pre>
    </div>
    </div>
    </div>
    })
</div>
);
};
```

ФАЙЛ 57/92: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jp
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60'
```

```

        className="w-full h-full object-cover group-hover:scale-105"
      </div>
      <div className="p-4">
        <h3 className={`text-lg font-bold mb-1 ${c.className}`}>{c.title}</h3>
        <p className="text-slate-400 text-sm">
          {c.desc} <span className={`font-bold ${c.className}`}>{c.status}</span>
        </p>
        <div className={`mt-3 text-xs font-mono px-2`}>
          {c.details}
        </div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 58/92: `src/components/MobileToc.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы – показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
```

```

<div className="absolute inset-0 bg-slate-950" />
<div className="absolute inset-y-0 left-0 w-[100px] h-[100px]
  te-[tocIn_.18s_ease-out]">
  <div className="flex items-center justify-between" />
    <span className="font-bold text-white text-sm">
      <button
        onClick={() => setOpen(false)}
        className="p-2 -mr-2 text-slate-400 hover:text-white"
        aria-label="Заккрыть содержание"
      />
      <X className="w-5 h-5" />
    </button>
  </div>
  <div className="flex-1 min-h-0">
    <Sidebar
      selectedChapterId={selectedChapterId}
      setSelectedChapterId={setSelectedChapterId}
      onNavigate={() => setOpen(false)}
    />
  </div>
</div>
</div>
</div>
  )}
</>
);
};

```

ФАЙЛ 59/92: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { BookOpen, Sparkles, FileDown, FileText, Code2,
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

```

```

>
<button
  id="tab-guide-btn"
  onClick={() => window.scrollTo({ top: 0, be
  className="hidden sm:flex flex-1 lg:flex-no
  ite bg-indigo-600/80 shadow-lg shadow-indig
  title="Содержание пособия"
>
  <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
  id="tab-compiler-btn"
  onClick={onToggleCompiler}
  aria-pressed={compilerOpen}
  title="Боковая панель Python-компилятора (н
  страницей"
  className={`flex-1 lg:flex-none flex items-
  uration-200 whitespace-nowrap ${
    compilerOpen
      ? 'bg-emerald-600 text-white shadow-lg
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Terminal className="w-4 h-4" /> Python
</button>
<a
  id="download-guide-pdf-btn"
  href={` ${import.meta.env.BASE_URL}export/gu
  download="guide_all_topics.pdf"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
  ver:bg-indigo-600/20 border border-indigo-5
  title="Скачать PDF-конспект: текст всех тем
>
  <GraduationCap className="w-4 h-4" /> PDF (
</a>
<a

```

ФАЙЛ 60/92: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотрим
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
          <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0">
            {(() => {
              const pts = [
                {id:0,x:160,y:40},
                {id:1,x:270,y:100},
                {id:2,x:230,y:230},
                {id:3,x:90,y:230},
                {id:4,x:50,y:100}
              ];
              const edges = [
                [0,1],[0,2],[0,3],[0,4],
                [1,2],[1,3],[1,4],
                [2,3],[2,4],
                [3,4]
              ];
              function findPt(id:number) { return pts.find(p => p.id === id); }
              function intersect(a:number,b:number,c:number,d:number) {
                if (a===c||a===d||b===c||b===d) return true;
                const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
                function ccw(ax:number,ay:number,bx:number,by:number) {
                  return (by-ay)*(ax-bx)>0;
                }
                const o1=ccw(p1.x,p1.y,p2.x,p2.y),o2=ccw(p1.x,p1.y,p3.x,p3.y),o3=ccw(p1.x,p1.y,p4.x,p4.y),o4=ccw(p2.x,p2.y,p3.x,p3.y),o5=ccw(p2.x,p2.y,p4.x,p4.y),o6=ccw(p3.x,p3.y,p4.x,p4.y);
                return (o1>0&&o2>0&&o3>0&&o4>0&&o5>0&&o6>0);
              }
            })()}
          </svg>
        </div>
      </div>
    </div>
  );
}
```

```

<div className="bg-slate-950 rounded-xl p-4 border"
  <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
  <svg className="w-full h-full absolute inset-0"
    {(() => {
      const pts = [
        {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:160,y:220},
        {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:60,y:60}
      ];
      const edges = [
        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
      ];
      function findPt(id:number) { return pts.find(p => p.id === id); }
      function intersect(a:number,b:number,c:number,d:number) {
        if (a===c||a===d||b===c||b===d) return true;
        const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
        if (!p1||!p2||!p3||!p4) return false;
        function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
          return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
          && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
      }
      const crosses:number[]=[];
      for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
          if (!crosses.includes(i)) crosses.push(i);
          if (!crosses.includes(j)) crosses.push(j);
        }
      }
      const lines = []; const circles = [];
      for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
          stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
      }
    })}
  </svg>
</div>

```

```

    ChevronRight,
    Copy,
    Eye,
    HelpCircle,
    Link2,
    Loader2,
    Pause,
    Play,
    Plus,
    RotateCcw,
    SkipBack,
    SkipForward,
    Square,
    Terminal,
    Unlink,
    Variable,
    X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */

```

```
-----  
}  
  
/* -----  
/*  Шаблон синхронизации: короткая шапка + минимальный  
/*  скелет, шаг-в-шаг с подсветкой визуализации (см. da  
/*  -----  
  
/* -----  
/*  Шпаргалка Python: команды сортировки, циклы, структ  
/*  «Вставить» кладёт снippet в редактор под курсор.  
/*  -----  
  
interface Snippet {  
    label: string;  
    tip: string;  
    code: string;  
}  
  
const SNIPPETS: { group: string; items: Snippet[] }[] =  
    {  
        group: "Сортировка",  
        items: [  
            {  
                label: "arr.sort() / sorted()",  
                tip: "In-place сортировка по возрастанию и нова  
                code: 'arr = [5, 2, 8, 1]\narr.sort()  
                ) # копия по убыванию\n',  
            },  
            {  
                label: "sorted(key=...)",  
                tip: "Сортировка по ключу: рёбра по весу, пары  
                code: 'edges = [(3, "A", "B"), (1, "B", "C"), (1  
                es)\n',  
            },  
        ],  
    },  
    {  
        group: "Циклы",
```



```

        {
            label: "Матрица n × m",
            tip: "Таблица расстояний: dist[i][j], бесконечны",
            code: 'n, m = 3, 4\nndist = [[float("inf")] * m
        },
        {
            label: "Множество visited",
            tip: "Посещённые вершины: in/add за O(1).",
            code: 'visited = set()\nvisited.add("A")\nprint
        },
    ],
},
{
    group: "Вывод",
    items: [
        {
            label: "f-строка",
            tip: "Подставить значения переменных шага в строку",
            code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}]
        },
    ],
},
];

```

```

/** Переживает сккрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string; } = {

```

```

export function PythonCompiler({ chapterId, chapterTitle, chapterTitleShort, compilerProps }) {
    /** Активная вкладка демонстрации страницы (у sparse-матрицы)
    const [demoId, setDemoId] = useState<string | null>(null)
    const sync = useMemo(() => getPageSync(chapterId, demoId))
    const syncVarBases = useMemo(
        () =>
        [...new Set(
            sync.variables.flatMap((v) =>
                v.name
                    .split(/\\s*(?:,|\\/)\s*/)

```

```

});
const prevTplRef = useRef({ base: initTemplate, full:

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пока
const [editHold, setEditHold] = useState<string | null>
/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
    if (typeof window === "undefined") return 1;
    const saved = Number(window.localStorage.getItem(PL
    return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
    window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
    savedEditor = { code, template: initTemplate, chapt
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId
useEffect(() => () => clearVizState(chapterId), [chap

// Смена страницы (или вкладки демо): нетронутый реда
// инициализацию; свой код не затираем – предлагаем с
useEffect(() => {
    const prev = prevTplRef.current;
    if (initTemplate === prev.base && template === prev
    prevTplRef.current = { base: initTemplate, full: te
    setPlaying(false);
    setEditHold(null);
    setDebug(null); // трасса ссылается на строки старо

```

```

* события: после st[i][j]= Python уже мог увеличить
*/
const snapshotAt = useCallback((result: DebugResult,
  const step = result.steps[idx];
  const next = result.steps[idx + 1];
  const afterLocals = next?.locals ?? result.finalLocals;
  const afterValues = next?.values ?? result.finalValues;
  const beforeValues = step?.values ?? {};
  const line = step?.line ? source.split("\n")[step.line - 1] : "";
  // Нас интересуют только простые скалярные цели (`i` или `i +=`). Контейнеры и так всегда берутся из afterValues.
  // внутри print(f"i={i}") не будет ошибочно принят `i` как переменная.
  const assignment = line.match(
    /^\\s*([A-Za-z_][A-Za-z0-9_]*)\\s*(?:\\s*=|\\s*[A-Za-z_][A-Za-z0-9_]*\\s*=)\\s*([A-Za-z_][A-Za-z0-9_]*)$/;
  );
  const assigned = new Set(assignment?.[1].split(",") || []);

  const variables = { ...afterValues };
  const locals = { ...afterLocals };
  for (const [name, value] of Object.entries(beforeValues)) {
    const scalar = value === null || typeof value !== "object";
    if (scalar && !assigned.has(name) && name in afterValues) {
      variables[name] = value;
      if (step?.locals[name] !== undefined) locals[name] = value;
    }
  }
  const changed = changedVars(step?.locals, locals);
  return {
    line: step?.line,
    func: step?.func,
    locals,
    variables,
    changed: Object.keys(changed).filter((name) => changed[name]),
  };
}, []);

const publishDebug = useCallback(
  (result: DebugResult, idx: number, source: string) => {

```

```

    });

    const stepTo = useCallback(({idx: number}) => goDebug(idx), [chapterId, code]);

    const stopDebug = useCallback(() => {
        setPlaying(false);
        setEditHold(code);
        setDebug(null);
        clearVizState(chapterId);
    }, [chapterId, code]);

    // Базовый интервал 650 мс; множитель скорости хранится в state
    useEffect(() => {
        if (!playing || !debug) return;
        if (debug.idx >= debug.result.steps.length - 1) {
            setPlaying(false);
            return;
        }
        const t = window.setTimeout(() => stepBy(1), 650 / playbackSpeed);
        return () => window.clearTimeout(t);
    }, [playing, debug, stepBy, playbackSpeed]);

    const debugRun = useCallback(async (autoPlay = false) => {
        if (running) return;

        const source = code;
        const input = stdin;
        const request = traceRequest;
        setRunning(true);
        setPlaying(false);
        setEditHold(null);
        setDebug(null);
        clearVizState(chapterId);
        setOutputOpen(false);
        setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "");

        const stdout: string[] = [];
        const stderr: string[] = [];
    }, [chapterId, code, stdin, traceRequest, runtimeReady]);

```

```

    }, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки)
useEffect(() => {
  if (!debug) return;
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLInputElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(1);
    }
    if (e.key === "ArrowLeft") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(-1);
    }
    if (e.key === " ") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying((v) => !v);
    }
    if (e.key === "Escape") stopDebug();
  };
  // capture-фаза: чтобы глобальный обработчик навигации работал
  window.addEventListener("keydown", onKey, true);
  return () => window.removeEventListener("keydown", onKey);
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[debug.result.steps.length - 1] } : undefined;
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
  if (curDebugLine === undefined || !listRef.current) return;
  const line = curDebugLine;
  const list = listRef.current;
  const item = list?.[line - 1];
  if (item) {
    item.scrollIntoView({
      behavior: "smooth",
      block: "center",
    });
  }
});

```

```

        const end = ta.selectionEnd ?? start;
        const padded = (start > 0 && !current.slice(0, start).includes('\n')) ? ' ' : '';
        const next = current.slice(0, start) + padded + current.slice(end);
        requestAnimationFrame(() => {
            ta.focus();
            ta.selectionStart = ta.selectionEnd = start + padded.length;
        });
        return next;
    });
}, []));

return (
    <aside
        aria-label="Python-компилятор"
        className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
            -[620px] flex flex-col bg-slate-900 border-t lg:h-[600px]
            style={typeof window !== "undefined" ? {
                ...(width && window.innerWidth >= 1024 ? { width: '100%' },
                ...(height ? { height: Math.min(height, window.innerHeight) },
                } : undefined)
            }
        >
        {/* Верхний край меняет высоту на телефоне и десктопе */}
        {onHeightChange && (
            <div
                role="separator"
                aria-orientation="horizontal"
                aria-label="Потяните, чтобы изменить высоту терминала"
                title="Потяните, чтобы изменить высоту терминала"
                className="absolute -top-1.5 left-0 right-0 h-10
                    bg-slate-800 border-top border-bottom border-x"
                onPointerDown={(event) => {
                    const el = event.currentTarget;
                    el.setPointerCapture(event.pointerId);
                    const startY = event.clientY;
                    const starth = el.parentElement?.getBoundingClientRect().height;
                    const onMove = (moveEvent: PointerEvent) => {
                        const maxHeight = window.innerHeight - (window.top ? 40 : 0);
                        const minHeight = Math.min(260, maxHeight);
                        const next = Math.round(starth + startY - moveEvent.clientY);
                    
```

```

        el.addEventListener("pointermove", onMove as EventListener)
        el.addEventListener("pointerup", onUp as EventListener)
    })
>
    <div className="absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
        width-500px height-500px transition-colors" />
</div>
})
{/* — Заголовок панели
<div className="shrink-0 flex items-center gap-2" />
    <span className={`w-2 h-2 rounded-full shrink-0`} />
    <Terminal className="w-4 h-4 text-emerald-400" />
    <span className="text-[13px] font-bold text-white" />
    <span className="hidden sm:inline text-[10px] text-white" />

<div className="ml-auto flex items-center gap-1" />
    <Tooltip
        side="bottom"
        content={running ? "Python выполняется..." : "Запустить автопроход"}
    >
        <button
            type="button"
            onClick={runOrTogglePlayback}
            disabled={running}
            className={`inline-flex items-center gap-2`}
        >
            <span>
                {playing
                    ? "bg-amber-500/15 text-amber-300 hover:bg-amber-500/30"
                    : "bg-emerald-500/15 text-emerald-300 hover:bg-emerald-500/30"}
            </span>
            <span>
                {aria-label={running ? "Python выполняется" : "Запустить автопроход"}}
            </span>
            <span>
                {running ? <Loader2 className="h-4 w-4 animate-spin" /> : ""}
            </span>
            <span>{playbackSpeed}x</span>
        </button>
    </Tooltip>

```

```

        setDesyncedFrom(null);
        setPlaying(false);
        setEditHold(null);
        setDebug(null);
        setTraceRequest((n) => n + 1);
    }}
    className="p-1.5 rounded-lg text-indigo-300"
    aria-label="Вставить эталонный код"
  >
    <Eye className="w-4 h-4" />
  </button>
</Tooltip>
<label
  className="ml-1 inline-flex items-center gap-1
  border border-slate-700 hover:border-emerald-500
  title="Скорость автоматического проигрывания"
  >
  {running ? <Loader2 className="w-3.5 h-3.5 ml-1" /> : ""}
  <span>{running ? "авто..." : "авто"}</span>
  <select
    aria-label="Скорость автопрохода"
    value={playbackSpeed}
    onChange={(event) => setPlaybackSpeed(Number(event.target.value))}
    className="cursor-pointer rounded bg-slate-800"
  >
    {PLAYBACK_SPEEDS.map((speed) => (
      <option key={speed} value={speed}>{speed}
    ))}
  </select>
</label>
</div>
</div>

/* — Мини-вкладки: переменные страницы / шпаргалка
<div className="shrink-0 border-b border-slate-800"
  <div className="flex items-center gap-1 px-2 pt-2"
    <button

```



```
</div>
```

```
{stripTab === "vars" ? {
  <div className="px-3 pb-2.5 space-y-2 max-h-300">
    <div className="flex items-center gap-2 min-w-100">
      <Tooltip side="bottom" content="Открыть э
        <button
          type="button"
          onClick={onOpenGuide}
          className="inline-flex items-center gap-2 p-2 text-slate-300 hover:text-white hover:bg-slate-700">
            <span className="truncate">{chapterTitle}</span>
          </button>
        </Tooltip>
      </div>
      {desyncedFrom !== null && {
        <Tooltip
          side="bottom"
          content="В редакторе ваш код, а страница была заменёна."
        >
          <button
            type="button"
            onClick={resync}
            className="flex items-center gap-1.5 p-2 text-amber-400 hover:bg-amber-500/20 hover:text-white">
            <Unlink className="w-3.5 h-3.5 shrink-0">
              Страница сменилась – подставить её перенумерацию
            </Unlink>
          </button>
        </Tooltip>
      }
    }
  }
  {hasViz && {
    <div className="flex flex-wrap items-center gap-2">
      {sync.variables.map((v) => {
        <Tooltip
          key={v.name}
          content={v.description}
        >
          <div className="flex items-center gap-2">
            <div className="flex flex-col items-center gap-2">
              <div className="w-10 h-10 rounded bg-slate-700/50">
                <img alt={v.icon} />
              </div>
              <div>{v.name}</div>
            </div>
            <div>{v.description}</div>
          </div>
        </Tooltip>
      })}
    </div>
  }
}
```

```

    <div key={gItem.group}>
      <div className="text-[9px] font-bold up">
        <div className="flex flex-wrap gap-1.5">
          {gItem.items.map((s) => (
            <Tooltip key={s.label} side="bottom">
              <button
                type="button"
                onClick={() => insertSnippet(s)}
                className="inline-flex items-center
- [10.5px] text-indigo-300 hover:text-white
                title=""
              >
                <Plus className="w-3 h-3 text-slate-600" />
                {s.label}
              </button>
            </Tooltip>
          ))}
        </div>
      </div>
    </div>
  </div>
  <p className="text-[10px] text-slate-600">H
</div>
))
</div>

```

```

{ /* — Редактор + ввод —————
<div className="flex-1 min-h-0 flex flex-col">
  {debug ? (
    <div className="flex-1 min-h-[80px] flex flex-col">
      { /* панель управления шагами */ }
      <div className="shrink-0 flex items-center">
        <Tooltip content={playing ? `Пауза автопр
роиграть трассу заново со скоростью ${playba
л)`}>
          <button
            type="button"
            onClick={runOrTogglePlayback}
            className={`mx-0.5 inline-flex items-

```

```

        </>
      ) : (
        "шагов нет — исправьте ошибку и начни
      )}
    </span>
    {debug.result.truncated && (
      <Tooltip content={`Выполнение остановлено`}
        <span tabIndex={0} className="cursor-help rounded-full px-1.5 py-0.5">
          лимит
        </span>
      </Tooltip>
    )}
    <Tooltip content={hasViz ? "Визуализация" : "Алгоритмический текст не требуется."} : "У страницы нет визуализации"
      <span
        tabIndex={0}
        className={`cursor-help inline-flex items-center justify-center gap-1 border-emerald-500/40 bg-emerald-500/10 text-emerald-500`}
      >
        {hasViz ? <Link2 className="w-3 h-3" href="#">Визуализация</Link2> : "свободный текст"}
      </span>
    </Tooltip>
    <Tooltip content="Вернуться в редактор (Еще раз)"
      <button type="button" aria-label="Редактор" className="p-1 rounded-md text-rose-400 hover:text-white"
        <Square className="w-3.5 h-3.5" />
        <span className="text-[9px] font-bold">Вернуться</span>
      </button>
    </Tooltip>
  </div>

```

```

  {/* листинг с подсветкой текущей строки */}
  <div ref={listRef} className="flex-1 min-h-[200px] p-1"
    {debug.src.split("\n").map((line, i) => {
      const ln = i + 1;
      const active = ln === curDebug?.step?.line
    })}
  >

```

```

        const synced = syncVarSet.has(name);
        return (
          <span
            key={name}
            className={`inline-flex items-bas
              changed
                ? "border-amber-500/50 bg-amb
                : synced
                ? "border-emerald-500/40 bg
                : "border-slate-700 bg-slat
            `}
          >
            <span className={changed ? "text-
              {name}
            </span>
            <span className="text-slate-500">
            <span className={changed ? "text-
          </span>
        );
      }}}
    </div>
  </div>
</div>
) : (
  <textarea
    ref={editorRef}
    value={code}
    onChange={(event) => {
      setEditHold(null);
      setCode(event.target.value);
    }}
    onKeyDown={(event) => {
      if ((event.ctrlKey || event.metaKey) && e
        event.preventDefault();
        void debugRun();
      }
      event.stopPropagation();
    }}
  >

```

```
        <span className="text-slate-600">{outputOpen}</span>
      </button>
      {outputOpen && (
        <pre className="flex-1 min-h-[70px] overflow-
          leading-5 text-slate-300">
          {output}
        </pre>
      )}
      <div className="px-3 py-1.5 border-t border-sla
    </div>
  </aside>
);
}
```

ФАЙЛ 62/92: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../v'
```

```
interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
  { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-violet-500', bubbleText: 'Матана!' },
  { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet-500 to-blue-500', bubbleText: 'Матана!' },
];
```

```

const [isServing, setIsServing] = useState(false)
const [shakeEmpty, setShake] = useState(false)
const [log, setLog] = useState('')

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяже');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PRESETS[counter % PRESETS.length];
  counter++;

  const newGuest: Customer = {
    id: Date.now().toString(),
    ...preset,
    animState: 'in',
  };

  setQueue(prev => [...prev, newGuest]);
  addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

  setTimeout(() => {
    setQueue(prev => prev.map(c => c.id !== newGuest.id));
  }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
  if (isServing) return;
  if (queue.length === 0) {
    setShake(true);
  }

```

```

    setIsServing(false);
    setLog([' Столовая сброшена. Снова голодная толпа!
    setTimeout(() => {
      setQueue(prev => prev.map(c => ({ ...c, animState
    }, 500));
  });

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-indigo-950/40 border border-in
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-indigo-400 tex
          <p className="text-xs text-slate-300 mt-1 lea
            В очереди за супом всё честно. Кто первый в
            Новые гости встают строго <b>в конец</b> оч
            Здесь нет обхода и блата – только строгий п
          </p>
        </div>
      </div>

      {/* УПРАВЛЕНИЕ */}
      <div className="flex items-center gap-3 flex-wrap
        <button
          onClick={enqueue}
          disabled={isServing}
          className="flex-1 min-w-[150px] bg-gradient-t
            d:opacity-40 disabled:cursor-not-allowed te
            ive:scale-95 transition-all cursor-pointer
          >
            <span> Встать в очередь (ENQUEUE)</span>
          </button>

        <button
          onClick={dequeue}
          disabled={isServing || queue.length === 0}
          className="flex-1 min-w-[150px] bg-gradient-t

```

```

        <div className="text-center">
          <span className="text-3xl block">    <
          <span className="text-[10px] font-bol
ase font-mono tracking-wider">
            ПОВАР
          </span>
        </div>
      </div>

```

```

    {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
    <div className="flex flex-col items-cente
      <span className="text-2xl animate-pulse
      <span className="text-[8px] font-mono t
    </div>

```

```

    {/* СПИСОК ОЧЕРЕДИ */}
    <div className="flex-1 flex items-end gap
      {displayQueue.length === 0 ? {
        <div className="flex-1 flex flex-col
          <span className="text-4xl mb-1"> </
          <p className="text-xs font-bold fon
          <p className="text-[10px]">Все сыты
        </div>
      } : {
        displayQueue.map(({customer, idx}) => {
          const isHead = idx === 0;
          const isTail = idx === displayQueue
          return (
            <div
              key={customer.id}
              className={`
                flex flex-col items-center ga
                ${customer.animState === 'in'
                ${customer.animState === 'eat
                ${customer.animState === 'out
              `}
            >
          {/* Облачко мыслей */}

```



```

    {/* Пол кухни */}
    <div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800" />
  </div>

  {/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
  <div className="md:col-span-3 bg-slate-900 rounded-3xl p-4" />
    <div className="absolute top-4 left-4 text-[11px] font-mono" />
      Сытые гости (Архив FIFO)
    </div>

    <div className="absolute top-4 right-4 flex items-center justify-center gap-2" />
      <div className="order-emerald-800/80 text-[9px] font-mono" />
        <Sparkles className="w-3.5 h-3.5 animate-pulse" />
        <span>СЫТЫ </span>
      </div>

    <div className="flex-1 flex flex-col justify-between p-4" />
      {servedHistory.length === 0 ? (
        <div className="flex-1 flex flex-col items-center justify-center gap-2" />
          <span className="text-5xl mb-2" />
          <p className="text-xs font-bold font-mono" />
          <p className="text-[10px] mt-1" />Здесь 6
        </div>
      ) : (
        <div className="flex flex-col gap-1.5 w-full" />
          {servedHistory.map((item, idx) => (
            <div
              key={idx}
              className="w-full py-1.5 px-3 rounded-3xl" />
            <div className="flex items-center gap-2 text-[11px] text-slate-300" />
              <span>{item}</span>
            </div>
          ))}
        </div>
      )}
    </div>
  </div>

```

```

    term_link: number;
    is_terminal: boolean;
    words: string[];
    depth: number;
    x?: number;
    y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR',
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: T
  const [simulationStep, setSimulationStep] = useState<
  const [bfsQueue, setBfsQueue] = useState<number[]>([
  const [currentNode, setCurrentNode] = useState<number
  const [speechText, setSpeechText] = useState<string>('
    'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
      буль!')
  );
  const [isTalking, setIsTalking] = useState<boolean>(f
  const [isBlinking, setIsBlinking] = useState<boolean>

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);

  // Эффект разговора при смене текста
  useEffect(() => {
    setIsTalking(true);
    const timer = setTimeout(() => setIsTalking(false),
    return () => clearTimeout(timer);
  }, [speechText]);

```

```

const layoutDFS = (u: number, depth: number) => {
  const childKeys = Object.values(newNodes[u].child
  newNodes[u].y = 40 + depth * paddingY;

  if (childKeys.length === 0) {
    newNodes[u].x = 40 + currentX * paddingX;
    currentX++;
  } else {
    let leftX = -1;
    let rightX = -1;
    childKeys.forEach(v => {
      layoutDFS(v, depth + 1);
      if (leftX === -1) leftX = newNodes[v].x!;
      rightX = newNodes[v].x!;
    });
    newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
    if (leftX === -1) currentX++;
  }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
  `Отлично! Я построил базовое префиксное дерево (БПД) для слова "Базис".  

  S, чтобы расставить fail-ссылки!`
);

};

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();

```

```

    });
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полный  

        ь полюбоваться таблицей переходов!'
    );
    return;
  }

  const r = bfsQueue[0];
  const newQueue = bfsQueue.slice(1);
  const updatedNodes = { ...nodes };
  const rNode = updatedNodes[r];
  const childrenIds = Object.values(rNode.children);

  let logs = `Рассматриваем вершину #${r} (${rNode.ch

  for (const [char, u] of Object.entries(rNode.children)) {
    newQueue.push(u);
    let v = rNode.fail;
    while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
      v = updatedNodes[v].fail;
    }
    const failTarget = updatedNodes[v].children[char];
    updatedNodes[u].fail = failTarget;

    // Рассчитываем терминальную ссылку (term_link)
    if (updatedNodes[failTarget].is_terminal) {
      updatedNodes[u].term_link = failTarget;
    } else {
      updatedNodes[u].term_link = updatedNodes[failTa
    }
  }

```

```

    </div>
  </div>

  { /* Верхняя панель: Говорящий лосось и диалоговое
  <div className="grid grid-cols-1 md:grid-cols-3 g
    { /* Аватар Лосося (Сеня) с анимацией моргания и
    <div className="flex flex-col items-center just
      <div className="w-40 h-40 relative flex items
        full border border-blue-500/30 shadow-inner
        { /* SVG Лосося */
        <svg
          viewBox="0 0 200 200"
          className={`w-36 h-36 transition-transform
            isTalking ? 'scale-105 drop-shadow-[0_0_
            ]`}
        >
          { /* Тело лосося */
          <ellipse cx="100" cy="100" rx="70" ry="45"
          <path d="M 40 85 Q 100 55 160 85 Q 100 115
            40 85"
            fill="#be123c"
            className="origin-[35px_100px] animate-
          />

          { /* Плавники */
          <path d="M 90 55 L 70 30 L 110 55 Z" fill=
          <path d="M 90 145 L 70 170 L 110 145 Z" f
          <path
            d="M 120 115 L 100 135 L 130 130 Z"
            fill="#f87171"
            className="origin-[120px_115px] animate-
          />

          { /* Глаз (с морганием) */
          <circle cx="145" cy="85" r="12" fill="#ff

```

```

        <Volume2 className={`w-4 h-4 ${isTalking ?
        <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leading-4"
      {speechText}
    </p>

    <div className="mt-4 pt-3 border-t border-slate-200"
      <div className="text-xs text-slate-400 flex justify-between"
        <Info className="w-3.5 h-3.5 text-blue-400" />
        Статус: {simulationStep === 0 ? 'Борется с
        оен!'}
      </div>

      <div className="flex gap-2">
        {simulationStep === 0 && (
          <button
            onClick={startBFS}
            className="bg-emerald-600 hover:bg-emerald-500
            shadow-lg shadow-emerald-900/40 transition-colors"
          >
            <Play className="w-4 h-4" /> Запустить
          </button>
        )}
        {simulationStep === 1 && (
          <button
            onClick={stepBFS}
            className="bg-blue-600 hover:bg-blue-500
            shadow-lg shadow-blue-900/40 transition-colors"
          >
            <span>Шаг BFS</span>
            <span className="bg-blue-800 px-1.5 py-0.5" />
          </button>
        )}
        {simulationStep === 2 && (
          <button
            onClick={() => buildInitialTrie(words)}

```

```

    { /* Draw Edges */ }
    { Object.values(nodes).map(node => {
      if (node.id === 0) return null;
      const parent = nodes[node.parent];
      if (!parent.x || !parent.y || !node.x || !node.y) return null;

      const isCurrent = currentNode === node;

      return (
        <g key={`edge-${node.id}`}>
          <line
            x1={parent.x}
            y1={parent.y + 16}
            x2={node.x}
            y2={node.y - 16}
            stroke={isCurrent ? '#60a5fa' : '#ccc'}
            strokeWidth="2"
            markerEnd="url(#arrowhead)"
          />
          <text
            x={({parent.x + node.x} / 2 - 10)}
            y={({parent.y + node.y} / 2)}
            fill="#94a3b8"
            fontSize="12"
            fontWeight="bold"
          >
            {node.char}
          </text>
        </g>
      );
    }) }

    { /* Draw Fail Links */ }
    { simulationStep > 0 && Object.values(nodes).map(node => {
      if (node.id === 0 || node.fail === 0) return null;
      // n draw all! But maybe wait, we can just draw the fail link
      // Wait, all nodes get fail link. Let's draw it when simulationStep
      // step >= 1 and computed it
    }) }
  </div>
</pre>

```

```

        fill = '#059669';
        stroke = '#34d399';
    }

    return (
      <g key={`node-${node.id}`}>
        <circle
          cx={node.x}
          cy={node.y}
          r="16"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
        />
        <text
          x={node.x}
          y={node.y + 4}
          fill="white"
          fontSize={isRoot ? "10" : "12"}
          fontWeight="bold"
          textAnchor="middle"
        >
          {isRoot ? 'R' : node.char}
        </text>
        {!isRoot && (
          <text
            x={node.x + 12}
            y={node.y - 12}
            fill="#94a3b8"
            fontSize="9"
          >
            {node.id}
          </text>
        )}
      </g>
    );
  }}}
</svg>

```



```

        className="bg-slate-800 border border-slate-900 border-
outline-none focus:border-blue-500"
      />
      <button
        type="submit"
        className="bg-slate-700 hover:bg-slate-600 transition-colors"
      >
        <Plus className="w-3.5 h-3.5" /> Добавить
      </button>
    </form>
  </div>

  { /* Таблица Вершин и суффиксных ссылок */ }
  <div className="lg:col-span-2 bg-slate-900/50 p-4" >
    <h5 className="text-white font-bold mb-3 flex" >
      <span className="flex items-center gap-1.5" >
        <span> </span> Таблица переходов и fail-
      </span>
      <span className="text-xs text-slate-400 font-normal" >
        Всего вершин: {Object.keys(nodes).length}
      </span>
    </h5>

    <div className="max-h-[220px] overflow-y-auto" >
      <table className="w-full text-left border-collapse" >
        <thead>
          <tr className="border-b border-slate-700" >
            <th className="py-2 px-3" >ID Вершины</th>
            <th className="py-2 px-3" >Символ</th>
            <th className="py-2 px-3" >Родитель</th>
            <th className="py-2 px-3" >fail-ссылка</th>
            <th className="py-2 px-3" >term_link</th>
            <th className="py-2 px-3" >Дети</th>
            <th className="py-2 px-3" >Слова {dict
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-800" >

```

```

        <td className="py-2.5 px-3">
          {node.id === 0 ? {
            <span className="text-slate-500">
              {node.fail}
            </span>
          } : {
            <span className="bg-indigo-950 text-white" style={{
              transition: 'background-color 0.2s, color 0.2s',
              cursor: 'help'
            }} title={node.term_link}>
              {node.term_link}
            </span>
          }}
        </td>
        <td className="py-2.5 px-3">
          {node.id === 0 || node.term_link ? {
            <span className="bg-rose-950 text-white" style={{
              transition: 'background-color 0.2s, color 0.2s',
              cursor: 'help'
            }} title={node.term_link}>
              {node.term_link}
            </span>
          } : {
            <span className="bg-rose-950 text-white" style={{
              transition: 'background-color 0.2s, color 0.2s',
              cursor: 'help'
            }} title={node.term_link}>
              {node.term_link}
            </span>
          }}
        </td>
        <td className="py-2.5 px-3 text-slate-500">
          {Object.keys(node.children).length === 0 ? '0' : Object.entries(node.children)
            .map(([c, id]) => `${c}→#${id}`)
            .join(', ')}
        </td>
        <td className="py-2.5 px-3">
          {node.words.length === 0 ? {
            <span className="text-slate-600">
              {node.words}
            </span>
          } : {
            <span className="bg-emerald-900 text-white" style={{
              transition: 'background-color 0.2s, color 0.2s',
              cursor: 'help'
            }} title={node.term_link}>
              {node.words}
            </span>
          }}
        </td>
      </tr>
    </tbody>
  </table>
);
}}

```

```

    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
  const n = arr.length;
  const steps: Step[] = [];
  let id = 0;
  const tree: Record<number, number> = {};

  const CODE = [
    "build(v, l, r) {", // 1
    "  if (l === r) {", // 2
    "    tree[v] = A[l]; return", // 3
    "  }", // 4
    "  mid = (l + r) >> 1", // 5
    "  build(2v, l, mid)", // 6
    "  build(2v+1, mid+1, r)", // 7
    "  tree[v] = tree[2v]+tree[2v+1]", // 8
    "}", // 9
  ];
  void CODE;
}

```

```

function go(v: number, l: number, r: number) {
  if (l === r) {
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
      , highlightNodes: [v], arrayHighlight: [l, r] });
    tree[v] = arr[l];
    steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}` });
  }
}

```

```

const m = (l + r) >> 1;
steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
  codeLine: 6, treeState: tree, highlightNodes: [v] });
const left = go(2 * v, l, m, qL, qR);
const right = go(2 * v + 1, m + 1, r, qL, qR);
const res = left + right;
steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
  rgeChildren: [2 * v, 2 * v + 1], arrayHighlight: [v] });
return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]:`,
  highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
  const n = arr.length;
  // Build iterative tree: leaves at [n..2n-1]
  const tree: Record<number, number> = {};
  for (let i = 0; i < n; i++) tree[n + i] = arr[i];
  for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

  const steps: Step[] = [];
  let id = 0;
  let l = qL + n;
  let r = qR + n + 1; // half-open [l, r)
  let res = 0;

  steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

  while (l < r) {
    const highlights: number[] = [];
    if (l & 1) {
      res += tree[l] ?? 0;
      steps.push({ id: id++, desc: `l=${l} нечётный → l++`, codeLine: 4, treeState: tree, highlightNodes: [l] });
      l++;
    }
    if (r & 1) {
      res += tree[r] ?? 0;
      steps.push({ id: id++, desc: `r=${r} нечётный → r--`, codeLine: 5, treeState: tree, highlightNodes: [r] });
      r--;
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `m = (l + r) >> 1`, codeLine: 6, treeState: tree, highlightNodes: [m] });
    const left = go(2 * m, l, m, qL, qR);
    const right = go(2 * m + 1, m + 1, r, qL, qR);
    const res = left + right;
    steps.push({ id: id++, desc: `res = left + right`, codeLine: 7, treeState: tree, highlightNodes: [res] });
    return res;
  }
}

```

```

" // -----",
" mid = (l + r) >> 1;",
" build(2*v, l, mid);",
" build(2*v+1, mid+1, r);",
" tree[v] = tree[2*v] + tree[2*v+1];",
"}"
];
const QUERY_TD_CODE = [
"query(v, l, r, ql, qr) {",
"  if (ql > r || qr < l) return 0;",
"  // -----",
"  if (ql <= l && r <= qr) return tree[v];",
"  // -----",
"  mid = (l + r) >> 1;",
"  // спускаемся в обоих детей",
"  return query(left) + query(right);",
"}"
];
const QUERY_BU_CODE = [
"queryBU(ql, qr) {",
"  res = 0;",
"  l = ql + n; r = qr + n + 1;",
"  // -----",
"  while (l < r) {",
"    if (l & 1) res += tree[l++];",
"    // -----",
"    if (r & 1) res += tree[--r];",
"    l >>= 1; r >>= 1;",
"  }",
"  return res; }"
];

```

// ===== STEP GENERATOR: массовое обновление с Lazy Propagation

```

const LAZY_CODE = [
"update(v, l, r) {", // 1
"  if (r < uL || uR < l) return", // 2
"  if (uL <= l && r <= uR) {", // 3
"    tree[v] += add * (r - l + 1)", // 4
"    lazy[v] += add // обещание", // 5

```

```

function pushDown(v: number, l: number, r: number) {
  if (lazy[v] !== 0) {
    const m = (l + r) >> 1;
    const d = lazy[v];
    snap(`push(${v}): обещание +${d} с узла ${v} уходит
      lazy.` , 8, [v, 2 * v, 2 * v + 1], { promisePush: true });
    tree[2 * v] = (tree[2 * v] ?? 0) + d * (m - l + 1);
    lazy[2 * v] = (lazy[2 * v] ?? 0) + d;
    tree[2 * v + 1] = (tree[2 * v + 1] ?? 0) + d * (r - m);
    lazy[2 * v + 1] = (lazy[2 * v + 1] ?? 0) + d;
    lazy[v] = 0;
    snap(`Обещание ${v} исполнено и снято (lazy[${v}])`);
  }
}

function update(v: number, l: number, r: number) {
  if (r < uL || uR < l) {
    snap(`update(v=${v}, [${l}..${r}]): отрезок не пересекся`);
    return;
  }
  if (uL <= l && r <= uR) {
    tree[v] = (tree[v] ?? 0) + add * (r - l + 1);
    lazy[v] = (lazy[v] ?? 0) + add;
    snap(`update(v=${v}, [${l}..${r}]): отрезок ЦЕЛИКОМ попал
      обещание lazy[${v}] = ${lazy[v]}. Детям сюда с обещанием`);
    return;
  }
  pushDown(v, l, r);
  const m = (l + r) >> 1;
  snap(`update(v=${v}, [${l}..${r}]): частичное пересечение`);
  update(2 * v, l, m);
  update(2 * v + 1, m + 1, r);
  tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  snap(`Возврат в v=${v}: tree[${v}] = tree[${2 * v}]`);
}

```

```

useEffect(() => {
  if (!playing || idx >= steps.length - 1) { if (idx <
    const t = setTimeout(() => setIdx(i => i + 1), speed);
    return () => clearTimeout(t);
  }, [playing, idx, steps, speed]);

  const step = steps[idx] || steps[0];
  if (!step) return null;

  const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
    border-indigo-500 bg-indigo-600', childRing: 'ring-
    -900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
    30 border-emerald-500 bg-emerald-600', childRing: '
    -slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
    der-amber-500 bg-amber-600', childRing: 'ring-rose-!
    [color];

  return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
  'indigo' | 'emerald' | 'amber'; arr: number[]; liveN
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

  const vTree = useMemo(() => buildVisualTree(step.tree
  const renderNode = useCallback((nd: VNode): React.Re
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const isLive = liveNode !== null && liveNode !== un
    const has = nd.val !== null;
    const promise = step.lazyState?.[nd.v] ?? 0;

```

```

    -emerald-950/50' : 'bg-amber-950/50'} border-b
    <h4 className="font-bold text-sm text-white fle
    <span className="text-[10px] font-mono text-sla
    </div>

```

```

{/** Array strip */}
<div className="px-3 py-2 bg-slate-950 border-b b
  {arr.map((v, i) => {
    const inRange = step.arrayHighlight && i >= s
    return (
      <div key={i} className={`flex flex-col item
      go' ? 'bg-indigo-950 border-indigo-500' : c
      er-500'} -translate-y-0.5` : 'bg-slate-900 l
      <span className="text-[7px] text-slate-50
      <span className="text-[11px] font-bold for
      </div>
    );
  })}
</div>

```

```

{/** Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-sla
  <div className="scale-[0.85] origin-top min-w-m
  </div>

```

```

{/** Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10p
  x">
  {code.map((line, i) => {
    const ln = i + 1;
    const active = step.codeLine === ln;
    return (
      <div key={ln} className={`px-2 py-0.5 round
      2 border-indigo-500 text-indigo-200' : colo
      ' : 'bg-amber-600/20 border-l-2 border-ambe
      <span className="inline-block w-4 text-sl
      </div>
    );
  })}

```



```

        </div>

        {/* Progress bar */}
        <div className="h-1 bg-slate-950"><div className=
          + 1) / total) * 100 : 0}%` }} /></div>
      </div>
    );
  }

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const runtime = useVizRuntime();
  const liveI = vizNumber(runtime?.variables?.i);
  const liveV = vizNumber(runtime?.variables?.v);
  const liveAdd = vizNumber(runtime?.variables?.add);
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
  const [customInput, setCustomInput] = useState('5, 8,');
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [uL, setUL] = useState(1);
  const [uR, setUR] = useState(3);
  const [uAdd, setUAdd] = useState(10);
  const [view, setView] = useState<'build' | 'queries'>

  // Clamp query range when array changes
  useEffect(() => {
    setQL(prev => Math.min(prev, arr.length - 1));
    setQR(prev => Math.min(prev, arr.length - 1));
  }, [arr]);

  const handleApply = (e: React.FormEvent) => {
    e.preventDefault();
    const parsed = customInput.split(',').map(s => parseInt(s));
    if (parsed.length >= 2 && parsed.length <= 16) {
      setArr(parsed);
    }
  }
}

```

```

        />
        <button type="submit" className="bg-indigo-500 text-white py-2 px-4 rounded">Применить</button>
    </div>
    <div className="flex flex-wrap gap-1.5 pt-1">
        {arr.map((v, i) => (
            <span key={i} className={`border px-2 py-1 text-sm text-slate-500 ring-2 ring-amber-400/30`} : "liveI === i ? "text-indigo-500" : ""`}>
                <span className={liveI === i ? "text-indigo-500" : "text-slate-500"}>{v}</span>
            </span>
        ))}
        <span className="text-xs text-slate-500 font-medium">{arr.length}</span>
    </div>
</form>

{/* Query range */}
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
    <label className="text-[10px] font-bold uppercase">Запрос</label>
    <div className="flex items-center gap-2">
        <div className="flex items-center gap-1">
            <span className="text-xs text-slate-400">От</span>
            <input
                type="number"
                min={0}
                max={arr.length - 1}
                value={qL}
                onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
                className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
            />
        </div>
        <div className="flex items-center gap-1">
            <span className="text-xs text-slate-400">До</span>
            <input
                type="number"
                min={0}
                max={arr.length - 1}
                value={qR}
            />
        </div>
    </div>

```

```

{view === 'build' && {
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=" "
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
}}

{view === 'queries' && {
  <div className="grid grid-cols-1 xl:grid-cols-2"
    <SimPanel
      key={`qtd-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху`
      icon="↑ "
      steps={queryTDSteps}
      code={QUERY_TD_CODE}
      color="emerald"
      arr={arr}
    />
    <SimPanel
      key={`qbu-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Снизу`
      icon="↑ "
      steps={queryBUSteps}
      code={QUERY_BU_CODE}
      color="amber"
      arr={arr}
    />
  </div>
}}

{view === 'lazy' && {
  <div className="space-y-4">
    <div className="bg-slate-950 p-4 rounded-xl b

```

```

<div className="space-y-5">
  {/* Why it splits this way */}
  <div className="bg-slate-950 p-5 rounded-xl b
    <h4 className="text-base font-bold text-whi
    во разбилось именно так для N={arr.length}?
    <p className="text-sm text-slate-300 leadin
      Каждый узел берёт <code className="bg-sla
      к получает <code className="bg-slate-900 px
      -slate-900 px-1 rounded font-mono text-emerg
      евая часть забирает на 1 больше (округление
    </p>
    <div className="bg-slate-900 p-4 rounded-lg
      <div className="text-slate-400">Корень: L
      <div className="text-slate-300">mid = |(0
      <div className="text-indigo-300">→ Левый:
      <div className="text-emerald-300">→ Правы
      h - 1) >> 1)) эл.)</div>
      <div className="text-slate-500 mt-2">Всег
      } внутренних.</div>
    </div>
  </div>

```

```

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols
  <div className="bg-slate-950 p-5 rounded-xl
    <div className="absolute -top-3 left-4 bg
    e border border-emerald-500 shadow-md">
      1 Запрос Сверху-вниз (Рекурсия)
    </div>
    <p className="text-xs text-slate-300 mb-3
      Начинаем с корня. Если отрезок узла полн
      наче спускаемся в обоих детей.
    </p>
    <div className="space-y-1.5 text-xs">
      <div className="flex justify-between bo
      span className="text-rose-400 font-mono">0(
      <div className="flex justify-between bo
      ><span className="text-rose-400 font-mono">

```

ФАЙЛ 65/92: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";
```

```
interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId }) => {
  const [query, setQuery] = useState("");
```

```
  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    const map = new Map<string, typeof chapters>();
    for (const c of filtered) {
      const key = c.category?.trim() || "Прочие темы";
      if (!map.has(key)) map.set(key, []);
      (map.get(key) as typeof chapters).push(c);
    }
    return Array.from(map.entries());
  }, [query]);
```

```
  return (
    <aside className="w-full bg-slate-900/80 lg:bg-transparent" >
      <div className="p-4 pb-3 shrink-0">
        <h3 className="text-xs font-bold text-slate-400">
```

```

        onNavigate?.();
        window.scrollTo({ top: 0, behavior: 'smooth' });
    }}
    className={`w-full text-left px-3 py-2 border-1
        isSelected
            ? "bg-indigo-600/15 border-indigo-600/15"
            : "bg-slate-800/40 border-transparent"
    }`}
>
    {hasViz && {
        <span
            className="w-1.5 h-1.5 rounded-full border-1"
            title="Есть интерактивная визуализация"
        />
    }}
    <span className="font-semibold text-slate-300">
    <ChevronRight
        className={`w-4 h-4 shrink-0 mt-0.5
            isSelected ? "text-indigo-400" : "text-slate-300"
        }`>
    />
    </button>
    );
    }}}
</div>
</div>
    )}}
</div>

```

```

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-600/15 to-indigo-600/0"
    ] text-slate-300 space-y-1.5 hidden lg:block">
    <div className="font-bold text-indigo-400 flex items-center">
        <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
    </div>
    <p className="leading-relaxed">
        Подчёркнутые термины в тексте раскрываются по клику на иконку.
    </p>

```

```

        <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-indigo-300">Один
      </div>
    </div>
  </div>

  <div className="flex flex-wrap gap-2 mb-5">
    {[
      { id: "cases" as Tab, label: "1 · Механика: Z",
      { id: "big" as Tab, label: "2 · Большое дерев
      { id: "play" as Tab, label: "3 · Собери сам
    ]).map((t) => {
      <button
        key={t.id}
        onClick={() => setTab(t.id)}
        className={`px-4 py-2 rounded-lg text-xs font
          tab === t.id ? "bg-indigo-600 text-white"
        }`>
      >
        {t.label}
      </button>
    })}
  </div>

  {tab === "cases" && <SplayRotationsViz />}
  {tab === "big" && <SplayWalkViz />}
  {tab === "play" && <SplayRotationSandbox />}

  {tab === "play" && {
    <p className="text-xs text-slate-500 mt-3">
      Порядок поворотов в песочнице сверяется с пра
    </p>
  }}
</div>
);
};

export default SplayPlayground;

```

```

    const n2: TNode = { key: 2, left: n1, right: leaf(7) };
    const n4: TNode = { key: 4, left: n2, right: leaf(7) };
    return { key: 6, left: n4, right: leaf(7) };
  },
},
{
  name: "Змейка",
  hintCase: "Zig-Zag",
  target: 3,
  build: () => {
    const n3 = leaf(3);
    const n2: TNode = { key: 2, left: leaf(1), right: leaf(7) };
    const n4: TNode = { key: 4, left: n2, right: leaf(7) };
    return { key: 6, left: n4, right: leaf(7) };
  },
},
{
  name: "Один уровень",
  hintCase: "Zig",
  target: 2,
  build: () => ({ key: 4, left: { key: 2, left: leaf(1), right: leaf(7) }, right: leaf(7) });
},
{
  name: "Смешанный",
  hintCase: "Zig-Zag, затем Zig-Zig",
  target: 5,
  build: () => {
    const n5 = leaf(5);
    const n6: TNode = { key: 6, left: n5, right: leaf(7) };
    const n4: TNode = { key: 4, left: leaf(3), right: leaf(7) };
    return { key: 8, left: n4, right: leaf(9) };
  },
},
];

```

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode | null => {


```

-----
const p = path[path.length - 2];
const g = path.length >= 3 ? path[path.length - 3] : null;

if (!g) return { node: x.key, kind: "Zig" };

const xLeft = p.left === x;
const pLeft = g.left === p;
if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
return { node: x.key, kind: "Zig-Zag (сначала нижняя ветвь)" };
}

/* ----- раскладка ----- */

interface Placed {
  key: number;
  x: number;
  y: number;
  depth: number;
}

function layout(root: TNode | null): { nodes: Placed[]; edges: [number, number][] } {
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth + 1)));
  const colW = 46;

```



```

        style={{ transition: "all 700ms cubic-bezier(0.4, 0.0, 0.2, 1.0)"
      }}
    );
  }}}

  {nodes.map((n) => {
    const isTarget = n.key === target;
    const clickable = !solved && n.depth > 0;
    return (
      <g
        key={n.key}
        onClick={() => clickable && clickNode(n)}
        style={{
          transform: `translate(${n.x}px, ${n.y}px)`,
          transition: "transform 700ms cubic-bezier(0.4, 0.0, 0.2, 1.0)",
          cursor: clickable ? "pointer" : "default",
        }}
      >
        {isTarget && (
          <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
        )}
        <circle
          r={16}
          fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475568"}
          stroke={isTarget ? "#a5b4fc" : "#475568"}
          strokeWidth={2}
        />
        <text textAnchor="middle" dominantBaseline="middle">
          {n.key}
        </text>
      </g>
    );
  })}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
  <button

```

```

    {solved && (
      <div
        className={`mt-3 rounded-lg border px-3 py-2`}
        wrong === 0
          ? "bg-emerald-950/40 border-emerald-700/60"
          : "bg-rose-950/30 border-rose-800/60 text-rose-500"
        `)}
      >
        <div className="flex items-center gap-2 font-mono">
          {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500"/> : <X className="w-4 h-4 text-rose-500"/>}
          {wrong === 0
            ? `Узел ${target} в корне за ${moves.length} ходов`
            : `Узел ${target} в корне за ${moves.length} ходов`
          }
        </div>
        {wrong > 0 && (
          <ul className="list-disc pl-5 space-y-0.5">
            {moves.map(
              (m, i) =>
                !m.correct && (
                  <li key={i}>
                    ход {i + 1}: крутили <span className="font-mono">{m.expected}</span>
                  </li>
                )
            )}
          </ul>
        )}
      </div>
    )}
    {wrong === 0 && moves.length > minimal && (
      <div className="opacity-80">Порядок верный,
    </div>
    )}
  </div>
);
};

```

```

interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
  frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
  pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
  edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x – уже корень, деда нет",
  rotations: "1 поворот",
  keys: { x: 20, p: 40 },
  inorder: ["A", "20", "B", "40", "C"],
  ranges: "A < 20 · B между 20 и 40 · C > 40",
  frames: [
    {
      ...ZIG_BEFORE,
      kind: "start",

```

```

label: "Zig-Zig",
when: "x и p — дети с ОДНОЙ стороны (оба слева или оба справа)",
rotations: "2 поворота, первым — верхний",
keys: { x: 20, p: 40, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
    {
        ...ZZ_S0,
        kind: "start",
        title: "Было: прямая линия 60 → 40 → 20",
        text: "Три узла выстроились в «бамбук»: каждый следующий узел  
зглаживает.",
        ms: RESULT_MS,
    },
    {
        ...ZZ_S0,
        kind: "aim",
        title: "Прицел 1: ВЕРХНЕЕ ребро 60 — 40",
        text: "Главная хитрость этого случая: первым крутит 60, а не 40",
        focus: ["g", "p"],
        ms: AIM_MS,
    },
    {
        ...ZZ_S1,
        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему по  
убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 — 20",
        text: "А теперь обычный одиночный поворот — тот же самый",
        focus: ["p", "x"],
    },
]

```

```

        title: "Было: 60 → влево на 20 → вправо на 40",
        text: "Узлы идут «змейкой»: сначала шаг влево, по
            ",
        ms: RESULT_MS,
    },
    {
        ...ZG_S0,
        kind: "aim",
        title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
        text: "Дед (60) стоит на месте и в этом повороте
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        ...ZG_S1,
        kind: "result",
        title: "Поворот 1: змейка выпрямилась",
        text: "40 заняло место 20 и забрало его к себе ле
        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и
        focus: ["g", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [180, 50], p: [90, 130], g: [275, 130],
        edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
        kind: "result",
        title: "Стало: 20 и 60 – два сына сорока",
        text: "Дерево стало идеально симметричным: все че
            очку». ",
        moved: ["x", "p", "g", "C"],
        ms: RESULT_MS,
    }

```

```

    const r = isSubtree(id) ? 15 : 20;
    const dx = to[0] - from[0];
    const dy = to[1] - from[1];
    const len = Math.hypot(dx, dy) || 1;
    const pad = r + 8;
    return (
      <g key={`ghost-${id}`} opacity={0.55}>
        <circle cx={from[0]} cy={from[1]} r={r} fill="white"
          {len > pad * 2 + 6 && (
            <line
              x1={from[0] + (dx / len) * pad}
              y1={from[1] + (dy / len) * pad}
              x2={to[0] - (dx / len) * pad}
              y2={to[1] - (dy / len) * pad}
              stroke="#a5b4fc"
              strokeWidth={1.5}
              strokeDasharray="4 3"
              markerEnd="url(#splay-arrow)"
            />
          )}
        </g>
      );
    )}
  )}
}

```

```

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];
  if (!pa || !pb) return null;
  const hot = Boolean(focus && focus.includes(a));
  return (
    <line
      key={` ${a} - ${b} - ${i} `}
      x1={pa[0]}
      y1={pa[1]}
      x2={pb[0]}
      y2={pb[1]}
      stroke={hot ? "#fbbf24" : "#475569"}
      strokeWidth={hot ? 5 : 2}
    />
  );
})}

```



```

        </g>
      );
    }
  </svg>
);
};

export const SplayRotationsViz: React.FC = () => {
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveCase = vizString(vars?.case);
  const liveFrame = vizNumber(vars?.i);
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

  // Компилятор: case = "zig" / "zig-zig" / "zig-zag",
  useEffect(() => {
    if (liveCase) {
      const ci = CASES.findIndex((c) => c.key.toLowerCase === liveCase);
      if (ci >= 0) setCaseIdx(ci);
    }
  }, [liveCase]);
  useEffect(() => {
    if (liveFrame !== null) setFrame(Math.min(liveFrame, CASES[caseIdx].frames.length - 1));
  }, [liveFrame, caseIdx]);

  const active = CASES[caseIdx];
  const total = active.frames.length;
  const idx = Math.min(frame, total - 1);
  const current = active.frames[idx];
  const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
  const duration = useMemo(() => Math.round(current.ms - prevPos), [current, prevPos]);
  const last = idx === total - 1;

  useEffect(() => {
    setFrame(0);
  });
};

```

```

        : "bg-indigo-500/20 text-indigo-300"
      }` }
    >
    {current.kind === "aim" ? "прицел" : current.title}
  </span>
  <span className="text-sm font-bold text-white">
    Шаг {idx + 1} из {total}. {current.title}
  </span>
</div>
<p className="text-[13px] text-slate-400 leading-relaxed">
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
  <Tree frame={current} prev={prevPos} keys={activeKeys}>

    {/* обход слева направо – одинаковый на всех кадрах */}
    <div className="mt-2 pt-2 border-t border-slate-800">
      <div className="flex items-center gap-1.5 flex-wrap">
        <span className="text-slate-500 mr-1">Обход по шагам</span>
        {active.inorder.map((t, i) => (
          <span
            key={i}
            className={`px-1.5 py-0.5 rounded font-mono text-sm ${
              /\d/.test(t) ? "bg-emerald-500/15 text-white" : "text-slate-400"
            }`}
          >
            {t}
          </span>
        ))}
        <span className="text-emerald-400/80 ml-1">или по кнопкам</span>
      </div>
      <p className="text-[11px] text-slate-500 mt-1">
        {current.title}
      </p>
    </div>

    {/* индикатор автопрокрутки */}
    <div className="h-1 w-full bg-slate-800 rounded-full">
      <div

```

```

        {playing ? "Стоп" : "Авто"}
    </button>
    <button
      onClick={() => setSlow((s) => !s)}
      title="Автопрокрутка ещё медленнее"
      className={`flex items-center gap-1.5 px-3 py-3
        slow
          ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
          : "bg-slate-800 border-slate-700 text-slate-200"}
    >
      <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.5"}
    </button>
    <button
      onClick={() => {
        setFrame(0);
        setPlaying(false);
      }}
      className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors"
    >
      <RotateCcw className="w-3.5 h-3.5" /> Сброс
    </button>

    <div className="flex items-center gap-3 ml-auto"
      {([ "x", "p", "g" ] as const).map((id) =>
        active.keys[id] === undefined ? null : (
          <span key={id} className="flex items-center justify-center"
            <span className="w-3 h-3 rounded-full" style={styles[id]} />
          </span>
        )
      )}
    </div>
  </div>

  <p className="mt-2 text-[11px] text-slate-500">
    Листай кнопкой «Дальше» в своём темпе – кадры не
  </p>

```

```

    if (l) {
        n.left = l;
        l.parent = n;
    }
    if (r) {
        n.right = r;
        r.parent = n;
    }
    return n;
}

export function bstInsert(root: SN | null, key: number) {
    if (!root) return { key, left: null, right: null, parent: null };
    const r = cloneWithParents(root)!;
    let cur: SN = r;
    for (;;) {
        if (key < cur.key) {
            if (!cur.left) {
                cur.left = { key, left: null, right: null, parent: cur };
                return r;
            }
            cur = cur.left;
        } else {
            if (!cur.right) {
                cur.right = { key, left: null, right: null, parent: cur };
                return r;
            }
            cur = cur.right;
        }
    }
}

/** Поднять child над его родителем (один атомарный поворот)
export function rotateUp(child: SN): void {
    const p = child.parent;
    if (!p) return;
    const g = p.parent;
    if (p.left === child) {

```

```

    }
    return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
  kind: SplayStepKind;
  root: SN | null;
  aimEdge?: [number, number];
  /** Кадр cut: ребро, по которому отстёгивается среднее */
  cutEdge?: [number, number];
  cutLabel?: string;
  /** Куда β переедет: сторона у верхнего узла. */
  cutDest?: string;
  pathKeys?: number[];
  caseLabel?: string;
  note: string;
  rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
  rootIn: SN | null,
  key: number,
  isInsert = false,
): { steps: SplayStep[]; found: boolean } {
  let root: SN | null = cloneWithParents(rootIn);
  const steps: SplayStep[] = [];
  if (!root)
    return { steps: [{ kind: "done", root: null, note:

  // --- спуск ---
  let cur: SN | null = root;
  const pathKeys: number[] = [];
  let found = false;
  let last: SN = root;
  while (cur) {
    last = cur;

```

```

    steps.push({
      kind: "cut",
      root: cloneWithParents(root)!,
      cutEdge: [child.key, mid ? mid.key : child.key],
      cutLabel: mid ? `β = поддерезо ${mid.key}` : "β п",
      cutDest: mid ? (p.left === child ? "справа у p" :
      caseLabel: label,
      note: mid
        ? `Отстёгиваем среднее поддерезо β = ${mid.key}
          й у них общий – после поворота переедет ${p.l
          : `Среднего поддереза нет (β пусто): ${child.ke
      rotCount,
    });
  const before = collectEdges(root);
  rotateUp(child);
  fixRoot();
  rotCount += 1;
  const diff = diffEdges(before, collectEdges(root));
  steps.push({
    kind: "rot",
    root: cloneWithParents(root)!,
    caseLabel: label,
    note: `Поворот: ${child.key} встал над ${p.key}.
    rotCount,
  });
};

while (target.parent) {
  const p = target.parent;
  const g = p.parent;
  if (!g) {
    emitRotation(
      target,
      "Zig",
      [p.key, target.key],
      `Zig: у ${target.key} нет деда – хватит одного
    );
    continue;
  }
}

```

```

        caseLabel: "Zig-Zag",
        note: `Второе ребро: ${target.parent.key}—${target.key}
        rotCount,
    });
    emitRotation(
        target,
        "Zig-Zag",
        [target.parent.key, target.key],
        `Доводим: ${target.key} над ${target.parent.key}
    );
}
}

steps.push({
    kind: "done",
    root: cloneWithParents(root)!,
    note: `    ${target.key} в корне за ${rotCount} ${rotCount}
    rotCount,
});
return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
    let d = 0;
    let cur = t;
    while (cur) {
        if (key === cur.key) return d;
        cur = key < cur.key ? cur.left : cur.right;
        d += 1;
    }
    return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
    t.key, ...inorderKeys(t.left), ...inorderKeys(t.right)

export const INITIAL KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

```

```

x1="30" y1="86" x2="48" y2="126"
stroke={isCut ? "#f59e0b" : "#64748b"}
strokeWidth={isCut ? 3 : 2}
strokeDasharray={isCut ? "5,3" : "none"}
/>
<circle cx="55" cy="46" r="15" fill="#1e293b"
<text x="55" y="50" textAnchor="middle" fill=
<circle cx="30" cy="86" r="15" fill="#78350f"
<text x="30" y="90" textAnchor="middle" fill=
<circle cx="12" cy="126" r="13" fill="#0f172a"
<text x="12" y="130" textAnchor="middle" fill=
<circle
  cx="48" cy="126" r="13"
  fill={isCut ? "#78350f" : "#0f172a"}
  stroke={isCut ? "#f59e0b" : "#64748b"}
  strokeWidth={isCut ? 2.5 : 1.5}
  strokeDasharray={isCut ? "4,3" : "none"}
/>
<text x="48" y="130" textAnchor="middle" fill=
<circle cx="80" cy="126" r="13" fill="#0f172a"
<text x="80" y="130" textAnchor="middle" fill=
<text x="98" y="92" fill={isCut ? "#f59e0b" :
  {isCut ? " β отстёрнута" : "β при x"}}
</text>
<text x="16" y="160" fill="#64748b" fontSize=
</g>
<text x="163" y="96" fill="#10b981" fontSize="2
<g transform="translate(186,6)">
  <text y="10" fill="#94a3b8" fontSize="11" fon
  <line x1="70" y1="46" x2="45" y2="86" stroke=
  <line x1="70" y1="46" x2="95" y2="86" stroke=
  <line x1="45" y1="86" x2="27" y2="126" stroke=
  <line
    x1="45" y1="86" x2="63" y2="126"
    stroke={isRot ? "#10b981" : "#64748b"}
    strokeWidth={isRot ? 3 : 2}
  />
  <circle cx="70" cy="46" r="15" fill="#059669"

```



```

    if (!n) return;
    walk(n.left, d + 1);
    pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d });
    slot += 1;
    walk(n.right, d + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const SplayWalkViz: React.FC = () => {
  const initial = useMemo(buildInitial, []);
  const [steps, setSteps] = useState<SplayStep[] | null>();
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [input, setInput] = useState("");
  const [treeVers, setTreeVers] = useState(0);
  const baseRef = useRef<SN>(initial);
  const timer = useRef<ReturnType<typeof setTimeout> | null>();

  // — Синхронизация с Python-компилятором: key (или x) — ключ,
  // i — перейти к кадру расшейвливания.
  const runtime = useVizRuntime();
  const liveKey = vizNumber(runtime?.variables?.key) ?? 0;
  const liveIdx = vizNumber(runtime?.variables?.i);
  const appliedKey = useRef<number | null>(null);
  useEffect(() => {
    if (liveKey !== null && appliedKey.current !== liveKey) {
      appliedKey.current = liveKey;
      runKeyRef.current?.(liveKey);
    }
  }, [liveKey]);
  useEffect(() => {
    if (liveIdx !== null && steps) setIdx(Math.min(liveIdx, steps));
  }, [liveIdx, steps]);

  const runKey = (key: number, isInsert = false) => {
    const { steps: st } = splaySteps(baseRef.current, key, isInsert);
  };
};

```

```

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.w);
const autoH = Math.max(300, ...nodes.map((n) => n.py);

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1]) : 0));
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1]) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1]) : 0));
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
  if (!steps || !step || idx <= 0) return null;
  return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {
  if (!prevEdgeSet) return 0;
  return edges.filter((e) => !prevEdgeSet.has(`${e.a}${e.b}`)).length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps[steps.length - 1].root) : ''));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
  const v = parseInt(input, 10);
  if (Number.isNaN(v)) return;
  const withNew = bstInsert(baseRef.current, v);
  baseRef.current = withNew;
  setTreeVers((t) => t + 1);
  runKey(v, true);
  setInput("");
};

const stepColor = (k: number): string => {
  if (!step) return "#0f172a";
  if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f1c40f";
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e[0] === k) return "#1e293b";
  }

```

```

        if (has) runKey(v);
        else doInsert();
        setInput("");
    }
}
}}
placeholder="ключ + Enter"
className="w-28 bg-slate-950 border border-
s:border-indigo-500"
/>
<button
  onClick={() => {
    baseRef.current = buildInitial();
    setSteps(null);
    setIdx(0);
    setPlaying(false);
    setTreeVers((v) => v + 1);
  }}
  className="px-3 py-2 rounded-lg text-xs fon
>
  Исходное
</button>
<button
  onClick={() => {
    const shuffled = [...INITIAL_KEYS].sort(() =>
    let r: SN | null = null;
    for (const k of shuffled) r = bstInsert(r, k);
    baseRef.current = r!;
    setSteps(null);
    setIdx(0);
    setPlaying(false);
    setTreeVers((v) => v + 1);
  }}
  className="px-3 py-2 rounded-lg text-xs fon
>
  Перемешать
</button>
</div>

```

Кликни по любому узлу – splay покадрово

</div>

}}

<svg viewBox={`0 0 \${autoW} \${autoH}`} className=

{edges.map((e) => {

const a = pos.get(e.a);

const b = pos.get(e.b);

if (!a || !b) return null;

const isAim = step?.aimEdge && step.aimEdge

const isCutEdge =

step?.kind === "cut" && step.cutEdge && s

.cutEdge[1];

const isMoved = !isAim && !isCutEdge && pre

return {

<g key={`e-\${e.a}-\${e.b}`}>

<line

x1={a.px} y1={a.py} x2={b.px} y2={b.py}

stroke={isAim ? "#f59e0b" : isCutEdge

strokeWidth={isAim ? 4 : isCutEdge ?

strokeDasharray={isAim ? "6,4" : isCu

className="transition-all duration-500

/>

{isCutEdge && (

<text x={({a.px + b.px} / 2 + 14} y={({

nospace" className="transition-all duration

{step?.cutLabel} → {step?.cutDest

</text>

))

</g>

);

}}}

{nodes.map((n) => {

const aimE = step?.aimEdge ?? (step?.kind =

const isAimNode = aimE?.includes(n.key);

return {

<g

key={`n-\${n.key}-\${treeVers}`}

transform={`translate(\${n.px}, \${n.py})`}

```

        {step ? step.note : "Клики по узлу или в
ке справа – механика одного поворота."}
    </p>
    {steps && step && step.kind !== "search" &&
      <p className={`text-xs mt-2 ${checkOk ? "
        {checkOk ? " " : " "} проверка кадра: o
        {inorderKeys(viewRoot).length}
        {movedCount > 0 ? ` · переехало рёбер:
      </p>
    }}
  </div>
  <div className="w-full lg:w-[300px] shrink-0">
    <MechanismInset step={step} />
  </div>
</div>
</div>

```

```

{ /* Счётчики + код */ }
<div className="flex flex-col lg:flex-row gap-6 m
  <div className="w-full lg:w-1/2 bg-rose-950/10
    <h4 className="text-sm font-bold text-rose-300
      {steps ? {
        <table className="w-full text-sm">
          <tbody className="font-mono">
            <tr className="border-b border-slate-800">
              <td className="py-2 text-slate-400">r
              <td className="py-2 text-right text-s
            </tr>
            <tr className="border-b border-slate-800">
              <td className="py-2 text-slate-400">v
              <td className="py-2 text-right text-s
            </tr>
            <tr>
              <td className="py-2 text-slate-400">n
              <td className="py-2 text-right text-s
            </tr>
          </tbody>
        </table>

```

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizStack';

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pa', isDirty: true },
  { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-f', isDirty: true },
  { name: 'Блюдец от торта', emoji: '🍰', color: 'from-p', isDirty: true },
  { name: 'Тарелка от тако', emoji: '🌮', color: 'from-y', isDirty: true },
  { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-s', isDirty: true },
  { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-e', isDirty: true },
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>([
    { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'idle' },
    { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'idle' },
    { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', animState: 'idle' },
  ]);

  const runtime = useVizRuntime();
  const liveStack = vizArray(runtime?.variables?.st) ?? [];
  const displayStack: Plate[] = liveStack
```

```

    setDirtyStack(prev => [...prev, newPlate]);
    addLog(`  PUSH: Положили ${newPlate.emoji} сверху с

    setTimeout(() => {
      setDirtyStack(prev => prev.map(p => p.id === newP
    }, 500);
  }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${top

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id === topPla
setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and
  const cleanPlate: Plate = {
    ...topPlate,
    isDirty: false,
    animState: 'clean',
  };

  setCleanRack(prev => [cleanPlate, ...prev].slice(
  setDirtyStack(prev => prev.slice(0, prev.length -
  setShowSoap(false);

```

```

    { /* УПРАВЛЕНИЕ */ }
    <div className="flex items-center gap-3 flex-wrap"
      <button
        onClick={pushPlate}
        disabled={isWashing}
        className="flex-1 min-w-[150px] bg-gradient-to-r
          from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
          font-medium rounded-2xl scale-95 transition-all cursor-pointer flex items-center justify-center"
      >
        <span> Положить грязную (PUSH)</span>
      </button>

      <button
        onClick={popPlate}
        disabled={isWashing || dirtyStack.length === 0}
        className="flex-1 min-w-[150px] bg-gradient-to-r
          from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
          font-medium rounded-2xl scale-95 transition-all cursor-pointer flex items-center justify-center"
      >
        <span> Помыть верхнюю (POP)</span>
      </button>

      <button
        onClick={reset}
        className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700"
      >
        Сброс
      </button>
    </div>

    { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
    <div className="grid grid-cols-1 md:grid-cols-12 gap-4"
      >
      { /* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */ }
      <div className="md:col-span-7 bg-slate-900 rounded-2xl p-4"
        >
        <div className="absolute top-4 left-4 text-[1.2em] font-medium"

```



```

        er gap-1">
            <ArrowUp className="w-3.5 h-3.5" />
            <span>ВЕРШИНА (TOP)</span>
        </span>
        <span className="text-blue-400 animate-pulse">
    </div>

    {/* Тарелки (отрисовываем в обратном порядке) */}
    {displayStack.slice().reverse().map((plate, idx) => {
        const isTop = idx === topIndex;
        return (
            <div
                key={plate.id}
                className={`
                    w-full max-w-[280px] h-10 rounded
                    shadow-md select-none transition-all duration-200
                    ${plate.color}
                    ${plate.animState === 'in' ? 'animate-in' : ''}
                    ${plate.animState === 'washing' ? 'animate-washing' : ''}
                    ${isTop ? 'ring-4 ring-blue-500/50' : ''}
                `}
            >
                <span className="text-lg">{plate.name}</span>
                <span className="text-slate-800 text-sm">
                    {isTop && (
                        <span className="text-[9px] bg-blue-500/50 px-1">
                            LIFO
                        </span>
                    )}
                </span>
            </div>
        );
    })}
    </div>
</div>
)}

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-200 to-slate-300">

```

```
      <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800">
    </div>
  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="text-[10px] font-mono text-slate-400">
      {log.map((entry, idx) => {
        <div
          key={idx}
          className={`text-xs font-mono flex items-center justify-between p-2`}
        >
          {idx === 0 ? <span className="text-blue-500 font-weight-bold">История действий</span> : null}
          <span>{entry}</span>
        </div>
      )})}
    </div>
  </div>
);
};
```

ФАЙЛ 71/92: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string): any[] {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];
```

```

const steps: any[] = [];

let l = 0, r = 0;
steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

for (let i = 1; i < n; i++) {
    steps.push({ i, l, r, z: [...z], desc: `Шаг i=$
    if (i <= r) {
        steps.push({ i, l, r, z: [...z], desc: `Вни
        !` });
        z[i] = Math.min(r - i + 1, z[i - l]);
        steps.push({ i, l, r, z: [...z], desc: `Пре
        }. Копипаста сработала.` });
    }

    steps.push({ i, l, r, zTarget: z[i], zSource: i
    while (i + z[i] < n && s[z[i]] === s[i + z[i]])
        steps.push({ i, l, r, zTarget: z[i], zSource
        падают! Окно растёт.` });
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource
        [i]]}' разные. Стоп.` });
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Упё
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксиру

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: ` 0
    }

```

```

        timer.current = setInterval(() => {
            setStepIdx(prev => {
                if (prev >= steps.length - 1) { set
                return prev + 1;
            });
        }, 1200);
    }
    return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.ha
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
    ? {
        ...baseStep,
        i: liveI ?? baseStep.i,
        j: liveJ ?? baseStep.j,
        l: liveL ?? baseStep.l,
        r: liveR ?? baseStep.r,
        [mode === "kmp" ? "pi" : "z"]: hasLiveValues
        desc: `Компилятор: i=${liveI ?? "-"}${mode ===
    }
    : baseStep;

return (
    <div className="space-y-4">

```

```

        // KMP Logic
        if (mode === "kmp") {
            if (step.highlight?.include
                borderColor = "border-a
                bgColor = step.match ===
-900/40");
        }
    }

    // Z Logic
    if (mode === "z") {
        if (index >= step.l && index
            bgColor = "bg-emerald-900
            borderColor = "border-e
        }
        if (step.zTarget === index
            borderColor = "border-a
            bgColor = step.match ===
-900/40");
    }
}

return {
    <div key={index} className=
        {/* L/R markers for Z */}
        {mode === "z" && step.l
            <div className="abs
        }}
        {mode === "z" && step.r
            <div className="abs
        }}

        {/* Index */}
        <span className={`text-

        {/* Char Box */}
        <div className={`w-8 h-

```

```

                                <span className=
                                </div>
                            }}
                            {mode === "z" && step.z}
                                <div className="abs
                                    <span className=
                                    </div>
                            }}

                            {/* Found flash effect
                            {step.found && mode ===
                                <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
                            }}
                            {step.found && mode ===
                                <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
                            }}
                        </div>
                    )
                }}}}
            </div>
        </div>

<div className="flex flex-col sm:flex-row g
    {/* Controls */}
    <div className="flex bg-slate-900 borde
        <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
            <Undo strokeWidth={3} size={16}
        </button>
        <button onClick={playPause} classNa
x items-center justify-center min-w-[80px]"
            {autoplay ? <><Pause size={16} <
        </button>
        <button onClick={nextStep} disabled
r:bg-slate-800 rounded-lg disabled:opacity-
            <SkipForward strokeWidth={3} si

```

```

        <span>Код C++ ({mode === 'kmp' ? 'КМП'
    </div>
    <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
    }
}`}

    </pre>
    </div>
    </div>
    );
}
```

ФАЙЛ 72/92: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
```

```
interface TooltipProps {
```

```
    /** Содержимое подсказки (можно JSX). */
```

```
    content: React.ReactNode;
```

```
    /** Элемент, на который наводят курсор / фокус. */
```

```
    children: React.ReactNode;
```

```
    /** С какой стороны показывать. По умолчанию — сверху
```

```
    });  
};
```

ФАЙЛ 73/92: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";  
import { useVizStepSync, vizArray, vizNumber } from "...";  
import {  
  Play,  
  Pause,  
  RotateCcw,  
  Clock,  
} from "lucide-react";
```

```
interface TNode {  
  id: number;  
  x: number;  
  y: number;  
  label: string;  
  name: string;  
}
```

```
interface TEdge {  
  u: number;  
  v: number;  
}
```

```
const dagNodes: TNode[] = [  
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },  
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },  
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },  
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },  
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },  
];
```



```

const dfsStack: number[] = [];

const recordStep = (desc: string, currentNode: number) => {
  listSteps.push({
    desc,
    visited: new Set(visited),
    fullyProcessed: new Set(fullyProcessed),
    currentNode,
    topoList: [...topoList],
    dfsStack: [...dfsStack],
  });
};

recordStep(
  "Начало топологической сортировки. Используем кла",
  null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
  visited.add(u);
  dfsStack.push(u);
  recordStep(
    `DFS: зашли в вершину ${u} ("${dagNodes[u].name",
    u,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs(to);
    } else if (!fullyProcessed.has(to)) {
      // Explaining potential cycle detected (not in
      recordStep(
        `⚠ Внимание: Рёбра в серую вершину ${to} о",
        u,
      );
    }
  }
  fullyProcessed.add(u);
  topoList.push(u);
};

dfs(0);

return topoList;
}

export default topologicalSort;

```

```

    topoList: [],
    dfsStack: [],
  };

  useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1) {
      timer = setInterval(() => {
        setCurrentStepIdx((prev) => prev + 1);
      }, 1600);
    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

  const togglePlay = () => setIsPlaying(!isPlaying);
  const reset = () => {
    setIsPlaying(false);
    setCurrentStepIdx(0);
  };

  const getTopoListString = () => {
    return [...step.topoList]
      .reverse()
      .map((id) => dagNodes[id].name)
      .join(" → ");
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Clock className="w-5 h-5 text-indigo-400" />
          Топологическая сортировка (Зависимости обучен
        </h3>

        <div className="flex items-center gap-2 bg-slate-

```

```

        </marker>
        <marker
          id="dag-arrow-active"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#808080" />
        </marker>
      </defs>

      {/* Edges */}
      {dagEdges.map(({edge, idx}) => {
        const u = dagNodes.find((n) => n.id === edge.u)
        const v = dagNodes.find((n) => n.id === edge.v)

        const isActive =
          (step.currentNode === edge.u || step.currentNode === edge.v) ||
          step.visited.has(edge.v);

        return (
          <line
            key={`edge-${idx}`}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isActive ? "#818cf8" : "#334155"}
            strokeWidth={isActive ? "3" : "2"}
            markerEnd={`url(#${isActive ? "dag-arrow-active" : "dag-arrow"})`}
            className="transition-all duration-300"
          />
        );
      })}
    )}
  )}
}

```

```

        <text
          x={node.x}
          y={node.y - 4}
          textAnchor="middle"
          fill="white"
          fontSize="11px"
          fontWeight="bold"
          className="pointer-events-none"
        >
          ({node.label}) {node.name.split(" "
        </text>
        <text
          x={node.x}
          y={node.y + 12}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="9px"
          className="pointer-events-none"
        >
          {node.name.split(" ").slice(1).join
        </text>
      </g>
    );
  }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800">
    <p className="text-xs text-indigo-300 min-h-[100px]">
      {step.desc}
    </p>

```

```

    </div>

    <div className="mt-4 pt-3 border-t border-slate-200">
      <span>
        Шаг {currentStepIdx + 1} из {steps.length}
      </span>
      <div className="flex gap-1">
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev) => prev - 1)}
          className="bg-slate-900 border border-slate-700 text-white"
        >
          Назад
        </button>
        <button
          disabled={currentStepIdx >= steps.length - 1}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-700 text-white"
        >
          Вперед
        </button>
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 74/92: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useState } from "react";
```

```

    { key: 9, grade: 3 },
    { key: 7, grade: 2 },
    { key: 12, grade: 0 },
    { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован
export const INPUT_ORDER: Pair[] = [
    { key: 3, grade: 5 },
    { key: 45, grade: 8 },
    { key: 3, grade: 6 },
    { key: 1, grade: -4 },
    { key: 7, grade: 2 },
    { key: 6, grade: 5 },
    { key: 2, grade: 4 },
    { key: 9, grade: 3 },
    { key: 12, grade: 0 },
];
// (3,5) дубль ключа/приоритета – уберём: ввод без повт
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => !p.key || !p.grade);

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) -----

export function insert(root: TNode | null, p: Pair): TNode | null {
    if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade };
    if (p.key <= root.key) root.left = insert(root.left, p);
    else root.right = insert(root.right, p);
    return root;
}

export function buildTree(ps: Pair[]): TNode | null {
    let root: TNode | null = null;
    for (const p of sortedByY(ps)) root = insert(root, p);
    return root;
}

```

```

    if (a.grade > b.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
        `ем его правое поддереву с (${b.key}, ${b.grade})`
      });
      a.right = go(a.right, b);
      return a;
    }
    trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
      ` (${a.key}, ${a.grade}) с его левым поддеревом.`
    });
    b.left = go(a, b.left);
    return b;
  };
  return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {
  const trace: { aId: NodeId | null; note: string }[] = [];
  const go = (t: TNode | null): TNode | null => {
    if (!t) return null;
    if (key < t.key) {
      trace.push({ aId: t.id, note: `${key} < ${t.key}` });
      t.left = go(t.left);
      return t;
    }
    if (key > t.key) {
      trace.push({ aId: t.id, note: `${key} > ${t.key}` });
      t.right = go(t.right);
      return t;
    }
    trace.push({ aId: t.id, note: `Нашли (${t.key}, ${t.grade})` });
    const m = mergeTree(t.left, t.right);
    trace.push(...m.trace.map((s) => ({ aId: s.chosenId, note: s.note })));
    return m.root;
  };
  return { root: go(root), trace };
}

const cloneTree = (t: TNode | null): TNode | null =>

```

```

const es: { from: NodeId; to: NodeId }[] = [];
const walk = (n: TNode | null) => {
  if (!n) return;
  if (n.left) { es.push({ from: n.id, to: n.left.id });
  if (n.right) { es.push({ from: n.id, to: n.right.id });
};
walk(root);
return es;
};

```

```

interface TreeViewProps {
  root: TNode | null;
  width?: number;
  height?: number;
  nodeColor?: (id: NodeId) => string;
  nodeStroke?: (id: NodeId) => string;
  edgeColor?: (from: NodeId, to: NodeId) => string;
  edgeDash?: (from: NodeId, to: NodeId) => boolean;
  pulseId?: NodeId | null;
  subtitle?: string;
}

```

```

const TreeView: React.FC<TreeViewProps> = ({ root, width, height, ...props }) => {
  const pos = useMemo(() => layoutTree(root), [root]);
  const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
  const edges = useMemo(() => treeEdges(root), [root]);
  const autoW = Math.max(420, ...nodes.map((n) => n.px));
  const autoH = Math.max(260, ...nodes.map((n) => n.py));
  const w = width ?? autoW;
  const h = height ?? autoH;
  const byId = (id: NodeId) => nodes.find((n) => n.id === id);
  return (
    <div>
      <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
        {edges.map((e) => {
          const a = byId(e.from);
          const b = byId(e.to);
          if (!a || !b) return null;

```



```

-----
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6) / 4);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 16) / 8);

interface PlaneProps {
  points: Pair[];
  drawnEdges: { a: Pair; b: Pair }[];
  pending: Pair | null;
  interactive?: boolean;
  selected?: Pair | null;
  cursor?: { x: number; y: number } | null;
  badEdge?: { a: Pair; b: Pair } | null;
  onPointDown?: (p: Pair) => void;
  onPointUp?: (p: Pair) => void;
  onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,
  }) => {
  const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.key) });
  return (
    <svg
      viewBox={`0 0 ${PL_W} ${PL_H}`}
      className={`w-full ${interactive ? "touch-none select-none" : ""}`}
      onPointerMove={onMove}
      style={interactive ? { cursor: "crosshair" } : undefined}
    >
      <defs>
        <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
          <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
        </pattern>
      </defs>
      <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
      <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
      <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1"/>
      {[0, 10, 20, 30, 40].map((k) => {
        <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} text-align="center">{k}</text>
      })}
      {[{-4, 0, 4, 8].map((g) => {
        <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} text-align="center">{g}</text>
      })}
    </svg>
  );
};

```

```

    });
  }}}

  {pending && (
    <g className="transition-all duration-500">
      <circle cx={PX(pending.key)} cy={PY(pending.g
        >
        <text x={PX(pending.key)} y={PY(pending.grade
          ly="monospace">{pending.key}</text>
        </g>
      )}
    </svg>
  );
};

/* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родители

export const CANON_GAME: GEdge[] = (() => {
  const es: GEdge[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ a: { key: n.key, grade: n.g
    if (n.right) { es.push({ a: { key: n.key, grade: n.g
  };
  walk(buildTree(POINTS));
  return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
  edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa

```

```

}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair): boolean {
  if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" };
  if (a.grade === b.grade)
    return { ok: false, reason: "«!» равны – «кто выше?»" };
  const parent = a.grade > b.grade ? a : b;
  const child = a.grade > b.grade ? b : a;
  if (parentOf(edges, gpId(child)))
    return { ok: false, reason: `У точки ${child.key} уже есть родительская линия.` };
  if (reaches(edges, parent, child))
    return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
  const next = [...edges, { a: parent, b: child }];
  if (!forestIsBst(next, POINTS))
    return { ok: false, reason: "Линия вниз по y допустима только в крайнем левом случае" };
  return { ok: true };
}

/* ===== РЕЖИМ BUILD ===== */

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
  const pairs: Pair[] = [];
  const errors: string[] = [];
  const seen = new Set<string>();
  const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
  for (const chunk of chunks) {
    const nums = chunk.split(/[\sxx*]+/).filter((v) => !Number.isNaN(Number(v)));
    if (nums.length !== 2 || nums.some((v) => !Number.isInteger(v))) {
      errors.push(`«${chunk}» – нужно ровно два числа: целые и не меньше 0`);
      continue;
    }
    const id = `${nums[0]}:${nums[1]}`;
    if (seen.has(id)) continue;
    seen.add(id);
    pairs.push({ key: id, grade: Math.min(...nums) });
  }
  return { pairs, errors };
}

const build = (raw: string, algo: SortAlgo, order: ConnectOrder): { pairs: Pair[]; errors: string[] } => {
  const { pairs, errors } = parsePairs(raw);
  if (pairs.length === 0) return { pairs, errors };
  pairs.sort((a, b) => {
    if (order === "y-desc") return b.grade - a.grade;
    if (order === "left-right") return a.key - b.key;
    return 0;
  });
  if (algo === "quicksort") pairs = quicksort(pairs);
  if (algo === "counting") pairs = countingSort(pairs);
  return { pairs, errors };
}

const game = (raw: string, algo: SortAlgo, order: ConnectOrder): { pairs: Pair[]; errors: string[] } => {
  const { pairs, errors } = build(raw, algo, order);
  const { ok, reason } = tryConnect(pairs, pairs[0], pairs[1]);
  return { pairs, errors, ok, reason };
}

```

```

// цикл: родитель уже сидит в компоненте ребёнка (реб
const childComp = new Set<string>([pid(child)]);
let grew = true;
while (grew) {
  grew = false;
  for (const e of edges) {
    if (childComp.has(pid(e.b)) && !childComp.has(pid
      childComp.add(pid(e.a));
      grew = true;
    }
  }
}
if (childComp.has(pid(parent))) return { ok: false, r
return {
  ok: true,
  edge: { a: parent, b: child },
  reason:
    x.grade > y.grade ? undefined : "линия в treap вс
};
}

```

```

export function checkGameState(
  points: Pair[],
  edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
  const n = points.length;
  if (n < 2) return { heapOk: true, singleParent: true,
  const heapOk = edges.every((e) => e.a.grade > e.b.gra
  const parentCount = new Map<string, number>();
  for (const e of edges) parentCount.set(pid(e.b), (par
  const singleParent = [...parentCount.values()].every(
  // связность: компоненты через union-find
  const comp = new Map<string, string>(points.map((p) =>
  const find = (x: string): string => {
    while (comp.get(x) !== x) {
      comp.set(x, comp.get(x)!);
      x = comp.get(x)!;
    }
  }
}

```

```

        right = sorted[1];
    } else if (sorted.length === 1) {
        // одиночный ребёнок: сторона важна – определяем
        if (sorted[0].key <= p.key) left = sorted[0];
        else right = sorted[0];
    }
    if (left) walk(left);
    inorder.push(p.key);
    if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
    if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

```

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode | null {
    const parentOf = new Map<string, GameEdge>();
    for (const e of edges) parentOf.set(pid(e.b), e);
    const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
    if (!roots.length && edges.length) return null;
    const nodes = new Map<string, TNode>();
    const make = (p: Pair): TNode => ({
        id: pid(p),
        key: p.key,
        grade: p.grade,
        left: null,
        right: null,
    });
    const rootPair = roots.length ? roots[0].a : edges[0];
    if (!rootPair) return null;
    const build = (p: Pair): TNode => {
        const node = make(p);
        nodes.set(pid(p), node);
        const kids = edges.filter((e) => pid(e.a) === pid(p) &&

```

```

    walk(userTree);
    return es;
  }, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
  if (!playing || phase === "game") return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return; }
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.l
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 900);
  return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, ca

const startConnect = (ord: ConnectOrder) => {
  setOrder(ord);
  setPhase("connect");
  setConnIdx(0);
  setPlaying(true);
};

// Компилятор: i двигает демонстрацию – шаг сортировки
useEffect(() => {
  if (live.i === null) return;
  if (phase === "sort") setSortIdx(Math.min(live.i, 3)
  else if (phase === "connect") setConnIdx(Math.min(l
}, [live.i, phase, leftRightEdges.length]);

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);

```

```

    }
  };
  const pointDown = (p: Pair) => {
    if (phase !== "game") return;
    if (!sel) {
      setSel(p);
      setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь...`);
    } else if (sel.key === p.key && sel.grade === p.grade) {
      setSel(null);
      setGameMsg(null);
    } else {
      attemptConnect(sel, p);
      setSel(null);
    }
  };
  const pointUp = (p: Pair) => {
    if (phase !== "game" || !sel) return;
    if (sel.key === p.key && sel.grade === p.grade) return;
    attemptConnect(sel, p);
    setSel(null);
    setCursor(null);
  };
  const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
    if (!sel) return;
    const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
    setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
  };

  const undoEdge = () => {
    setGameEdges((prev) => prev.slice(0, -1));
    setGameMsg(null);
  };

  const showSolution = () => {
    setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
    setHintUsed(true);
    setGameMsg("Решение показано – собери сам в следующий раз");
  };

```

```

<h4 className="text-base font-bold text-emera
<div className="flex items-center gap-2">
  {phase === "sort" ? (
    <>
      <span className="text-xs text-slate-400
        {[ "quicksort", "counting" ] as const }.m
      <button
        key={a}
        onClick={() => { setAlgo(a); setSor
Math.round(n * Math.log2(n)) : 0} сравнений
        disabled={n < 2}
        className={`px-3 py-1.5 rounded-lg
600 text-white" : "bg-slate-700 text-slate-
      >
        {a === "quicksort" ? "quicksort (ср
      </button>
    )}}
  </>
) : (
  <>
    <span className="text-xs text-slate-400
    <button
      onClick={() => startConnect("y-desc")
      className={`px-3 py-1.5 rounded-lg te
bg-emerald-600 text-white" : "bg-slate-700
    >
      по y ↓ (алгоритм)
    </button>
    <button
      onClick={() => startConnect("left-right
      className={`px-3 py-1.5 rounded-lg te
"bg-emerald-600 text-white" : "bg-slate-700
    >
      слева направо
    </button>
    <button
      onClick={startGame}
      disabled={n < 2}

```



```

    ) : phase === "game" ? (
      gameWon ? (
        <> Готово! {hintUsed ? "Решение подсказано": "Решение найдено"}
        ево: линии вниз по y, обход по x отсортировано
      ) : gameMsg ? (
        <>△ {gameMsg}</>
      ) : (
        <>Кликни точку-родителя (выше по «!»), за
      )
    ) : connDone ? (
      <> Проведено {connEdges.length} линий в по
      </b>: точки прибиты, дерево одно. А теперь
    ) : (
      <>Линий проведено: {connIdx} из {connEdgesA
    })
  </div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
  er-slate-700 bg-slate-900/40`}>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    {phase === "game" ? (
      <>
        <li>{verdict.heapOk ? " " : " "}</li> каждое р
        <li>{verdict.singleParent ? " " : " "}</li> у
        <li>{verdict.connected ? " " : " "}</li> все т
        <li>{verdict.countOk ? " " : " "}</li> линий р
        <li>{verdict.bstOk ? " " : " "}</li> обход сле
      </>
    ) : (
      <>
        <li>{connDone ? " " : " "}</li> каждое ребро в
        <li>{connDone ? " " : " "}</li> обход слева-на
        <li>{connDone ? " " : " "}</li> линий ровно <b>
      </>
    )
  }
}

```



```

const { l, r } = useMemo(() => splitTree(buildTree(P0
const merged = useMemo(() => mergeTree(cloneTree(l),
const steps = merged.trace;
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
  <div className="space-y-4">
    <div className="flex items-center gap-3">
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">mar {i
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 g
      <div className="bg-sky-950/20 rounded-xl p-3 bo
        <h5 className="text-sm font-bold text-sky-300
          <TreeView root={l} width={480} height={220} n
        </div>
      <div className="bg-rose-950/20 rounded-xl p-3 b
        <h5 className="text-sm font-bold text-rose-300
          <TreeView root={r} width={480} height={220} n
        </div>
    </div>

    <div className="text-xs text-slate-300 bg-slate-9
      <LiveChip live={live} />{' '}
      {cur?.chosenId ? (
        <>⊗ {cur.note}</>
      ) : (
        <>Готово: {done ? "деревья склеены обратно в
          />
      )}
    </div>
  )

```

```

<div className="space-y-4">
  <div className="flex flex-wrap items-center gap-3">
    <span className="text-sm text-slate-300">Удалить
    <LiveChip live={live} />
    <select value={key} onChange={(e) => { setKey(N
      te-700 rounded-lg px-3 py-1.5 text-sm font-mo
      {POINTS.map((p) => <option key={p.key} value=
    </select>
    <button onClick={() => setStepIdx(Math.max(0, i
      bg-slate-700 text-white disabled:opacity-30">
    <button onClick={() => setStepIdx(Math.min(step
      font-bold bg-indigo-600 text-white disabled:op
    <span className="text-xs text-slate-400">шаг {i
  </div>

  {!done ? (
    <div className="bg-slate-900/40 rounded-xl p-4"
      <TreeView
        root={fullTree}
        nodeColor={(id) => (pathSet.has(id) ? "#783
        nodeStroke={(id) => (pathSet.has(id) ? "#f5
        subtitle="Куча сама умеет удалять только вер
      />
    </div>
  ) : (
    <div className="bg-slate-900/40 rounded-xl p-4"
      <h4 className="text-sm font-bold text-slate-300">
      <TreeView root={res.root} subtitle="Валидность"
      <ul className="mt-2 space-y-1 text-sm">
        <li> обход слева-направо по-прежнему сортир
        <li> каждое ребро вниз по y (куча целая) —
      </ul>
    </div>
  )}

  <div className="text-xs text-slate-300 bg-slate-900/40 p-4">
    {steps[idx]?.note}
  </div>

```

```

{pick === "heap" ? {
  <div className="bg-slate-900/40 rounded-xl p-4" >
    <h4 className="text-sm font-bold text-slate-300" >
      4>
    <div className="flex gap-2 overflow-x-auto mb-2" >
      {heapArr.slice(1).map((v, i) => {
        <div key={i} className="px-3 py-2 rounded" >
          <div className="text-slate-500 text-[10px]" >
            <div className="text-white font-bold">{v}</div>
            <div className="text-emerald-400 text-[10px]" >
              {i}</div>
          </div>
        </div>
      })}
    </div>
    <p className="text-xs text-slate-400">Полное  $2i/2i+1$  попадает в узел.</p>
  </div>
) : {
  <div className="bg-slate-900/40 rounded-xl p-4" >
    <h4 className="text-sm font-bold text-rose-300" >
      >
    <TreeView root={full} subtitle="Попробуй «посмотреть»" />
    <div className="flex gap-2 overflow-x-auto mt-2" >
      {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => {
        const id = slots[i];
        return (
          <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-950" >
            <div className="text-slate-500 text-[10px]" >
              {id ? <div className="text-white font-[0.8em]" >
                {v}</div> : v}</div>
            </div>
          </div>
        );
      })}
    </div>
    <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два уровня указателями (left/right), а не арифметикой.</p>
  </div>
}
}

```

```

        нашли за ${steps.length + 1} сравнение(й).` }));
    if (sortedByKey[mid].key < target) { steps.push({
        } — вся левая половина выброшена.` })); lo = mid
    else { steps.push({ lo, hi, mid, found: false, no
        ` })); hi = mid - 1; }
    }
    return steps;
}, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      {stage < 2 && (
        <button onClick={() => { setStage(stage === 0
          digo-600 text-white">
            {stage === 0 ? "Найти y для x=7 бинарным по
          </button>
        )}
      {stage === 2 && (
        <button onClick={() => setBidx(Math.min(bsStep
          -1.5 rounded-lg text-xs font-bold bg-indigo
          Шаг поиска →
        </button>
      )}
      <button onClick={() => { setStage(0); setBidx(0
        Сброс</button>
    </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
  {(stage === 2 ? sortedByKey : shuffled).map((p,
    const highlight = stage === 1 && i === wrongMid
    const inBs = stage === 2 && (() => { const s
    const isMid = stage === 2 && (() => { const s
    const isFound = stage === 2 && bidx >= bsStep
    return (

```



```

        <text x={PX(p.key)} y={PY(p.grade) + 24}
      </text>
    </g>
  );
  }}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
  ate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    <li>{v.heap ? " " : " "} каждое ребро – вниз
    <li>{v.oneParent ? " " : " "} у каждой точки
    <li>{v.acyclic ? " " : " "} циклов нет</li>
    <li>{v.bst ? " " : " "} обход слева-направо –
    <li>{v.complete ? " " : " "} линий  $n-1 = \{POI$ 
  </ul>
  {win && {
    <p className="mt-3 text-sm font-bold text-еме
      ения просто не существует.</p>
  }}
</div>

  <div className="text-xs text-slate-300 bg-slate-9
</div>
  );
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "lay

const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split ≈ " },
  { id: "merge", label: "Merge  " },

```



```

        </div>
      </div>
    </div>

    <div className="flex flex-wrap gap-2 mb-5">
      {TABS.map((t) => (
        <button
          key={t.id}
          onClick={() => setTab(t.id)}
          className={`px-4 py-2 rounded-lg text-xs font-medium border
            900/60 text-slate-400 hover:text-white border
          >
            {t.label}
          </button>
        )})}
    </div>

    {tab === "build" && <BuildMode />}
    {tab === "split" && <SplitMode />}
    {tab === "merge" && <MergeMode />}
    {tab === "erase" && <EraseMode />}
    {tab === "layers" && <LayersMode />}
    {tab === "search" && <SearchMode />}
    {tab === "game" && <GameMode />}
  </div>
);
};

```

ФАЙЛ 75/92: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
```

```
import { ArticulationPointsViz } from "../ArticulationPointsViz";
```

```
import BellmanFordViz from "../BellmanFordViz";
```

- * Единый реестр интерактивных демонстраций.
 - * Используется и для панели под главой, и для всплывающей подсказки
 - * и для модального окна «Открыть симулятор».
- */

```
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, что нужно для обхода в глубину",
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую",
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .",
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разрезанная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится",
    Component: SparseTableViz,
  },
  treap: {
    title: "Treap: плоскость → дерево → Split/Merge/Erase",
    hint: "7 вкладок: Собрать (сортировка + соединение)
```

```

    hint: "Слева – непланарные графы с подсветкой пересечений",
    Component: () => (
      <div className="space-y-10">
        <PlanarityDemo />
        <ColoringViz />
      </div>
    ),
  },
  dijkstra: {
    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксируем",
    Component: () => (
      <>
        <ChapterImage vizType="dijkstra" />
        <DijkstraViz />
      </>
    ),
  },
  "bellman-ford": {
    title: "Форд–Беллман",
    Component: () => (
      <>
        <ChapterImage vizType="bellman-ford" />
        <BellmanFordViz />
      </>
    ),
  },
  floyd: {
    title: "Флойд–Уоршелл",
    Component: () => (
      <>
        <ChapterImage vizType="floyd" />
        <FloydViz />
      </>
    ),
  },
  "johnson-algo": { title: "Алгоритм Джонсона", Component: () => {} },
  "mst-kruskal": { title: "Краскал и DSU", hint: "Сортировка по весу",

```

```

-----
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
    id: number;
    label: string; // путь от корня
    char: string;
    x: number; // Координаты для SVG водопада
    y: number;
    depth: number;
    fail: number;
    term_link: number;
    is_terminal: boolean;
    words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
    0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0, fail: 0, term_link: 0, is_terminal: false, words: [] },
    // Ветка H -> E -> R -> S
    1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
    2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
    3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
    4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4, fail: 0, term_link: 0, is_terminal: true, words: ['HERS'] },
    // Ветка H -> I -> S
    5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
    6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['HIS'] },
    // Ветка S -> H -> E
    7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
    8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
    9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['SHE'] },
    // Ветка E -> H -> S
    10: { id: 10, label: 'E', char: 'E', x: 600, y: 160, depth: 1, fail: 2 (HE), term_link: 0, is_terminal: false, words: [] },
    11: { id: 11, label: 'EH', char: 'H', x: 630, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
    12: { id: 12, label: 'EHS', char: 'S', x: 660, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: true, words: ['EHS'] },
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
    0: { H: 1, S: 7 },
    1: { E: 2, I: 5 },
    2: { R: 3 },
    3: { S: 4 },
    4: {},
    5: { S: 6 },
    6: {},
    7: {},
    8: {},
    9: {},
    10: { H: 11 },
    11: { S: 12 },
    12: {},
    13: { H: 14 },
    14: { S: 15 },
    15: {},
    16: { H: 17 },
    17: { S: 18 },
    18: {},
    19: { H: 20 },
    20: { S: 21 },
    21: {},
    22: { H: 23 },
    23: { S: 24 },
    24: {},
    25: { H: 26 },
    26: { S: 27 },
    27: {},
    28: { H: 29 },
    29: { S: 30 },
    30: {},
    31: { H: 32 },
    32: { S: 33 },
    33: {},
    34: { H: 35 },
    35: { S: 36 },
    36: {},
    37: { H: 38 },
    38: { S: 39 },
    39: {},
    40: { H: 41 },
    41: { S: 42 },
    42: {},
    43: { H: 44 },
    44: { S: 45 },
    45: {},
    46: { H: 47 },
    47: { S: 48 },
    48: {},
    49: { H: 50 },
    50: { S: 51 },
    51: {},
    52: { H: 53 },
    53: { S: 54 },
    54: {},
    55: { H: 56 },
    56: { S: 57 },
    57: {},
    58: { H: 59 },
    59: { S: 60 },
    60: {},
    61: { H: 62 },
    62: { S: 63 },
    63: {},
    64: { H: 65 },
    65: { S: 66 },
    66: {},
    67: { H: 68 },
    68: { S: 69 },
    69: {},
    70: { H: 71 },
    71: { S: 72 },
    72: {},
    73: { H: 74 },
    74: { S: 75 },
    75: {},
    76: { H: 77 },
    77: { S: 78 },
    78: {},
    79: { H: 80 },
    80: { S: 81 },
    81: {},
    82: { H: 83 },
    83: { S: 84 },
    84: {},
    85: { H: 86 },
    86: { S: 87 },
    87: {},
    88: { H: 89 },
    89: { S: 90 },
    90: {},
    91: { H: 92 },
    92: { S: 93 },
    93: {},
    94: { H: 95 },
    95: { S: 96 },
    96: {},
    97: { H: 98 },
    98: { S: 99 },
    99: {}
};

```

```

    }, [liveText]);
    useEffect(() => {
      if (liveI !== null) setCurrentIndex(Math.max(0, Math.min(liveI, textToScan.length)));
    }, [liveI, textToScan.length]);
    useEffect(() => {
      if (liveNode !== null && WATERFALL_NODES[liveNode]) {
        setLiveNode(liveNode);
      }
    }, [liveNode]);

    const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
      const clean = e.target.value.toUpperCase().replace(/[^a-zA-Z0-9_ ]/g, '');
      setTextToScan(clean);
      resetSearchSimulation();
    };

    const resetSearchSimulation = () => {
      setCurrentIndex(0);
      setCurrentNode(0);
      setIsSearchDone(false);
      setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
      setJumpPath(null);
      setFoundMatches([]);
      setActiveSearchCodeLine(1);
    };

    const stepSearchSimulation = () => {
      if (currentIndex >= textToScan.length) {
        setIsSearchDone(true);
        setSearchLog('Мы проплыли весь текст! Лосось доволен');
        setActiveSearchCodeLine(4);
        return;
      }

      const char = textToScan[currentIndex];
      let curr = currentNode;
      let logMsg = `[Символ '${char}']`;

      let jumpType: 'normal' | 'fail' | null = null;
      void jumpType;

```

```

        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
    logMsg += `    БИНГО! Найдено слово: ${matchedWords}
    setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
    setFoundMatches((prev) => [...prev, ...currentMat
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
            рние вершины сразу инициализируются с fail = RO
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'ø',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 1,
        title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
        desc: 'Мы обрабатываем первую вершину из очереди -
            просто не существует. fail[H] автоматически нап
        codeLine: 5,
        scoutPos: 0,
    },

```

```

    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТ
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плавает в ROOT и ищет ветку "H". У корня
        суффикс "H"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'R',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 6,
    title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
    desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
        но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'S',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 9,
    title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК СТ
    desc: '    МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
        ба плавает в #1 (H) и ищет ветку "E". У вершины

```

```

    {/* Переключатель режимов */}
    <div className="bg-slate-900 p-1.5 rounded-xl b
      <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p
          activeMode === 'training'
            ? 'bg-indigo-600 text-white shadow-lg sl
            : 'text-slate-400 hover:text-white'
          `}
      >
      <GitBranch className="w-4 h-4" />
      <span>Режим 1: Полное обучение (BFS от корн
    </button>
    <button
      onClick={() => setActiveMode('search')}
      className={`flex items-center space-x-1.5 p
        activeMode === 'search'
          ? 'bg-blue-600 text-white shadow-lg sha
          : 'text-slate-400 hover:text-white'
        `}
    >
      <Play className="w-4 h-4" />
      <span>Режим 2: Поиск в тексте</span>
    </button>
  </div>
</div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
  /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
  <div className="grid grid-cols-1 lg:grid-cols-3
    {/* Управление шагами обучения */}
    <div className="bg-slate-900/90 p-5 rounded-x
      <div>
        <h5 className="text-indigo-300 font-bold
          <span> </span> Полный обход в ширину (В
        </h5>

```



```

        {trainStep === TRAINING_STEPS_INFO.length
        </button>
    </div>
</div>

/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
    >
    <div>
        <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
                <span></span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950
                Вершина: #{targetTrainingNode.id} ({t
            </span>
        </h5>
        <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
                -l-4 border-indigo-500 text-indigo-300 font
                    <span>1. let u = trie[r][char]; // Те
                        {currentTrainInfo.codeLine === 1 && <
                    тут</span>}
                </div>
                <div className={`p-1.5 rounded flex ite
                    -l-4 border-indigo-400 text-indigo-200 font
                        <span>2. let v = fail[r]; // Подъем к
                    ainInfo.inspectedParentFail).label}}</span>
                        {currentTrainInfo.codeLine === 2 && <
                    к родителю</span>}
                </div>
                <div className={`p-1.5 rounded flex ite
                    l-4 border-amber-500 text-amber-300 font-bo
                        <span>3. while (v !== root && trie[v]
                            {currentTrainInfo.codeLine === 3 && <
                    т, возврат в корень</span>}
                </div>
                <div className={`p-1.5 rounded flex ite

```

```

        type="text"
        value={textToScan}
        onChange={handleTextChange}
        maxLength={15}
        placeholder="ВВЕДИТЕ ТЕКСТ..."
        className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
      />
    </div>

    <div>
      <div className="mb-4 flex flex-wrap gap-1">
        <span className="text-xs text-slate-400">
          {textToScan.split('').map((char, idx) =>
            <span
              key={idx}
              className={`w-7 h-8 flex items-center justify-center
                ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
                idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
                'bg-slate-800 text-slate-500'}`}
            >
              {char}
            </span>
          )}}
        </div>
        {currentIndex >= textToScan.length && (
          <span className="bg-emerald-600 text-white">
            ГOTOBO ✓
          </span>
        )}
      </div>

      <button
        onClick={stepSearchSimulation}
        disabled={isSearchDone || textToScan.length === 0}
        className={`w-full py-2.5 rounded-lg font-medium text-white

```

```

        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span <
овпадение!</span>}
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
  </h5>
  <div className="bg-slate-800 p-3.5 rounde
tems-center">
    {searchLog}
  </div>
</div>
</div>
</div>
))

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
  {/* Водопадные фоновые эффекты */}
  <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat
    <div className="w-12 bg-blue-300 h-full anima
    <div className="w-6 bg-blue-500 h-full animat
  </div>

  <div className="flex items-center justify-betwe
    <h5 className="text-white font-bold text-base
      <span> </span> Ветвящийся синий водопад (Бор
    </h5>
    <div className="flex flex-wrap gap-4 text-xs">
      <span className="flex items-center gap-1 te

```

```

    /* Рендер прямых переходов (потоков воды)
    {Object.entries(TRIE_TRANSITIONS).map(([fromNode, toNode]) => {
      const fromNode = WATERFALL_NODES[Number(fromNode)];
      return Object.entries(children).map(([childId, child]) => {
        const toNode = WATERFALL_NODES[toId];

        // Подсветка в режиме поиска
        let isHighlighted = activeMode === 'search' ||
        type === 'normal';

        // Подсветка в режиме обучения (когда ищем)
        let isTrainingExamined = activeMode === 'train' ||
        let isMatchBranch = isTrainingExamined && childId === toId;

        return (
          <g key={`edge-${fromNode.id}-${toNode.id}`}>
            /* Линия течения воды */
            <line
              x1={fromNode.x}
              y1={fromNode.y}
              x2={toNode.x}
              y2={toNode.y}
              stroke={isHighlighted ? '#3b82f6' : '#ccc'}
              strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
              className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
            />
            /* Символ перехода */
            <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
              r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : '#ccc'}
            />
            <text
              x={{(fromNode.x + toNode.x) / 2}}
              y={{(fromNode.y + toNode.y) / 2} + 15}
              fill={isTrainingExamined ? (isMatchBranch ? 'white' : 'black') : 'black'}
              fontSize="12"
              fontWeight="bold"
              textAnchor="middle"
            />
          </g>
        );
      });
    });
  );
}

```

```

const cy = node.y + dy / 2 - 30;

return (
  <g key={`fail-${node.id}`}>
    <path
      d={`M ${node.x} ${node.y} Q ${cx} $
      fill="none"
      stroke={isHighlighted ? '#f43f5e' :
      strokeWidth={isHighlighted || isJust
      strokeDasharray="6,6"
      className={isHighlighted || isJustE
      opacity={isHighlighted || isJustEst
    />
    {isJustEstablished && (
      <text x={cx} y={cy - 10} fill="#10b
    >
      fail[{node.label}] = {targetNode.
    </text>
    )}
  </g>
);
}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
  const hasWords = node.words.length > 0;

```

```

return (
  <g key={`node-${node.id}`} className={isCurrent ? 'active' : ''}>
    {/* Внешнее свечение для активной вершины */}
    {isCurrent && (
      <circle cx={node.x} cy={node.y} r={30} fill="none" stroke="#f43f5e" stroke-width={2} />
    )}
    {isTargetChild && (
      <circle cx={node.x} cy={node.y} r={30} fill="none" stroke="#10b9d1" stroke-width={2} />
    )}

```

```

        <text x="0" y="3" fontSize="10" f
            *
        </text>
    </g>
    })
</g>
);
}}
}}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
<g>
    {/* Круг-подложка под рыбу */}
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezi
        className="animate-float select-none"
    >
        {activeMode === 'training' ? ' ♂ ' :
    </text>
</g>

</svg>
</div>
</div>

```

ФАЙЛ 77/92: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 411 | РАЗМ: 10.5 КБ

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import {
  emitVizDemo,
  useVizRuntime,
  vizArray,
  vizNumber,
  VizChapterContext,
} from "../../data/vizStepBus";
```

```
/**
```

```
 * Префиксные суммы: 1D и 2D.
```

```
 *
```

```
 * Главная фишка – наведение на число:
```

```
 * * в 1D наводим на P[i] – подсвечивается кусок массива
```

```
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
```

```
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
```

```
 * а при выбранном запросе показывается формула включения-исключения
```

```
 */
```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];
```

```
const cellBase =
```

```
  "flex items-center justify-center rounded-md font-mono text-sm";
```

```
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";
```

```

    []
  );

  /** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "a" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
  /** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
  // i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
  ? liveI !== null
    ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
    : Math.max(0, Math.min(A1.length, (liveP?.length || 0)))
  : null;

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        {/* индексы */}
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={` ${cellBase} !h-5`}>
              {i}
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
);

```



```

const active = hover?.kind === "pref" &&
const isL = i === range.l;
const isR = i === range.r + 1;
const liveValue = liveP?.[i];
const blank = hasLiveP && (liveValue ===
const masked = !hasLiveP && shownDriven != null;
const isDrivenCur = shownDriven !== null;
const shownValue = hasLiveP ? liveValue :
return (
  <div
    key={i}
    onMouseEnter={() => setHover({ kind:
    onMouseLeave={() => setHover(null)}
    className={`${cellBase} cursor-help $
      isDrivenCur
        ? "bg-emerald-600 text-white ring
        : blank
        ? "bg-slate-950 text-slate-600
        : masked
        ? "bg-slate-800/50 text-slate-7
        : active
        ? "bg-emerald-500 text-white
        : isR
        ? "bg-emerald-700 text-white
        : isL
        ? "bg-rose-700 text-white
        : "bg-slate-900 text-slate-
    }`}
    title={blank || masked ? `P[${i}] - n
  >
    {blank || masked ? "." : String(shown
  </div>
);
}}}
</div>
</div>
</div>

```

```

        Наведите курсор на любое число: y <b className=
        y <b className="text-indigo-400">a</b> – ку
    </p>
  })
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
  <div className="text-xs text-slate-400 mb-2">
    Запрос суммы на отрезке (кликайте по массиву
  </div>
  <p className="font-mono text-sm sm:text-base te
    sum(a[{range.l}..{range.r}]) = P[{range.r + 1
    <span className="text-emerald-400">{pref[range
    <span className="text-rose-400">{pref[range.l
    <span className="text-indigo-300 font-bold">{
  </p>
  <p className="text-[11px] text-slate-500 mt-1">
    Один вычитание вместо цикла: запрос за O(1) п
  </p>
</div>
</div>
});
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
  const n = A2.length;
  const m = A2[0].length;
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveI = vizNumber(vars?.i);
  const liveJ = vizNumber(vars?.j);
  const liveS = vizArray(vars?.S);
  const hasLiveS = !!vars && Object.prototype.hasOwnProperty.call(
    vars, 'S');

  const P = useMemo(() => {
    const p = Array.from({ length: n + 1 }, () => Array

```

```

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      /* Исходная матрица */
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">
          <div className="inline-block">
            {A2.map((row, i) => {
              <div key={i} className="flex gap-1.5 mb-1">
                {row.map((v, j) => {
                  const inRect = i >= rect.r1 && i <= r1
                  return (
                    <div
                      key={j}
                      onClick={() =>
                        setRect((r) =>
                          i < r.r1 || j < r.c1 ? { r1: i, c1: j } : r
                        )
                    }
                    className={` ${cellBase} cursor-pointer ${
                      inRect ? "bg-indigo-500 text-white" : ""
                    }`}
                  />
                )
              )
            })}
          </div>
        </div>
      </div>
    </div>
  )

  /* Матрица префиксных сумм */
  <div className="overflow-x-auto">
    <div className="text-xs text-slate-400 mb-2">
      <div className="inline-block">
        {P.map((row, i) => {

```

```

        </span>
      })
    </div>
  );
  }}}
</div>
  )))
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
  {hover ? {
    <p className="text-slate-300">
      <b className="text-amber-400">
         $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hover.j] +$ 
      </b>{" "}
      – сумма всего прямоугольника от левого верхнего угла до
    </p>
  } : {
    <p className="text-slate-500">Наведите курсор на элемент
  }}
</div>

<div className="bg-slate-900 border border-indigo-500"
  <p className="font-mono text-sm sm:text-base text-slate-500">
    сумма = <span className="text-emerald-400">A</span> + <span
    <span className="text-rose-400">C</span> + <span
    <span className="text-indigo-300 font-bold">{
  </p>
  <p className="text-[11px] text-slate-500 mt-1">
    Вычли два «хвоста» и вернули дважды вычитенный
  </p>
</div>
</div>
);
};

```

```
// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);

for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}

for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
      for (let c = 0; c + (1 << ky) <= 8; c++) {
        if (kx > 0) {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx-1][ky],
            ST2D[r + (1 << (kx-1))][c][kx-1][ky]
          );
        } else {
          ST2D[r][c][kx][ky] = Math.min(
```

```

        onClick={() => setTab("2d_query")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-
          se-600 text-white" : "bg-slate-800 text-sla
      >
      3. Запрос 2D (Формула)
    </button>
  </div>

  <AnimatePresence mode="wait">
    {tab === "1d" ? (
      <Viz1D key="1d" />
    ) : tab === "2d_build" ? (
      <Viz2DBuild key="2d_build" />
    ) : (
      <Viz2DQuery key="2d_query" />
    )}
  </AnimatePresence>
</div>
);
};

```

```

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
  const runtime = useVizRuntime();
  const runtimeVars = runtime?.variables;
  const liveI = vizNumber(runtimeVars?.i);
  const liveJ = vizNumber(runtimeVars?.j);
  const liveSource = vizArray(runtimeVars?.a);
  const liveRows = vizArray(runtimeVars?.st);
  const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
  const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
  const compilerLinked = liveI !== null || liveJ !== null;
  const shownSource = hasLiveSource
    ? liveSource?.length
    ? liveSource
    : Array.from({ length: N }, () => null)
    : arr1D;

```

```

        <button
          onClick={() => setStep(Math.min(maxStep, st
          className="p-2 bg-indigo-600 hover:bg-indig
          disabled={step === maxStep}
        >
          <ChevronRight size={20} />
        </button>
        <button
          onClick={() => setStep(0)}
          className="p-2 bg-slate-800 hover:bg-slate-
        >
          <RotateCcw size={20} />
        </button>
      </div>
    </div>

    <div className="bg-slate-900 rounded-xl border bo
      <div className="text-[11px] uppercase tracking-t
      <div className="flex gap-1 sm:gap-1.5 overflow-
        {shownSource.map((v, idx) => {
          const inCover = !!covered && idx >= covered
          const display = v === undefined || v === nu
            ? "."
            : typeof v === "object"
              ? JSON.stringify(v)
              : String(v);
          return (
            <div key={idx} className="flex flex-col i
              <div
                className={`flex items-center justify
                w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
                inCover ? "bg-indigo-500 text-white
              }`}
            >
              {display}
            </div>
            <span className="text-[9px] text-slate-
          </div>
        )}
      </div>

```

```

        rap">
          2^1 (L=2)
        </div>
        <div className="text-center font-bold text-
        rap">
          2^2 (L=4)
        </div>
        <div className="text-center font-bold text-
        rap">
          2^3 (L=8)
        </div>

    /* Matrix */
    {Array.from({ length: N }).map((_, i) => {
      <React.Fragment key={i}>
        <div className="text-slate-500 font-mon
          {i}
        </div>
        {Array.from({ length: LOG }).map((_, j) => {
          const isValid = i + (1 << j) <= N;
          const liveRow = vizArray(liveRows?.[i]
          const liveValue = liveRow?.[j];
          // Уровень j=0 – это исходный a[i], о
          // очищаемого рабочего массива и не д
          const sourceValue = j === 0 ? liveSou
          const runtimeValue = liveValue ?? sou
          const runtimeHasValue = runtimeValue
          const manualVisible =
            isValid &&
            (j === 0 || computeSteps.findIndex(
          const isVisible = compilerLinked ? run

      let isActive = !!liveTarget && liveTa
      let isSrc1 = false;
      let isSrc2 = false;

      if (compilerLinked && liveTarget && l
        isSrc1 = i === liveTarget.i && j ==

```



```

        hover && hover.i === i && h
        ? "bg-sky-500 text-white
        : isActive
        ? "bg-indigo-500 text-whi
        : isSrc1
        ? "bg-emerald-500 text-v
        : isSrc2
        ? "bg-amber-500 text-r
        : isVisible
        ? "bg-slate-800 tex
        : compilerLinked
        ? "bg-slate-950 t
        : "bg-transparent
    }`}
    title={compilerLinked ? `st[$
  >
    {displayValue}
  </motion.div>
)}

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
  <motion.svg
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
    className="absolute pointer-e
    style={{
      width: 100,
      height: isSrc2 ? 40 + (1 <<
      right: -50,
      top: 24,
      overflow: "visible",
    }}
  >
    <path
      d={`M 0 0 C 30 0, 30 ${isSrc
      fill="none"
      stroke={isSrc1 ? "#10b981"

```

```

        ST[{c.i + half}][{c.j - 1}]
      </span>
    ) &rarr; min(
      <span className="text-emerald-400">{st1D[c.i + half][c.j - 1]}
      <span className="text-amber-400">
        {st1D[c.i + half][c.j - 1]}
      </span>
    ) ={" "}
    <span className="text-indigo-400 font-b"
      {st1D[c.i][c.j]}
    </span>
  </p>
  );
  })()
) : (
  <p className="text-slate-500 italic">
    Нажмите далее, чтобы увидеть, как блоки сво
  </p>
  )}
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
  const [k, setK] = useState(1);
  const [hover, setHover] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
  const [locked, setLocked] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveK = vizNumber(vars?.k);
  const liveR = vizNumber(vars?.r);
  const liveC = vizNumber(vars?.c);
  const liveTable = vizRecord(vars?.st2);
  const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
  const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
  const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
  const codeCell = liveR !== null && liveC !== null ? {

```



```

        if(isCurr) {
            if(locked && locked.
            else setLocked({r, c
        }
    }}
    className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
title={hasLiveTable ? `st2[
>
    {isBlank ? "." : String(sho
</div>
    );
    }}}
</div>
</div>
</div>
</React.Fragment>
)
    }}}
</div>
</div>

```

```

<div className="flex-1 min-w-0 flex flex-col gap-
  <div className="bg-slate-900 p-4 sm:p-6 rounded
    <h3 className="text-lg font-bold text-slate-3
      {shownK === 0 ? {
        <div className="text-slate-400 text-sm lea
          {liveTableEmpty ? {
            <div className="rounded-xl border bord
              <p className="font-bold text-amber-3
                <p className="mt-2 text-xs text-slat
                  Матрица <span className="font-mono
                    аписи базового слоя:
                  </p>
                  <div className="mt-3 rounded-lg bg-[
                    <span className="text-indigo-400">
                  </div>

```

```

    {activeCell ? {
      <div className="bg-slate-950 p-5 rounded
.2)] mt-2 transition-all">
        <p className="text-slate-300 mb-3 t
          <span>Пример для ячейки:</span>
          {locked && <span className="text
ded border border-rose-500/20"><Lock size={
        </p>
        <p className="text-indigo-300 borde
          <span className="font-bold text-w
        }][{shownK}][{shownK}]</span>
          <span className="text-slate-400">
        </p>

        <div className="space-y-2 pt-3 pl-2
          {(() => {
            const prevL = 1 << (shownK -
            return (
              <>
                <div className="flex ite
                  <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
                  <span className="text
tiveCell.r][activeCell.c][shownK-1][shownK-
                </div>
                <div className="flex ite
                  <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
                  <span className="text
tiveCell.r][activeCell.c + prevL][shownK-1]
                </div>
                <div className="flex ite
                  <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                  <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
                </div>

```

```
// Координаты запроса – те же захардкоженные имена,
useEffect(() => {
  const vars = runtime?.variables;
  const nr1 = vizNumber(vars?.r1);
  const nc1 = vizNumber(vars?.c1);
  const nr2 = vizNumber(vars?.r2);
  const nc2 = vizNumber(vars?.c2);
  if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
  if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
  if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
  if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
}, [runtime]));

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
  <div className="flex flex-col xl:flex-row gap-6"
    <div className="flex-1 min-w-0 bg-slate-900"
      <h3 className="text-xl font-bold text-ro
      <p className="text-slate-400 text-[13px]
        еpx, r2, c2 - правый низ).</p>

      <div className="flex flex-col items-cent
        <div className="grid grid-cols-[repea
          ative shadow-inner w-max max-w-full">
            {Array.from({length: 64}).map((_,
              const r = Math.floor(idx / 8);
              const c = idx % 8;
              const isInQuery = r >= r1 && r
```

```

        <div className="bg-slate-950 p-4 rounded-
er-slate-800 shadow-[inset_0_2px_10px_rgba(
        <label className="flex items-center
        <span className="text-slate-
        <div className="flex gap-2">
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
        </div>
        </label>
        <label className="flex items-center
        <span className="text-slate-
        <div className="flex gap-2">
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
            <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
        </div>
        </label>
    </div>
</div>
</div>

<div className="flex-1 bg-slate-900 p-6 rounded-
    <h3 className="text-xl font-bold text-em
    <p className="text-sm font-sans text-sla
        Мы знаем, что максимальная степень дв
rounded text-white">{Lx}</span>. Аналогично
te">{Ly}</span>.
    </p>
    <p className="text-sm font-sans text-sla
        Чтобы покрыть весь прямоугольник, мы
Name="font-mono bg-[#0a0f1e] px-1 rounded b
роса. (Цветные прямоугольники слева).
    </p>

    <div className="bg-[#0a0f1e] rounded-xl

```

```

<h4 className="text-slate-400 font-bo
    Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{kx}][{ky}]
</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}>
    {Array.from({length: matRows * matCols}, (v, idx) => {
        const r = Math.floor(idx / matCols);
        const c = idx % matCols;
        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;

        let color = "bg-slate-800 text-white";
        let txt = ST2D[r][c][kx][ky].toString();

        let badge = null;
        if (isTL) { color = "bg-rose-500 text-white";
0 border-2"; badge = "TL"; }
        else if (isTR) { color = "bg-lime-500 text-white";
ue-300 border-2"; badge = "TR"; }
        else if (isBL) { color = "bg-emerald-500 text-white";
-emerald-300 border-2"; badge = "BL"; }
        else if (isBR) { color = "bg-amber-500 text-white";
mber-300 border-2"; badge = "BR"; }

        return (
            <div key={idx} className={color}
tion-all duration-300 relative shrink-0 ${c === c1 ? 'w-1/2' : 'w-1/4'} h-1/4
            {txt}
            {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded
            </div>
        )
    })
    </div>
</div>

```



```
<script>
  function setMode(mode) {
    document.getElementById('btn-mode-normal').className
      ? 'px-3 py-1 rounded text-sm font-bold'
      : 'px-3 py-1 rounded text-sm font-bold';

    document.getElementById('btn-mode-wtf').className
      ? 'px-3 py-1 rounded text-sm font-bold'
      : 'px-3 py-1 rounded text-sm font-bold';

    const aside = document.querySelector('aside');
    const headerMain = document.querySelector('header');
    const questionsContainer = document.getElementById('questions-container');
    const wtfContainer = document.getElementById('wtf-container');

    if (mode === 'wtf') {
      document.body.classList.add('wtf-mode');
      if (aside) aside.style.display = 'none';
      if (headerMain) headerMain.style.display = 'none';
      if (questionsContainer) questionsContainer.style.display = 'none';
      if (wtfContainer) wtfContainer.classList.remove('hidden');

      // Remove the URL hash to avoid scrolling
      history.pushState("", document.title, window.location.pathname);
      window.scrollTo(0, 0);
    } else {
      document.body.classList.remove('wtf-mode');
      if (aside) aside.style.display = '';
      if (headerMain) headerMain.style.display = '';
      if (questionsContainer) questionsContainer.style.display = '';
      if (wtfContainer) wtfContainer.classList.add('hidden');
    }
  }

  function setTheme(theme) {
    const body = document.body;
    body.classList.remove('theme-dark', 'theme-light');
  }

```

```

        drawerOverlay.classList.remove(
        // Timeout allows display:block
        setTimeout(() => drawerOverlay.
    }
    document.body.style.overflow = "hid
} else {
    mobileDrawer.classList.add("-transl
    if (drawerOverlay) {
        drawerOverlay.classList.add("op
        setTimeout(() => drawerOverlay.
    }
    document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
    mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
    closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
    drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
    link.addEventListener("click", () => {
        if (mobileDrawer && !mobileDrawer.c
            toggleDrawer();
    }
    });
});
});

```

```

const observerOptions = {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: 0
}

```

```

<script src="https://cdn.tailwindcss.com"></script>
<script src="https://polyfill.io/v3/polyfill.min.js"></script>
<script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.2/es5/tex-mml-chtml.js"></script>
<script>
  window.MathJax = {
    tex: {
      inlineMath: [['$', '$'], ['\\(', '\\)']],
      displayMath: [['$$', '$$'], ['\\[', '\\]']],
    },
  };
</script>
<style>
  @import url('https://fonts.googleapis.com/css2?family=Fira+Code&family=Arial+Black&display=block');
  @reference './src/index.css';
  .math-font { font-family: 'Fira Code', monospace; }
  .uno-card {
    width: 70px; height: 105px; border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
    align-items: center; justify-content: center; text-align: center;
    color: white; overflow: hidden; flex-shrink: 0;
    text-shadow: 2px 2px 0 #000, -1px -1px 0 #000;
    font-family: 'Arial Black', Impact, sans-serif;
  }
  .uno-card::before, .uno-card::after {
    content: attr(data-val); position: absolute;
    text-shadow: 1px 1px 0 #000, -1px -1px 0 #000;
  }
  .uno-card::before { top: 2px; left: 4px; }
  .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
  .uno-card.mini { width: 45px !important; height: 60px !important; }
  .uno-card.mini::before, .uno-card.mini::after {
    font-size: 0.8em;
  }

  /* Global formula scroll settings */
  .formula-scroll {
    overflow-x: auto;
    overflow-y: hidden;
    -webkit-overflow-scrolling: touch;
    scrollbar-width: none; /* Firefox */
  }

```

```

        px; }
    }
    .uno-inner {
        position: absolute; width: 110%; height: 75%;
        border-radius: 50%; transform: rotate(-30deg);
        box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }
    .uno-val { position: relative; z-index: 2; }
    .card-red { background-color: #e52521; } .card-
    .card-blue { background-color: #004dcf; } .card-
    .card-green { background-color: #339e35; } .card-
    .card-yellow { background-color: #ffc400; } .ca
    .card-black { background-color: #222; } .card-b
    .card-black::before, .card-black::after { text-
    .info-block { @apply bg-slate-800/80 border-l-4
    .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
    ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
    king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
    r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-ligh
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-ligh
    important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-

```

```

        animation: creepy-blink 5.2s infinite;
        animation-delay: 1.5s;
    }
    .creepy-emoji {
        display: inline-block;
        transform-origin: center;
        animation: creepy-blink 3.8s infinite;
    }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

    <div class="sticky top-0 z-50 w-full bg-slate-950/90
        center items-center gap-4 sm:gap-8 transition-colors">
        <div class="flex items-center gap-3">
            <span class="text-sm font-bold uppercase tracking-wider">
                <div class="bg-slate-800 p-1 rounded-lg flex items-center
                    transition-colors">Список тем</div>
                <button onclick="setMode('wtf')" id="btn-wtf"
                    class="text-slate-400 font-medium rounded-lg px-2 py-1
                    transition-colors">WTF</button>
            </div>
            <div class="flex items-center gap-3">
                <span class="text-sm font-bold uppercase tracking-wider">
                    <div class="bg-slate-800 p-1 rounded-lg flex items-center
                        transition-colors">Темная</div>
                    <button onclick="setTheme('light')" id="btn-light"
                        class="text-slate-400 font-medium rounded-lg px-2 py-1
                        transition-colors">Light</button>
                <div class="bg-slate-800 p-1 rounded-lg flex items-center
                    transition-colors">Заметки</div>
                <button onclick="setTheme('notebook')" id="btn-notebook"
                    class="text-slate-400 font-medium rounded-lg px-2 py-1
                    transition-colors">Notebook</button>
            </div>
        </div>
    </div>

    <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
    <svg width="0" height="0" class="absolute">

```

```

        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <!-- МЯСОРУБКА -->
    <g id="meat-grinder">
        <path d="М 10 40 L 15 20 Q 25 15 35 20
        <rect x="5" y="40" width="40" height="20"
        <!-- Ручка -->
        <path d="М 45 50 Q 60 50 60 30 L 75 30"
        <circle cx="75" cy="30" r="4" fill="#333"
        <!-- Выход -->
        <path d="М 5 45 L -5 45 L -5 55 L 5 55"
        <circle cx="25" cy="50" r="15" fill="#f00"
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="10"
        <rect x="3" y="5" width="14" height="5"
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
    0/50 z-50 hover:bg-blue-500 transition-transform
    <svg xmlns="http://www.w3.org/2000/svg" class="
        <path stroke-linecap="round" stroke-linejoin
    </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню

```

```
<details class="group">
  <summary class="list-none cursor-pointer-indigo-500 pl-3 font-bold text-indigo-300">
    2. Комплексные числа
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-base">
    <a href="#section-2-1" class="block text-indigo-500">
    <a href="#section-2-2" class="block text-indigo-500">
    <a href="#section-2-3" class="block text-indigo-500">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer-teal-500 pl-3 font-bold text-teal-300">
    3. Тригонометрическая форма
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-base">
    <a href="#section-3-1" class="block text-teal-500">
    <a href="#section-3-2" class="block text-teal-500">
    <a href="#section-3-3" class="block text-teal-500">
    <a href="#section-3-4" class="block text-teal-500">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer-amber-500 pl-3 font-bold text-amber-300">
    4. Степени и корни
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-base">
    <a href="#q4-def" class="block text-amber-500">
```

```
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
    7. Взаимно простые числа
  </a>
</summary>
<div class="pl-6 space-y-2 text-pink-300">
  <a href="#q7-1" class="block">
  <a href="#q7-2" class="block">
  <a href="#q7-3" class="block">
  <a href="#q7-4" class="block">
  <a href="#q7-5" class="block">
</div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
    8. Сравнимость и Поля Вейля
  </a>
</summary>
<div class="pl-6 space-y-2 text-violet-300">
  <a href="#q8-1" class="block">
  <a href="#q8-2" class="block">
  <a href="#q8-3" class="block">
  <a href="#q8-4" class="block">
  <a href="#q8-5" class="block">
  <a href="#q8-6" class="block">
  <a href="#q8-7" class="block">
</div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
```



```

        <a href="#q11-1" class="block">
        <a href="#q11-2" class="block">
нственности НОД</a>
        <a href="#q11-3" class="block">
ПИРАЛЬ)</a>
        <a href="#q11-4" class="block">
вращается!)</a>
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-blue-400">
        12. Взаимно простые мно
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-blue-500">
        <a href="#q12-1" class="block">
        <a href="#q12-2" class="block">
        <a href="#q12-3" class="block">
деление)</a>
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-500">
        13. Корни многочленов.
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-blue-500">
        <a href="#q13-1" class="block">
        <a href="#q13-2" class="block">
        <a href="#q13-3" class="block">
    </div>
</details>
```

```

        </a>
      </summary>
      <div class="pl-6 mt-2 space-y-2">
        <a href="#q16-1" class="block">
        <a href="#q16-2" class="block">
        <a href="#q16-3" class="block">
      </div>
    </details>

    <div class="mt-4 mb-2 text-xs font-l
    <a href="#question-17" class="block
-500 pl-3 text-slate-400">
      <span class="font-bold text-tea
    </a>
    <a href="#question-18" class="block
-500 pl-3 text-slate-400">
      <span class="font-bold text-blue
    </a>
    <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
      <span class="font-bold text-eme
    </a>
    <a href="#question-20" class="block
-500 pl-3 text-slate-400">
      <span class="font-bold text-ros
    </a>
    <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
      <span class="font-bold text-ind
    </a>
    <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
      <span class="font-bold text-fuc
    </a>

    <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m

```

```

        // Highlight active
        document.querySelector(
            a.style.fontWeight
            if (a.href.endsWith(
                a.style.fontSize
            )
        });

        // Open the parent
        const corresponding
        if (correspondingLink) {
            const details =
            if (details &&
                details.open
            document.querySelector(
                if (other
                other
            )
        });
    }
}

});
}, {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: [0, 0.5]
});

sections.forEach(section => {
    observer.observe(section);
});
});

function fallbackModal(contentRaw,
    const modal = document.createElement('div');
    modal.style.position = 'fixed';
    modal.style.top = '0';

```

```
const btnContainer = document.c
btnContainer.style.display = 'f
btnContainer.style.gap = '10px'
btnContainer.style.marginTop =
```

```
const copyBtn = document.create
copyBtn.innerText = 'Копировать'
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5
copyBtn.style.cursor = 'pointer'
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.wri
        copyBtn.innerText = 'Ск
    } else {
        textarea.select();
        document.execCommand('c
        copyBtn.innerText = 'Ск
    }
    setTimeout(() => copyBtn.inn
};
```

```
const closeBtn = document.creat
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px'
closeBtn.style.backgroundColor :
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '!
closeBtn.style.cursor = 'pointe
closeBtn.onclick = () => modal.
```

```
box.appendChild(header);
box.appendChild(subHeader);
```

```

        const link = document.createElement('a');
        link.href = url;
        link.download = 'vyisshaya_algebra.pdf';
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
    }

    function setTheme(theme) {

        const body = document.body;
        body.classList.remove('bg-slate-500');

        document.getElementById('theme-dark').checked = theme === 'dark';
        document.getElementById('theme-light').checked = theme === 'light';
        document.getElementById('theme-auto').checked = theme === 'auto';

        let oldStyle = document.getElementById('theme-style');
        if (oldStyle) oldStyle.remove();
        const style = document.createElement('style');
        style.id = 'theme-style';

        if (theme === 'dark') {
            body.classList.add('bg-slate-500');
            document.getElementById('theme-dark').checked = true;
        } else if (theme === 'light') {
            body.classList.add('bg-white');
            document.getElementById('theme-light').checked = true;
            style.innerHTML = `
                body.bg-white section h1 { color: #000000; }
                body.bg-white main { color: #000000; }
                body.bg-white main p { color: #000000; }
                /* Universal text color */
                body.bg-white .text-slate-500 { color: #000000; }
                body.bg-white .text-slate-400 { color: #000000; }
                body.bg-white .text-slate-300 { color: #000000; }
            `;
        } else {
            body.classList.add('bg-white');
            document.getElementById('theme-auto').checked = true;
        }
    }

```

```

        body.bg-white .text-cyan
        body.bg-white .text-amber
        body.bg-white .text-fuchsia
        body.bg-white .text-lime
        body.bg-white .text-blue
        body.bg-white .text-green
        body.bg-white .text-indigo
    `;
  } else if (theme === 'terminal') {
    body.classList.add('bg-black')
    document.getElementById('theme')
      .style.innerHTML = `
        body.bg-black .bg-slate-950 {
          background-color: transparent;
        }
        body.bg-black section h1 {
          color: #4ade80 !important;
          border-color: #22c55e !important;
          font-family: monospace;
        }
        body.bg-black svg path {
          stroke: #22c55e !important;
          fill: transparent !important;
        }
      `;
  }
}

document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo

```

```

        </div>
    </aside>

    <!-- Основной контент -->
    <main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
    -base min-w-0">
        <header class="text-center mb-12 border-b border-white" >
            <h1 class="text-4xl md:text-5xl font-bl
            4">
                ВЫСШАЯ АЛГЕБРА
            </h1>
            <p class="text-slate-400 italic text-lg">
                Высшая алгебра: теория и практика
            </p>

            <!-- Print Table of Contents -->
            <div class="hidden print:block mt-8 text-center" >
                <p>Print Table of Contents</p>
                <div class="font-bold text-xl mb-4">
                    <h2>Table of Contents</h2>
                    <ul class="text-base space-y-2">
                        <li><a href="#question-1">1. Алгебра</a>
                        <li><a href="#question-2">2. Коммутативная алгебра</a>
                        <li><a href="#question-3">3. Топология</a>
                        <li><a href="#question-4">4. Статистика</a>
                        <li><a href="#question-5">5. Группы</a>
                        <li><a href="#question-6">6. Декартовы координаты</a>
                        <li><a href="#question-7">7. Алгебраические структуры</a>
                        <li><a href="#question-8">8. Кольца</a>
                        <li><a href="#question-9">9. Координаты</a>
                        <li><a href="#question-10">10. Матрицы</a>
                        <li><a href="#question-11">11. Линейные пространства</a>
                        <li><a href="#question-12">12. Преобразования</a>
                        <li><a href="#question-13">13. Линейные операторы</a>
                        <li><a href="#question-14">14. Линейные функции</a>
                        <li><a href="#question-15">15. Линейные уравнения</a>
                        <li><a href="#question-17">17. Линейные операторы</a>
                        <li><a href="#question-18">18. Линейные функции</a>
                        <li><a href="#question-19">19. Линейные уравнения</a>
                        <li><a href="#question-20">20. Линейные операторы</a>
                        <li><a href="#question-21">21. Линейные функции</a>
                    </ul>
                </div>
            </div>
        </main>
    </div>

```

```

-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
  </a>
  <a href="#question-10" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
  </a>
  <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-5
    <div class="font-bold text-slate
  </a>

  <!-- Row 3: НОД и НОК / Алгоритм Ев
  <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
  </a>
  <a href="#question-12" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
  </a>
  <!-- Empty 3rd row cell -->
  <div class="hidden md:block"></div>

  <!-- Row 4: Факторизация / Структур
  <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
    <div class="text-xs text-yellow
    <div class="font-bold text-slate
  </a>
  <a href="#question-11" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange

```



```

        <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
        <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

        <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
        <div class="text-xs text-teal-500
        <div class="font-bold text-slate-700
        </a>
        <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
        <div class="text-xs text-blue-500
        <div class="font-bold text-slate-700
        </a>
        <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
        <div class="text-xs text-emerald-500
        <div class="font-bold text-slate-700
        </a>

        <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
        <div class="text-xs text-rose-500
        <div class="font-bold text-slate-700
        </a>
        <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
        <div class="text-xs text-indigo-500
        <div class="font-bold text-slate-700
        </a>
        <a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
        <div class="text-xs text-fuchsia-500
        <div class="font-bold text-slate-700
        </a>

        <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ

```

```

        <li><span class="text-w
k$).</li>
        <li><span class="text-w
        <li><span class="text-w
ово.</li>
        </ul>
    </div>

    <!-- От противного -->
    <div class="bg-slate-800/80 p-6
        <h3 class="text-xl text-fuch
        <p class="text-sm text-slat
        <ul class="list-disc pl-5 sp
            <li><span class="text-w
i>
            <li><span class="text-w
т.д.).</li>
            <li><span class="text-w
что так и задумано: "Ой, противоречие, теор
        </ul>
    </div>

    <!-- Единственность -->
    <div class="bg-slate-800/80 p-6
        <h3 class="text-xl text-eme
        <p class="text-sm text-slat
        <ul class="list-disc pl-5 sp
            <li><span class="text-w
, r_2$).</li>
            <li><span class="text-w
            <li><span class="text-w
счет ограничений (размер остатка) докажи, ч
        </ul>
    </div>

    <!-- Индукция -->
    <div class="bg-slate-800/80 p-6
        <h3 class="text-xl text-yel

```

```
        <h4 class="text-red-400">
        <p class="text-sm text-slate-400">
        <p class="text-xs font-slate-400">
$с$. Пока получается 0 – корень жив. Как то
корня минус один.</p>
    </div>
```

```
    <div class="bg-slate-800 p-4">
        <h4 class="text-red-400">
        <p class="text-sm text-slate-400">
        <p class="text-xs font-slate-400">
ициентов (они делятся на $p$), кроме старше
зложить.</p>
    </div>
```

```
    <div class="bg-slate-800 p-4">
        <h4 class="text-red-400">
        <p class="text-sm text-slate-400">
        <p class="text-xs font-slate-400">
. Чаще всего после этого Фейс-контроль $p$ .
    </div>
</div>
```

```
    <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
    <p class="text-sm text-slate-400">
тношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
    <ul class="space-y-4 pb-10">
        <li class="bg-slate-800/50 p-4">
            <span class="text-2xl">
            <div>
                <h4 class="text-cyan-400">
                <p class="text-sm text-slate-400">
                <code class="text-x">
            </div>
```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 81/92: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

ФАЙЛ 82/92: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили зашиты внутрь, интерпретатор
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а комментарии
 * которые в приложении всплывают по наведению, собираются в блок
 * в конце документа.
```

```
-----  
.export-term .formal { color: #94a3b8; font-size: .8rem;  
.export-term .cx { color: #6ee7b7; font-size: .78rem; f  
.export-foot { margin-top: 3rem; border-top: 1px solid  
@media print {  
  body { background: #fff; color: #0f172a; }  
  .export-note, details { break-inside: avoid; }  
  details { display: block; }  
  details > div, details > p { display: block !important  
  pre { white-space: pre-wrap; }  
}  
`;
```

```
const escapeHtml = (s: string) =>  
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на  
 * Возвращает готовый HTML главы и список найденных тер  
 */
```

```
function prepareBody(html: string): { body: string; term  
  const host = document.createElement("div");  
  host.innerHTML = html;  
  
  try {  
    annotateTerms(host);  
  } catch {  
    /* если браузер не осилил регулярку – выгружаем тек  
  }
```

```
const ids: string[] = [];  
host.querySelectorAll<HTMLElement>(".term-hint").forE  
  const id = el.dataset.termId;  
  if (!id) return;  
  if (!ids.includes(id)) ids.push(id);  
  const link = document.createElement("a");  
  link.className = "term-hint";  
  link.href = `#term-${id}`;  
  link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
```

```
-----
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
  <header class="export-head">
    <div style="font-size:.7rem;text-transform:uppercas
    <h1 style="color:#fff;font-size:1.6rem;font-weight:
    <div style="font-size:.72rem;color:#475569;margin-t
  </header>

  <div class="export-note">
    Это автономная копия: стили внутри файла, интернет
    Интерактивные тренажёры и анимации в статичный HTML
    они живут в онлайн-версии: <a href="${escapeHtml(op
  </div>

  <article class="prose prose-invert max-w-none">
${opts.body}
  </article>

${opts.gloss}

  <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```

```
function triggerDownload(html: string, filename: string) {
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
```

```

    chapters.forEach((c) => {
      const { body, termIds } = prepareBody(c.content);
      termIds.forEach((t) => {
        if (!allTerms.includes(t)) allTerms.push(t);
      });
      bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="
<p style="text-align:right;font-size:.72rem;margin:0 0 10px 0;">
      `);

      const body = `<nav id="toc" style="background:#0f172a;
        color:white;padding:10px 10px 0 10px;">
        <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0 0 10px 0;">
        <ol style="margin:0;padding-left:1.1rem;font-size:.85rem;">
</nav>
      ${bodies.join("\n")}`;

      const html = wrap({
        title: "Универсальное пособие по алгоритмам",
        subtitle: `Все темы, ${chapters.length} шт.`,
        css,
        body,
        gloss: glossarySection(allTerms),
        origin: window.location.origin,
      });
      triggerDownload(html, "posobie-vse-temy.html");
    }
  }

```

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

ФАЙЛ 83/92: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМЕР: 1.5 КБ

```

    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\\n`);
  const dump = (title, list) => {
    console.log(`${title} (${list.length}):`);
    list.forEach((r) => console.log(`  • ${r.title} [${r.num || "-"}]`));
    console.log();
  };
}
```



```

-----
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
    .replace(/<[^>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlight
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
  );

```

ФАЙЛ 86/92: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [A4.width, A4.height],
    margin: 0,
```

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];
```

```

    walk(ROOT);
}

return files
  .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
  .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
  .filter((rel) => CODE_EXT.has(path.extname(rel)))
  .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
  .sort((a, b) => {
    const ia = ORDER.indexOf(categoryOf(a));
    const ib = ORDER.indexOf(categoryOf(b));
    return ia !== ib ? ia - ib : a.localeCompare(b);
  });
}

/** Достаём заголовки глав из данных пособия (без запуска)
function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("src/"))) {
    const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
  return chapters.sort((a, b) => {
    const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
    const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
    return na - nb;
  });
}

function main() {

```

```

        push(`  # ${cat}`);
    }
    push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`);
});
push();
push();

current = null;
files.forEach((rel, i) => {
    const cat = categoryOf(rel);
    if (cat !== current) {
        current = cat;
        push(HR);
        push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
        push(HR);
        push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel));
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);

```

```

density: Number(process.env.EXPORT_DENSITY ?? 150),
/** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
monochrome: process.env.EXPORT_GRAYSCALE !== "1",
size: "A4",
font: "DejaVu-Sans-Mono",
fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
pointsize: 8,
/** Максимум символов в строке (далее – жёсткий пере
cols: 138,
/** Строк содержимого на странице (+2 служебные: шапка
rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore"
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get

```

```

        if (fs.existsSync(p)) return p;
    }
    return null;
}
const F_REG = findFont("DejaVuSans.ttf");
const F_BOLD = findFont("DejaVuSans-Bold.ttf");
const F_MONO = findFont("DejaVuSansMono.ttf");

// ----- HTML → блоки текста -----

type Block =
    | { kind: "h"; level: 2 | 3 | 4; text: string }
    | { kind: "p"; text: string }
    | { kind: "li"; text: string }
    | { kind: "code"; text: string }
    | { kind: "table"; rows: string[] };

const ENTITIES: Record<string, string> = {
    "&nbsp;": " ", "&amp;": "&", "&lt;": "<", "&gt;": ">",
    "&#39;": "'", "&rarr;": "→", "&larr;": "←", "&le;": "≤",
    "&ne;": "≠", "&times;": "×", "&minus;": "−", "&mdash;": "—",
    "&hellip;": "...", "&infin;": "∞", "&cup;": "∪", "&cap;": "∩",
    "&sub;": "⊂", "&isin;": "∈", "&notin;": "∉", "&pi;": "π",
    "&beta;": "β", "&gamma;": "γ", "&Delta;": "Δ", "&delt;": "δ",
    "&log;": "log", "&deg;": "°", "&plusmn;": "±", "&midd;": "⋮",
};

function decodeEntities(s: string): string {
    return s
        .replace(/&#x([0-9a-fA-F]+)/g, (_, h) => String.fromCharCode(parseInt(h, 16)))
        .replace(/&#(\d+)/g, (_, d) => String.fromCharCode(parseInt(d, 10)))
        .replace(/&[a-zA-Z#0-9]+;/g, (e) => ENTITIES[e] ?? e);
}

/** Убрать инлайн-теги, оставив чистый текст с пробелами */
function stripTags(html: string): string {
    return decodeEntities(
        html
    );
}

```

```

const lower = html.toLowerCase();
const nexts = skipTo.map((t) => lower.indexOf(t, i))
const liRe = /<li(\s[^\>]*)?>/gi;
liRe.lastIndex = i;
const liM = liRe.exec(lower);
if (liM) nexts.push(liM.index);
const trRe = /<tr(\s[^\>]*)?>/gi;
trRe.lastIndex = i;
const trM = trRe.exec(lower);
if (trM) nexts.push(trM.index);
const pRe = /<p(\s[^\>]*)?>/gi;
pRe.lastIndex = i;
const pM = pRe.exec(lower);
if (pM) nexts.push(pM.index);
if (!nexts.length) break;
const at = Math.min(...nexts);

if (lower.startsWith("<pre", at)) {
  const openEnd = html.indexOf(">", at) + 1;
  const { end, inner } = findCloseAny(html, openEnd);
  const code = decodeEntities(inner.replace(/<[^\>]+/g, "&#x2D;"));
  if (code.trim()) blocks.push({ kind: "code", text: code });
  i = end;
  continue;
}
if (/^<h[234]/.test(lower.slice(at, at + 4))) {
  const tag = lower[at + 2];
  const openEnd = html.indexOf(">", at) + 1;
  const close = html.toLowerCase().indexOf(`</h${tag}`, openEnd);
  const text = stripTags(html.slice(openEnd, close));
  if (text) blocks.push({ kind: "h", level: Number(tag), text });
  i = close < 0 ? html.length : close + 5;
  continue;
}
if (lower.startsWith("<summary", at)) {
  const openEnd = html.indexOf(">", at) + 1;
  const { end, inner } = findCloseAny(html, openEnd);
  const text = stripTags(inner);

```


// _____ SSR визуализаций → SVG _____

```
interface VizArt {
    svgs: string[];
    images: string[]; // dataURL
    grids: string[][]; // текстовые матрицы (div-таблицы
    error?: string;
}
```

```
/** Отрендерить визуализатор в статику и вытащить картинку
export function renderVizArt(chapterId: string): VizArt {
    const entry = getViz(chapterId);
    if (!entry) return null;
    try {
        const html = renderToStaticMarkup(
            createElement(VizChapterContext.Provider, { value: entry },
                <div></div>
            );
        const svgs: string[] = [];
        const images: string[] = [];
        let idx = 0;
        while (true) {
            const at = html.toLowerCase().indexOf("<svg", idx);
            if (at < 0) break;
            const openEnd = html.indexOf(">", at) + 1;
            // вложенные <svg> не встречаются, но ищем закрывающий
            let depth = 1;
            let pos = openEnd;
            while (depth > 0) {
                const no = html.toLowerCase().indexOf("<svg", pos);
                const nc = html.toLowerCase().indexOf("</svg>", pos);
                if (nc < 0) { pos = html.length; break; }
                if (no >= 0 && no < nc) { depth++; pos = html.indexOf(">", no) + 1; }
                else { depth--; pos = nc + 6; }
            }
            let svg = html.slice(at, pos);
            if (!/<?xmlns=/>.test(svg)) svg = svg.replace("<svg", "<img");
            // выкинуть элементы с tailwind-классами без инлайнов
            if (/<(?!<|>)tailwind/.test(svg)) continue;
            svgs.push(svg);
            images.push(html.slice(openEnd, pos));
            idx = pos;
        }
        return { svgs, images };
    } catch {
        console.error("renderVizArt failed");
        return null;
    }
}
```

```

    if (cells.length >= 2 && cells.length <= 24) rows.push(cells);
  }
  // подряд идущие строки одинаковой длины = матрица; о
  let cur: string[] = [];
  const flush = () => {
    if (cur.length >= 2) grids.push(cur);
    else if (cur.length === 1 && cur[0].split("\u2009").length > 1)
      cur = [];
  };
  for (const r of rows) {
    if (cur.length && r.length !== cur[cur.length - 1].length)
      cur.push(r);
  }
  flush();
  return grids.slice(0, 6);
}

```

// ————— вёрстка PDF —————

```

const A4: [number, number] = [595.28, 841.89];
const M = { top: 46, bottom: 48, left: 50, right: 50 };
const CONTENT_W = A4[0] - M.left - M.right;

```

```

interface Stats {
  chapterStart: Map<string, number>;
  total: number;
}

```

```

class Doc {
  doc: PDFDocument;
  pages = 0;
  chapterStart = new Map<string, number>();
  constructor(stream: fs.WriteStream) {
    this.doc = new PDFDocument({ size: "A4", margins: {
      top: 46, bottom: 48, left: 50, right: 50
    });
    this.doc.pipe(stream);
    this.doc.on("pageAdded", () => this.pages++);
    this.setupFonts();
    this.bodyFont();
  }
}

```

```

d.ensureSpace(lead * 2);
d.doc.fontSize(size)
  .fillColor(opts.color ?? "#1f2937")
  .text(text, M.left + (opts.indent ?? 0), d.y, {
    width: CONTENT_W - (opts.indent ?? 0),
    lineGap: lead - size,
    align: "left",
  });
d.doc.moveDown((opts.gap ?? 2) / lead);
}

function drawCodeBlock(d: Doc, code: string) {
  const size = 7.7;
  const lead = size * 1.5;
  const lines = code.split("\n");
  const height = lines.length * lead + 14;
  d.ensureSpace(Math.min(height, A4[1] - M.top - M.bottom));
  // если блок целиком не влезает – разрыв страницы перед
  if (d.y + height > d.pageBottom && lines.length > 3) {
    const top = d.y;
    const drawPortion = {from: number, to: number, yStart: number};
    const h = (to - from) * lead + 12;
    d.doc.save().rect(M.left - 4, yStart - 4, CONTENT_W + 8, yStart + h)
      .monoFont().fontSize(size).fillColor("#111827");
    let y = yStart + 2;
    for (let li = from; li < to; li++) {
      d.doc.text(lines[li], M.left, y, { width: CONTENT_W - 8 });
      y += lead;
    }
    return y;
  };
  // постранично
  let from = 0;
  let yTop = top;
  while (from < lines.length) {
    const avail = d.pageBottom - yTop;
    let fit = Math.floor((avail - 8) / lead);
    if (fit <= 0 && from === 0 && lines.length > 1) { d.doc.moveDown(1);

```

```

    d.doc.moveDown(2);
}

function drawSvg(d: Doc, svg: string) {
    const wAttr = Number(/width="([\d.]+)"/.exec(svg)?.[1]);
    const hAttr = Number(/height="([\d.]+)"/.exec(svg)?.[1]);
    const vb = /viewBox="([\d.\-\\s]+)"/.exec(svg)?.[1]??.trim();
    const natW = wAttr || (vb ? vb[2] : 480);
    const natH = hAttr || (vb ? vb[3] : 320);
    const maxW = CONTENT_W - 10;
    const maxH = 330;
    let w = Math.min(natW, maxW);
    let h = (natH / natW) * w;
    if (h > maxH) { h = maxH; w = (natW / natH) * h; }
    const avail = d.pageBottom - d.y;
    if (h + 20 > avail) {
        // пробуем ужать в остаток страницы, иначе — с новой
        if (avail > 170) {
            const k = (avail - 20) / h;
            h = avail - 20;
            w = Math.min(w * k, CONTENT_W - 10);
        } else {
            d.doc.addPage();
        }
    }
    const x = M.left + (CONTENT_W - w) / 2;
    try {
        SVGtoPDF(d.doc, svg, x, d.y + 6, { width: w, height: h });
    } catch {
        /* кривой SVG не роняем весь PDF */
    }
    d.y = d.y + h + 18;
}

```

```

function drawImage(d: Doc, dataUrl: string) {
    const b64 = dataUrl.split(",")[1];
    if (!b64) return;
    const buf = Buffer.from(b64, "base64");

```

```

        doc.end();
        stream.on("close", () => resolve(stats));
        stream.on("error", reject);
    };

    if (!opts.record) {
        // — обложка (первая страница уже создана) —
        d.bodyFont();
        doc.fillColor("#312e81").fontSize(26).font(F_BOLD)
            .text("Алгоритмы и структуры данных", M.left, 1, { width: CONTENT_W });
        doc.moveDown(0.6);
        doc.fontSize(15).fillColor("#4b5563").font(F_REG)
            .text("Универсальное пособие: билеты 1–24", { width: CONTENT_W });
        doc.moveDown(1.2);
        doc.fontSize(10.5).fillColor("#6b7280")
            .text("Текст всех тем + схемы интерактивных визуализаций", {
                width: CONTENT_W, align: "center",
            });
        doc.fontSize(9.5).fillColor("#9ca3af")
            .text(new Date().toLocaleDateString("ru-RU"), { width: CONTENT_W });
    }

    // — содержание —
    if (!opts.record) {
        doc.addPage(); // оглавление на отдельной странице
        const entries = chapters;
        let line = 0;
        const drawToCHeader = () => {
            doc.fontSize(18).font(F_BOLD ? "BodyBold" : "Header")
                .text("Содержание", M.left, M.top, { width: CONTENT_W });
            doc.moveDown(0.8);
            doc.y = doc.y;
            d.y = doc.y;
        };
        drawToCHeader();
        entries.forEach((ch, i) => {
            if (line > 0 && line % FRONT_TOC_LINES === 0) {
                doc.addPage();
            }

```

```

doc.y = M.top;
d.y = M.top;

// заголовок главы
doc.fontSize(16).font(F_BOLD ? "BodyBold" : "Helvetica")
  .text(ch.title, M.left, M.top, { width: CONTENT_WIDTH });
if (ch.description) {
  doc.moveDown(0.2);
  doc.fontSize(9).font(F_REG ? "Body" : "Helvetica")
    .text(ch.description, M.left, d.y, { width: CONTENT_WIDTH });
}
doc.moveDown(0.6);
d.y = doc.y;

// текст главы
const blocks = htmlToBlocks(ch.content);
for (const b of blocks) {
  if (b.kind === "h") {
    d.ensureSpace(40);
    doc.moveDown(0.4);
    const size = b.level === 2 ? 13 : b.level === 3 ? 11 : 10;
    doc.fontSize(size).font(F_BOLD ? "BodyBold" : "Helvetica")
      .text(b.text, M.left, d.y + 4, { width: CONTENT_WIDTH });
    d.y = doc.y + 4;
  } else if (b.kind === "p") {
    drawTextBlock(d, b.text, { gap: 3 });
  } else if (b.kind === "li") {
    drawTextBlock(d, "• " + b.text, { indent: 8, gap: 3 });
  } else if (b.kind === "code") {
    drawCodeBlock(d, b.text);
  } else if (b.kind === "table") {
    drawTableBlock(d, b.rows);
  }
}

// визуализация главы
const vizEntry = VIZ_REGISTRY[ch.id];
if (vizEntry) {
  const art = renderVizArt(ch.id);
  drawTextBlock(d, art, { gap: 3 });
}

```

```

}

async function main() {
  const outputPath = process.argv[2] || path.join(process.cwd(), 'report.json');
  if (!F_REG) throw new Error("Не найден шрифт DejaVuSans");
  const frontPages = 1 + tocPagesNeeded(); // обложка + оглавление

  // проход 1: считаем, на какой странице начинается контент
  const stats = await buildOne({ outputPath: "/dev/null",
    console.log(`Проход 1: ${stats.total} стр. тела`);

  // проход 2: обложка + содержание с номерами страниц
  await buildOne({
    outputPath,
    record: false,
    frontPages,
    pageMap: stats.chapterStart,
  });
  const size = fs.statSync(outputPath).size;
  console.log(`Готово: ${outputPath} (${(size / 1024).toFixed(1)} Кб)`);
  // отчёт для CI-summary: число страниц конечного документа
  const pages = await (async () => {
    const buf = fs.readFileSync(outputPath);
    const m = buf.toString("latin1").match(/\\/Count\\s+(\\d+)/);
    return m ? Number(m[1]) : 0;
  })();
  fs.writeFileSync(
    path.join(process.cwd(), "tmp/guide-pdf-report.json"),
    JSON.stringify({ pages, bytes: size, generatedAt: new Date().toISOString() });
  );
  console.log(`Страниц: ${pages}`);
}

main().catch((e) => {
  console.error(e);
  process.exit(1);
});

```

```

    }
    return parts;
}

function paginate(txt) {
    const source = txt.replace(/\r\n/g, "\n").split("\n");
    const pages = [];
    for (let i = 0; i < source.length; i += PAGE.rows) {
        pages.push(source.slice(i, i + PAGE.rows));
    }
    return pages;
}

async function main() {
    if (!fs.existsSync(TXT_PATH)) {
        throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
    }

    const im = imageMagickCmd();
    const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

    fs.rmSync(PAGES_DIR, { recursive: true, force: true });
    ensureDir(PAGES_DIR);

    const total = pages.length;
    const jobs = pages.map((lines, idx) => {
        const num = String(idx + 1).padStart(4, "0");
        const header = `Универсальное пособие • полный исходный код
        Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
        const body = [header, "-".repeat(PAGE.cols), ...lines];
        const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
        const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
        fs.writeFileSync(txtFile, body + "\n", "utf8");
        return { txtFile, pngFile, num };
    });

    const args = (job) => [

```



```
-----
    console.log(`      страниц: ${total}`);
    console.log(`      выход:   ${path.relative(ROOT, PAGE)}
  }

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 91/92: `.github/workflows/deploy-pages.yml`
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
# (у бота-интеграции нет admin-прав на них – API отвечает)
#
# 1. Settings → Pages → Build and deployment → Source
#    Если там «Deploy from a branch» (legacy) – шаг D
# 2. Settings → Environments → github-pages → Deployment
#    добавьте паттерн «arena/**» и ветку «main» (или
#    иначе деплой из новой рабочей ветки отклоняется)
```

on:

push:

branches:

- main

Позволяет проверить Pages прямо из рабочей ветки

- "arena/**"

workflow_dispatch:

```
deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    # Ранняя и понятная ошибка вместо невнятного «Fail»
    # когда в репозитории Pages не переключён на сбор
    - name: Preflight – Pages собирается через GitHub
      env:
        GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      run: |
        bt=$(gh api "repos/${ github.repository }/pages" |
        echo "Pages build_type: ${bt}")
        if [ "${bt}" != "workflow" ]; then
          echo "::error title=Pages не настроен::Setте этот запуск. См. комментарий в начале de
          exit 1
        fi

    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
```

ФАЙЛ 92/92: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 137 | РАЗМЕР: 5.1 КБ

name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:

#

исходный код → full code all pages.txt (script

```
if: "!contains(github.event.head_commit.message, '[
```

```
steps:
```

- name: Checkout
uses: actions/checkout@v6
with:
 fetch-depth: 0
 persist-credentials: true
- name: Setup Node.js
uses: actions/setup-node@v7
with:
 node-version: "24"
 cache: npm
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommen
 # На Ubuntu политика ImageMagick иногда запре
 # разрешаем то, что нужно пайплайну (text -> p
 for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png

```
with:
  name: full-code-pages-png
  path: tmp/export-pages/*.png
  if-no-files-found: error
  retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.png
    if git diff --cached --quiet; then
      echo "Экспорт не изменился – коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-pages-png"
    git push origin "HEAD:${GITHUB_REF_NAME}"
```